DOCUMENTAÇÃO DE TESTES — CICLO 03

Aplicação dos testes no sistema desenvolvido pelo grupo

Grupo: Produtos	**Sistema:** Módulo Produtos
Integrantes: Vinicius Ribeiro da Silva, Nathiele, Matheus Henrique de Jesus Carvalho da Silva, Gabriel, Vítor, Joabe, Matheus Carvalho de Almeida, Geovana	**Data:** 04/11/2026

Objetivo da atividade

Analisar e testar o sistema de Produtos, verificando se as funcionalidades funcionam corretamente, se os dados são tratados de forma adequada e se o sistema atende às necessidades propostas.

Como este documento foi produzido

Todos os testes foram executados na prática, contra o sistema em funcionamento (Docker Compose). Para cada teste estão registrados os dados utilizados, o resultado esperado, o resultado obtido e as evidências: captura de tela do sistema, print da requisição no Postman, saída da consulta SQL e, nas funcionalidades validadas pela tela, o print do trecho de código-fonte (arquivo e linhas) que executou aquela parte com as linhas principais destacadas em amarelo. Os resultados foram gravados exatamente como ocorreram, inclusive as falhas.

Ambiente e ferramentas

Sistema testado	Módulo Produtos: cadastro de produtos, categorias (com subcategorias), unidades de medida, histórico de preço e autenticação por e-mail/senha.
Arquitetura	Front-end Vue 3 + Vite (Nginx, porta 8080) · API Node.js 22 + Fastify 5 (porta 3000) · PostgreSQL 16 (porta 5433), todos em contêineres Docker.
Execução da interface	Google Chrome, resolução 1280×800, usuário de demonstração
Execução da API	Coleção Postman e requisições auxiliares HTTP diretas nos testes de integração, requisitos e impacto.
Evidência de código	Trechos do código-fonte real do repositório (front-end Vue, API Fastify, schemas Zod compartilhados e migrations SQL), com número das linhas.
Execução do banco	Dentro do contêiner app-product-postgres-1, banco produtos.
Dados de teste	Todos os registros criados pelos testes levam o identificador de execução 024197 no nome/SKU (ex.: NTB-024197), permitindo localizá-los no banco.

Resumo dos resultados

Foram executados **92 testes**: **91 aprovados** e **1 reprovados**. Os reprovados indicam defeitos reais do sistema e estão detalhados na seção “Problemas encontrados”.

Tipo de teste	Executados	Aprovados	Reprovados	% aprovação
1. Casos de teste	5	5	0	100%
2. Testes positivos	8	8	0	100%
3. Testes negativos	11	11	0	100%
4. Testes de API	36	36	0	100%
5. Testes de banco de dados	9	9	0	100%
6. Testes de integração	5	5	0	100%

Tipo de teste	Executados	Aprovados	Reprovados	% aprovação
7. Testes de requisitos	13	12	1	92%
8. Testes de impacto	5	5	0	100%
Total	**92**	**91**	**1**	**99%**

Tipos de teste realizados

1. Casos de teste

O que é: Definem o que precisa ser testado e como o teste será executado. Cada caso apresenta uma funcionalidade, os passos ou dados utilizados e o resultado esperado.

Aplicação neste sistema: Aqui foram definidos e executados os fluxos funcionais centrais do sistema (login, cadastro de usuário, listagem com busca, categorias e unidades de medida), sempre pela interface em um navegador real.

CT-01 — Login com credenciais válidas	Aprovado
Dados / passos utilizados	Tela /login · e-mail demo@sige.com.br · senha demo1234 · clicar em "Entrar"
Resultado esperado	Usuário autenticado e redirecionado para a listagem de produtos (/produtos).
Resultado obtido	Redirecionado para /produtos; listagem de produtos exibida.

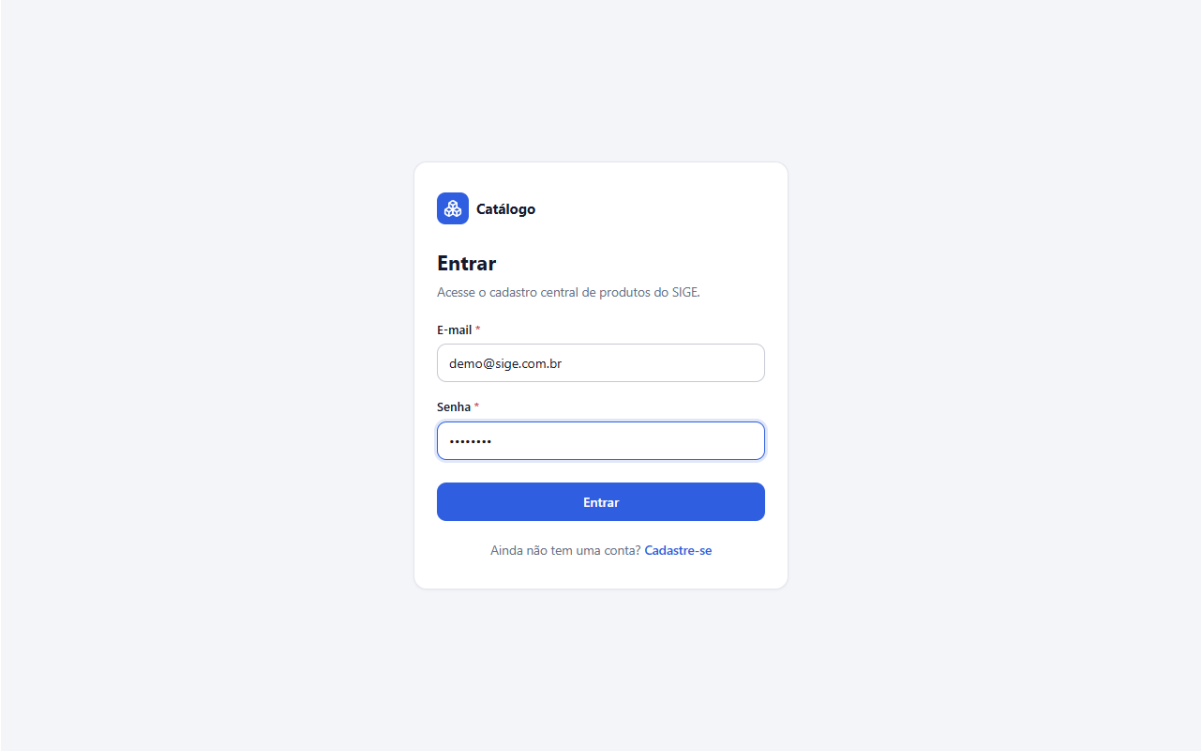


Figura 1 – CT-01: Login preenchido

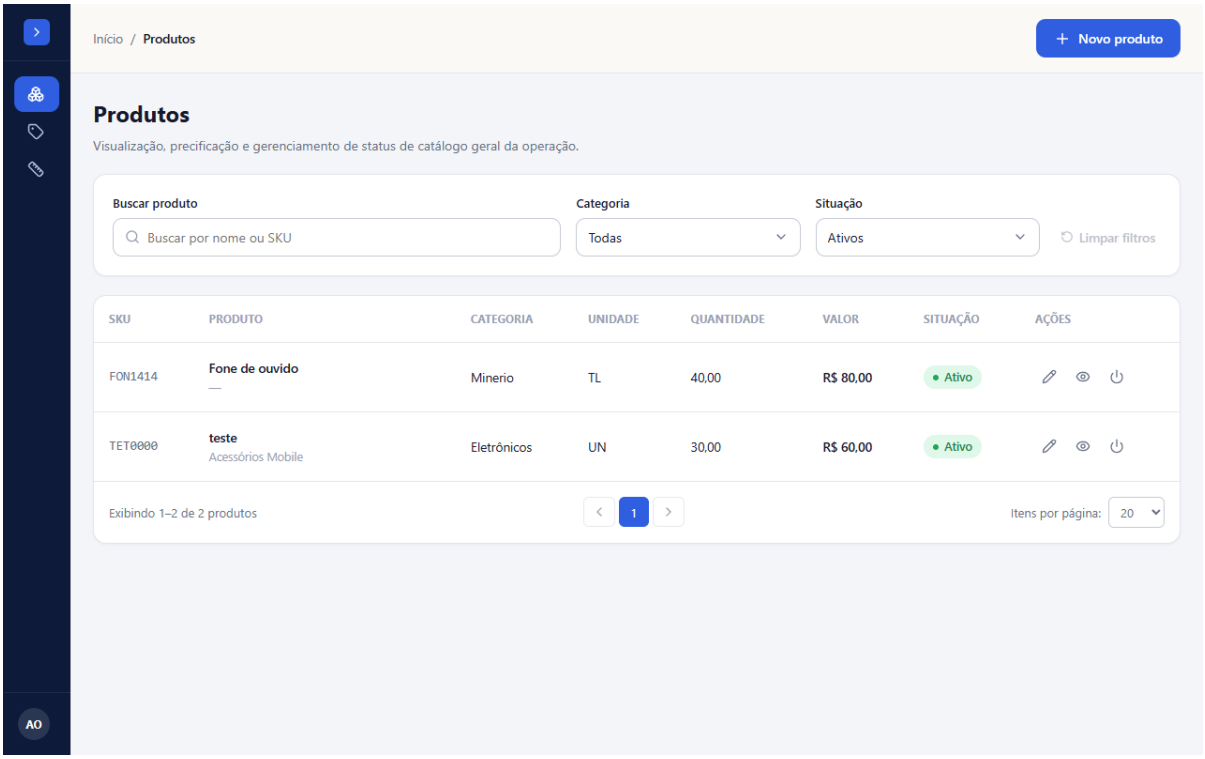


Figura 2 – CT-01: Listagem

CT-01 · Código executado no login

frontend/src/paginas/Login.vue linhas 31–45 — Tela: envia e-mail/senha e grava a sessão

```
31 const aoEnviar = handleSubmit(async (valores) => {
32   erroGeral.value = '';
33   enviando.value = true;
34   try {
35     const usuario = await authApi.login(valores);
36     sessao.definirUsuario(usuario);
37     const destino = route.query.redirecionarPara || '/produtos';
38     router.push(destino);
39   } catch (erro) {
40     // RF30: mensagem genérica — não revela qual campo errou
41     erroGeral.value = erro instanceof ErroApi ? erro.message : 'Não foi possível entrar. Tente novamente.';
42   } finally {
43     enviando.value = false;
44   }
45 });
```

backend/src/servicos/auth.servico.js linhas 16–23 — API: confere o hash bcrypt

```
16 export async function login({ email, senha }) {
17   const usuario = await usuariosRepo.buscarPorEmailComSenha(email);
18   const ok = usuario ? await bcrypt.compare(senha, usuario.senha) : false;
19
20   if (!ok) throw new ErroAplicacao('CREDENCIAIS_INVALIDAS');
21
22   return { id: usuario.id, nome: usuario.nome, email: usuario.email };
23 }
```

Figura 3 – CT-01: Codigo

CT-02 — Cadastro de novo usuário pela tela	**Aprovado**
Dados / passos utilizados	Tela /cadastro · nome "Testador Ciclo 03" · e-mail ciclo03.024197@teste.com · senha senha1234

Resultado esperado	Conta criada, login automático e redirecionamento para /produtos.
Resultado obtido	Usuário criado e autenticado automaticamente; listagem de produtos exibida.

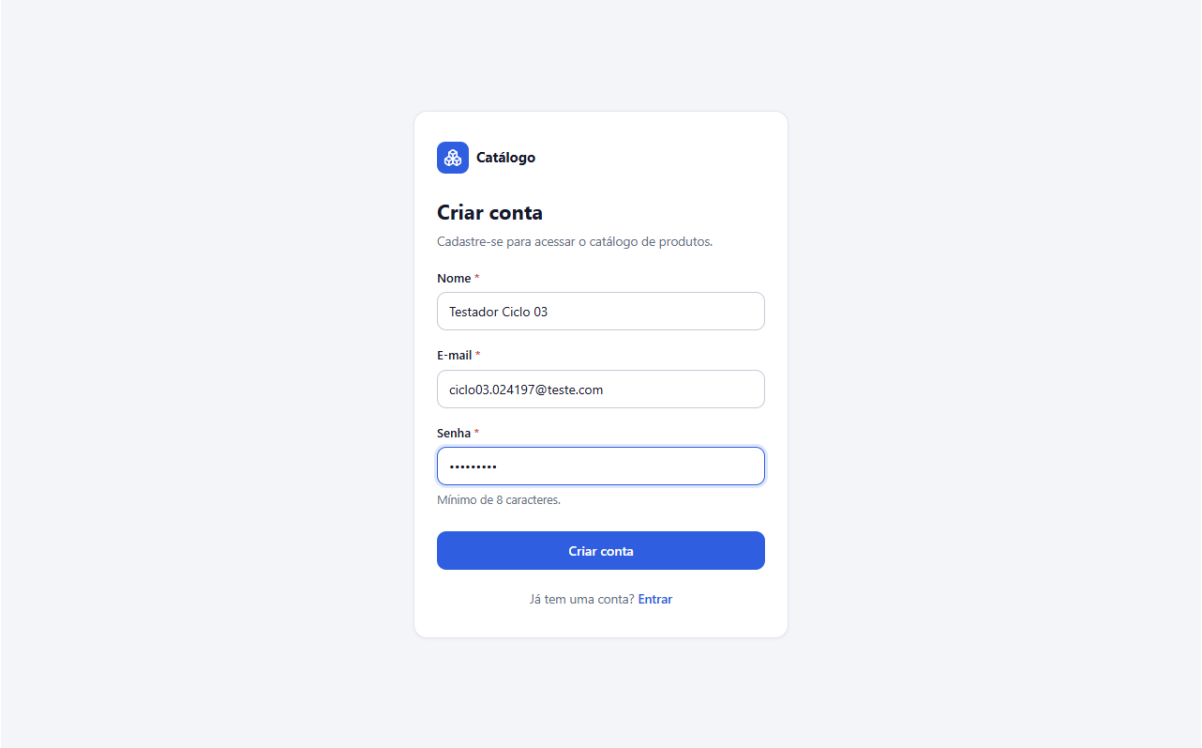


Figura 4 – CT-02: Cadastro usuario

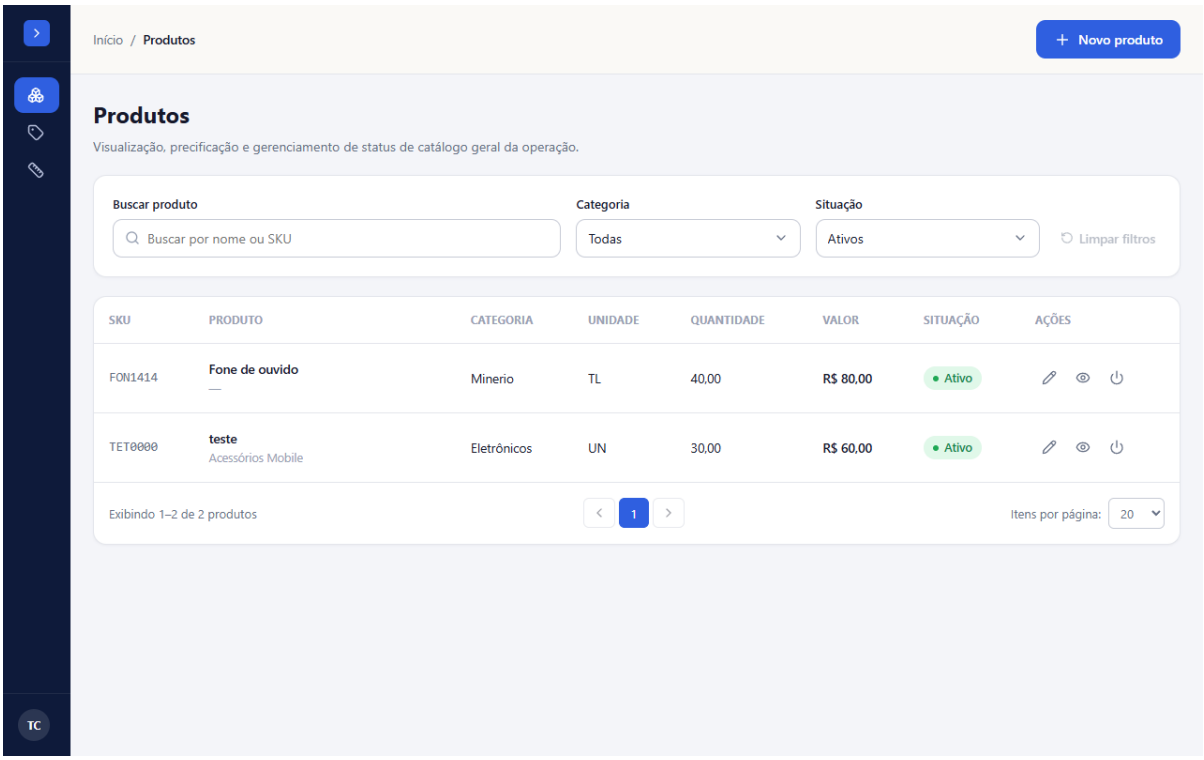


Figura 5 – CT-02: Apos cadastro

CT-02 · Código executado no cadastro de usuário

frontend/src/paginas/CadastroUsuario.vue linhas 29–42 — Tela: cadastra e já faz login

```vue
const aoEnviar = handleSubmit(async (valores) => {
  erroGeral.value = '';
  enviando.value = true;
  try {
    await authApi.cadastrar(valores);
    const usuario = await authApi.login({ email: valores.email, senha: valores.senha });
    sessao.definirUsuario(usuario);
    router.push('/produtos');
  } catch (erro) {
    erroGeral.value = erro instanceof ErroApi ? erro.message : 'Não foi possível concluir o cadastro.';
  } finally {
    enviando.value = false;
  }
});
```

backend/src/servicos/auth.servico.js linhas 7–13 — API: e-mail único + hash da senha

```vue
export async function cadastrar({ nome, email, senha }) {
  const existente = await usuariosRepo.buscarPorEmail(email);
  if (existente) throw new ErroAplicacao('EMAIL_JA_CADASTRADO');

  const hash = await bcrypt.hash(senha, env.BCRYPT_CUSTO);
  return usuariosRepo.inserir({ nome, email, senha: hash }); // RN15: nunca retorna a senha
}
```

Figura 6 – CT-02: Codigo

CT-03 — Listagem de produtos e busca por nome	Aprovado
Dados / passos utilizados	Usuário logado · tela /produtos · digitar "Fone" no campo de busca
Resultado esperado	Tabela filtrada exibindo apenas produtos cujo nome contém "Fone".
Resultado obtido	1 linha(s) retornada(s); todas contêm "Fone".

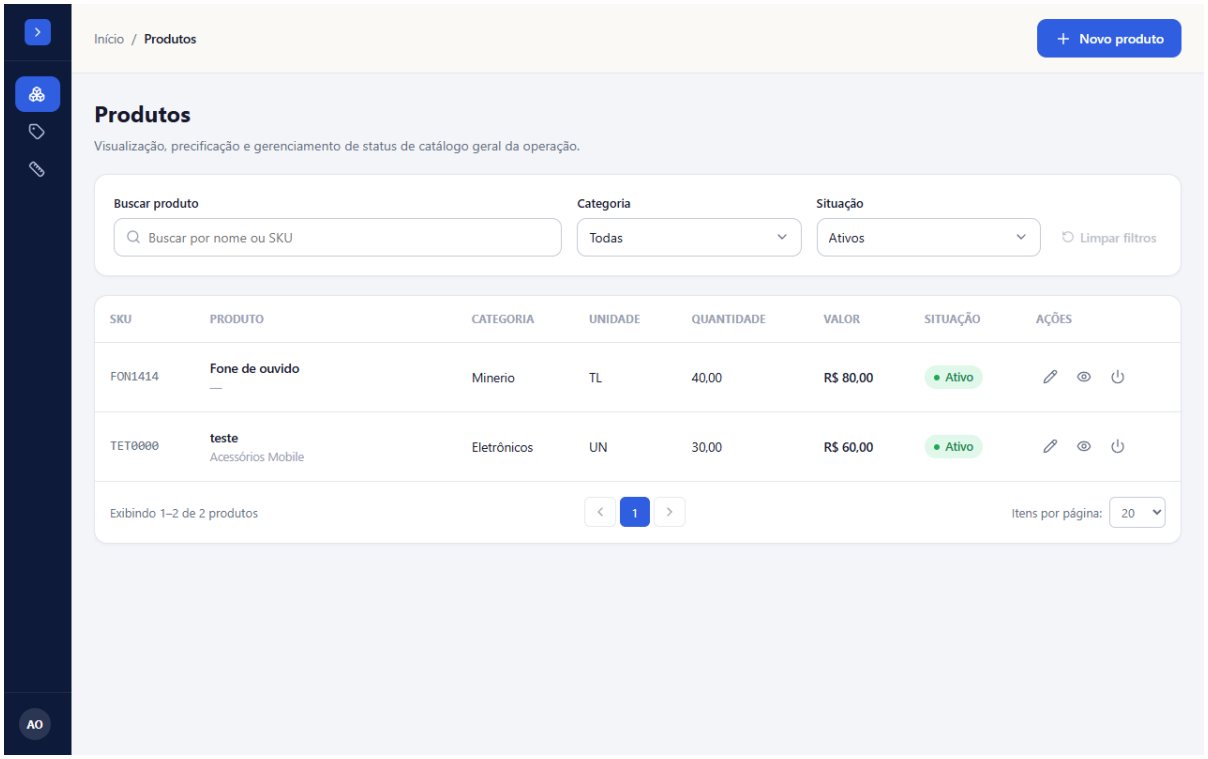


Figura 7 – CT-03: Listagem completa

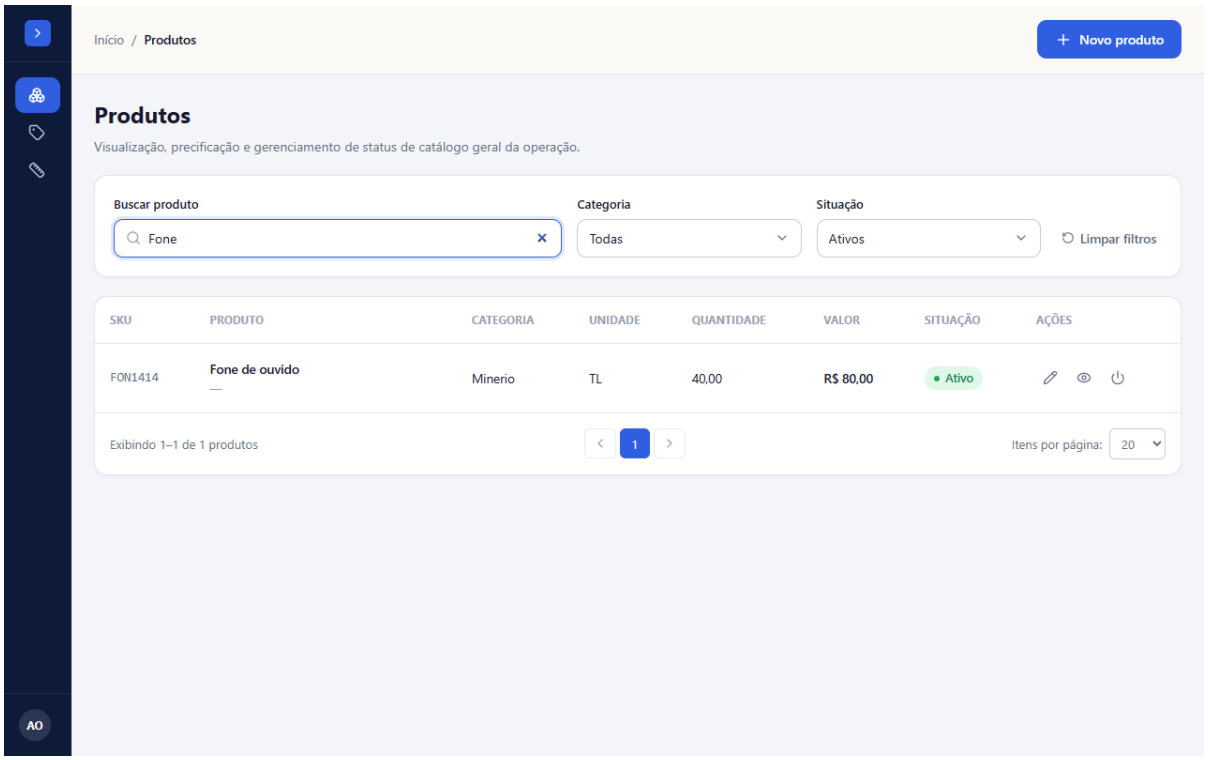


Figura 8 – CT-03: Busca fone

CT-03 · Código executado na pesquisa de produto

frontend/src/paginas/ListagemProdutos.vue linhas 28–35 — Tela: debounce de 350 ms no campo de busca

```
28 const termoBusca = ref(filtros.nome);
29 let debounceId;
30 watch(termoBusca, (valor) => {
31   clearTimeout(debounceId);
32   debounceId = setTimeout(() => {
33     filtros.definirFiltro({ nome: valor, pagina: 1 });
34   }, 350);
35 });
```

frontend/src/paginas/ListagemProdutos.vue linhas 54–66 — Tela: monta o filtro e chama a API

```
54 const filtroAtivo = computed(() => ({
55   nome: filtros.nome || undefined,
56   categoria_id: filtros.categoria_id || undefined,
57   situacao: filtros.situacao,
58   pagina: filtros.pagina,
59   limite: filtros.limite,
60 }));
61
62 const { data, isLoading, isError, refetch } = useQuery({
63   queryKey: ['produtos', filtroAtivo],
64   queryFn: () => produtosApi.listar(filtroAtivo.value),
65   placeholderData: (dadosAnteriores) => dadosAnteriores,
66 });
```

backend/src/repositorios/produtos.repo.js linhas 38–59 — Banco: WHERE com ILIKE

```
38 function montarFiltro({ nome, sku, categoria_id, situacao }) {
39   const condicoes = [];
40   const parametros = [];
41
42   if (nome) {
43     parametros.push(`%${nome}%`);
44     condicoes.push(`p.nome ILIKE $${parametros.length}`);
45   }
46   if (sku) {
47     parametros.push(`%${sku}%`);
48     condicoes.push(`p.sku ILIKE $${parametros.length}`);
49   }
50   if (categoria_id) {
51     parametros.push(categoria_id);
52     condicoes.push(`p.categoria_id = $${parametros.length}`);
53   }
54   if (situacao === 'ativo') condicoes.push('p.ativo = TRUE');
55   if (situacao === 'inativo') condicoes.push('p.ativo = FALSE');
56
57   const where = condicoes.length ? `WHERE ${condicoes.join(' AND ')}` : '';
58   return { where, parametros };
59 }
```

Figura 9 – CT-03: Codigo

CT-04 — Consulta de categorias e subcategorias	**Aprovado**
Dados / passos utilizados	Usuário logado · menu Categorias (/categorias) · expandir "Eletrônicos"
Resultado esperado	Categorias principais listadas e subcategorias visíveis ao expandir o card.
Resultado obtido	Card "Eletrônicos" expandido exibindo as subcategorias "Acessórios Mobile" e "Áudio e Som".

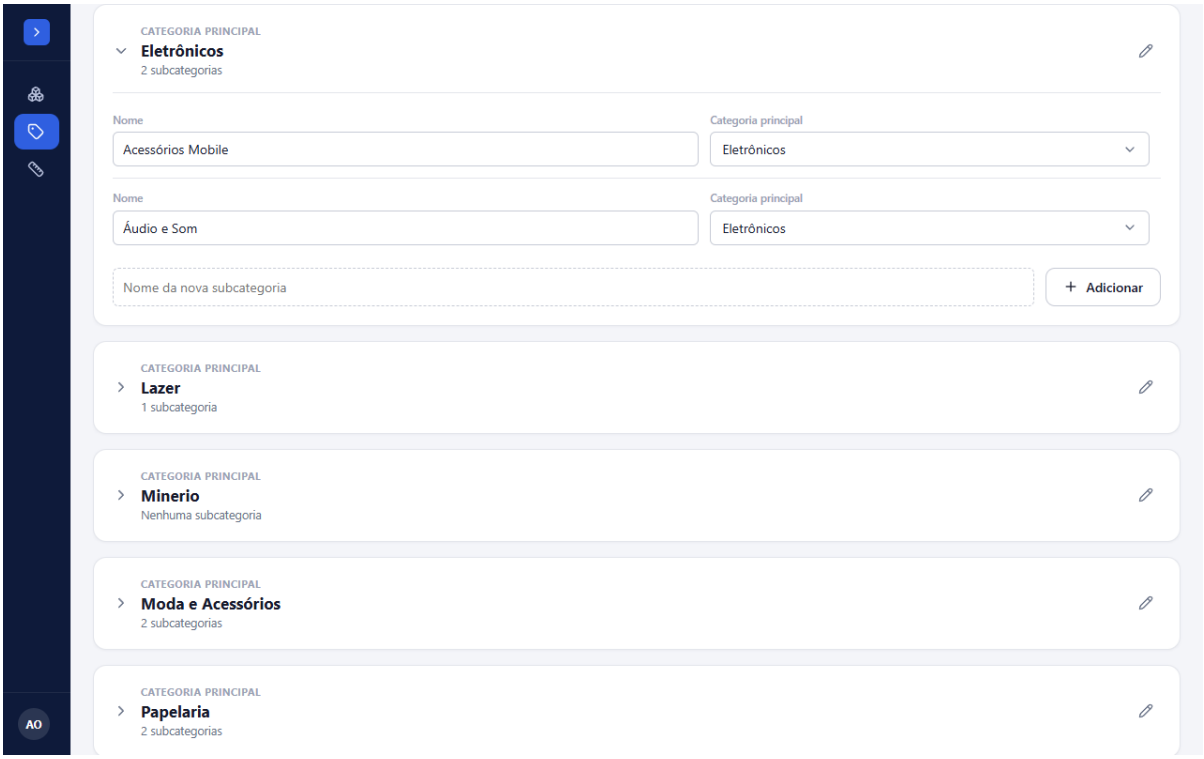


Figura 10 – CT-04: Categorias

CT-04 · Código executado na consulta de categorias

frontend/src/paginas/Categorias.vue linhas 25–28 — Tela: busca a árvore

```
25 const { data: categorias, isLoading, isError, refetch } = useQuery({
26   queryKey: ['categorias', 'todos-admin'],
27   queryFn: () => categoriasApi.listar({ situacao: 'todos' }),
28 });
```

backend/src/servicos/categorias.servico.js linhas 13–27 — API: agrupa subcategorias na categoria pai

```
13 function montarArvore(categorias) {
14   const porId = new Map(categorias.map((c) => [c.id, { ...c, subcategorias: [] }]));
15   const raizes = [];
16
17   for (const categoria of porId.values()) {
18     if (categoria.categoria_pai_id) {
19       const pai = porId.get(categoria.categoria_pai_id);
20       if (pai) pai.subcategorias.push(categoria);
21       else raizes.push(categoria);
22     } else {
23       raizes.push(categoria);
24     }
25   }
26   return raizes;
27 }
```

Figura 11 – CT-04: Codigo

CT-05 — Consulta de unidades de medida	**Aprovado**
Dados / passos utilizados	Usuário logado · menu Unidades de medida (/unidades-medida)
Resultado esperado	Unidades de medida cadastradas exibidas em lista (ex.: Unidade/UN).
Resultado obtido	Tela exibe as unidades cadastradas, incluindo "Unidade (UN)".

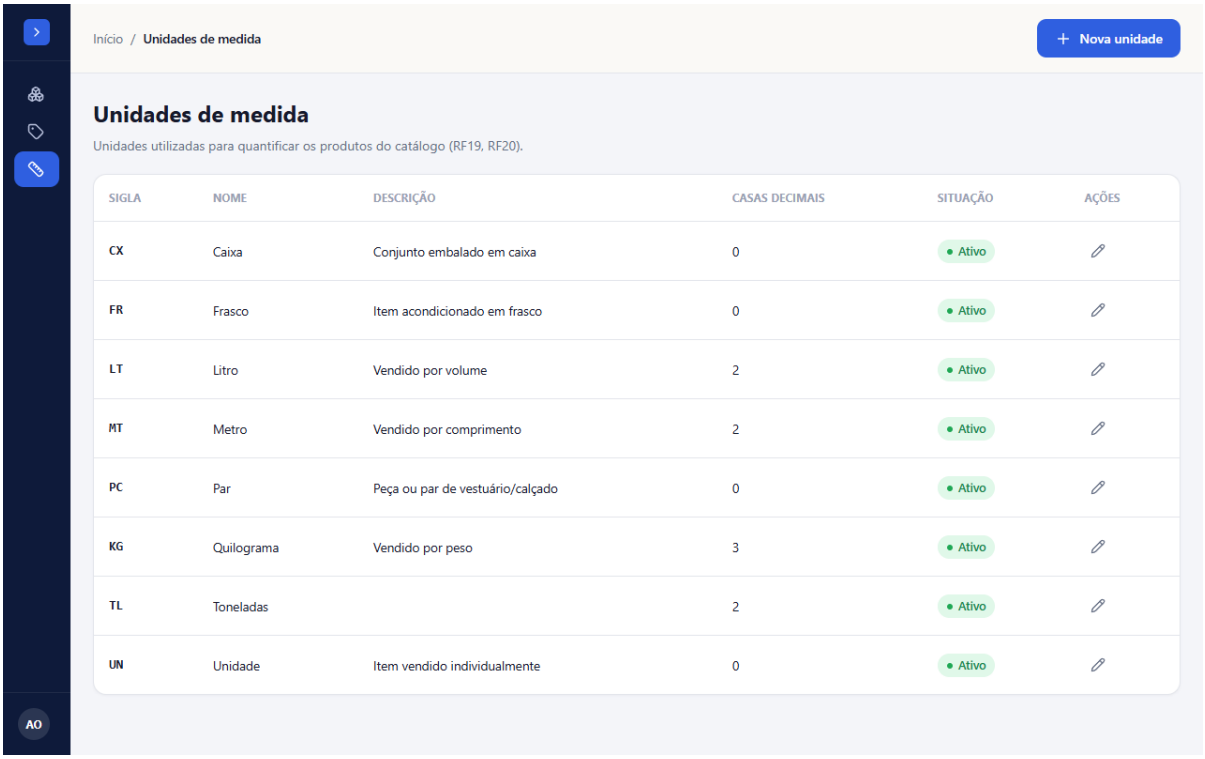


Figura 12 – CT-05: Unidades

CT-05 · Código executado na consulta de unidades

frontend/src/paginas/UnidadesMedida.vue linhas 23–26

```
23 const { data: unidades, isLoading, isError, refetch } = useQuery({
24   queryKey: ['unidades-medida', 'todos-admin'],
25   queryFn: () => unidadesApi.listar({ situacao: 'todos' }),
26 });
```

backend/src/servicos/unidades.servico.js linhas 4–6

```
4 export async function listar({ situacao }) {
5   return unidadesRepo.listar({ situacao });
6 }
```

Figura 13 – CT-05: Codigo

2. Testes positivos

O que é: Verificam se o sistema funciona corretamente quando recebe dados válidos e quando o usuário realiza as operações da maneira esperada.

Aplicação neste sistema: Foram cadastrados, editados, inativados e reativados produtos, além de categorias, subcategorias e unidades de medida, conferindo o resultado na tela.

TP-01 — Cadastrar produto com todos os campos válidos	**Aprovado**
Dados / passos utilizados	Tela /produtos/novo · nome "Notebook Ciclo03 024197" · SKU NTB-024197 · EAN-13 7890241971114 · categoria Acessórios Mobile · unidade Unidade (UN) · custo 2500 · venda 3499,90 · estoque 15
Resultado esperado	Produto salvo, página de detalhe exibida com confirmação e produto presente na listagem.
Resultado obtido	Produto criado com ID 13; detalhe exibido e produto localizado na listagem pelo SKU.

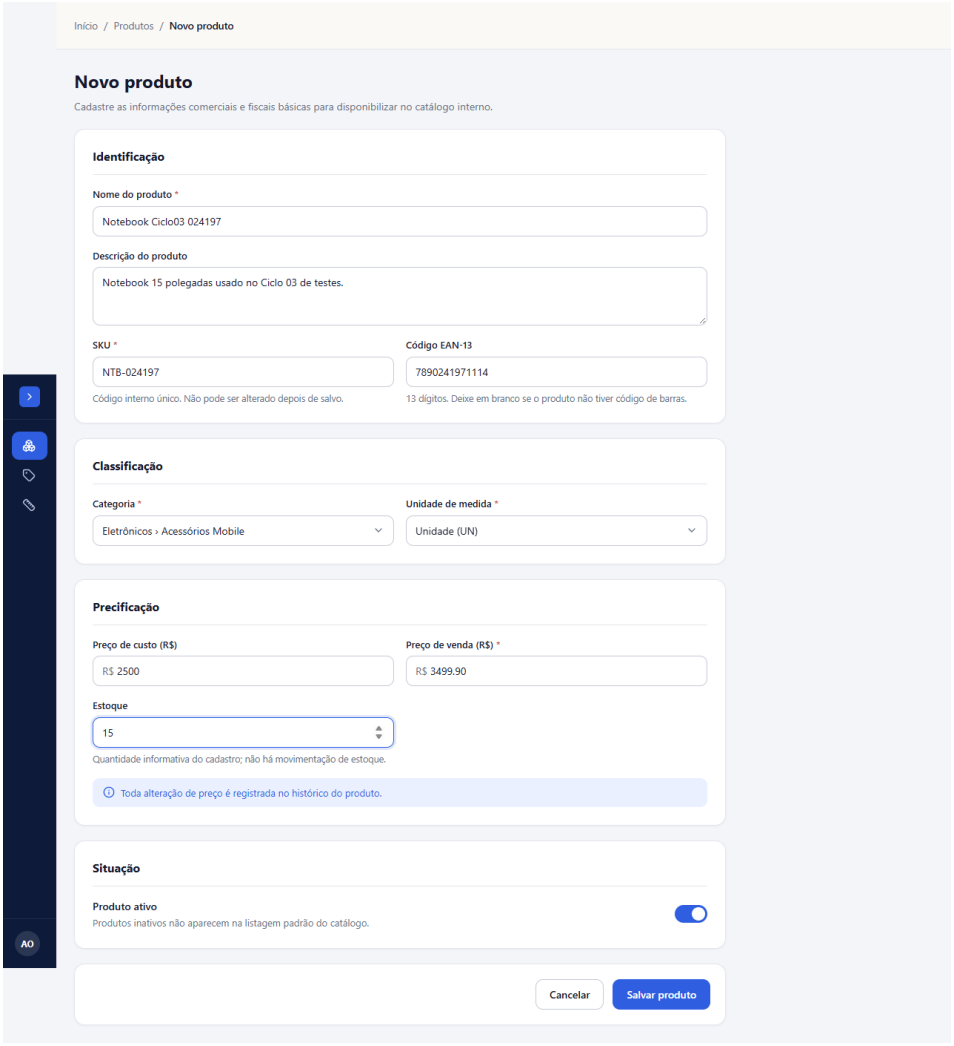


Figura 14 – TP-01: Form preenchido

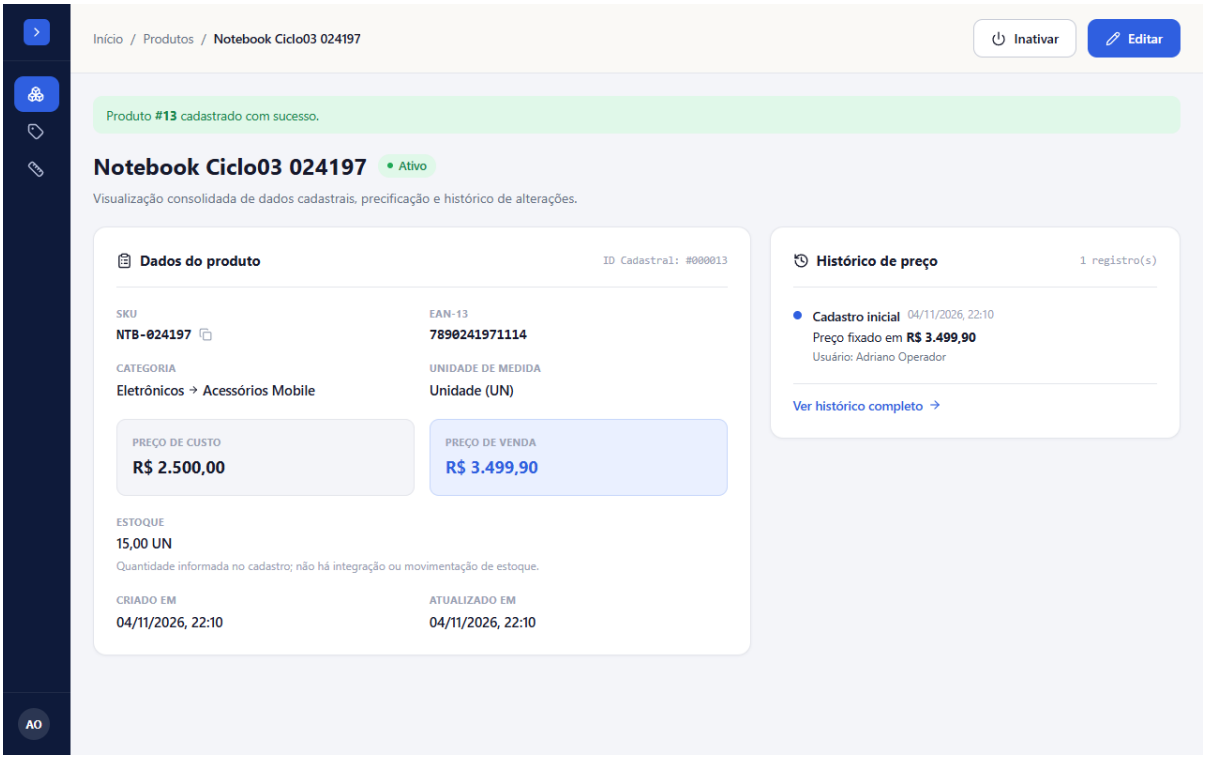


Figura 15 – TP-01: Detalhe criado

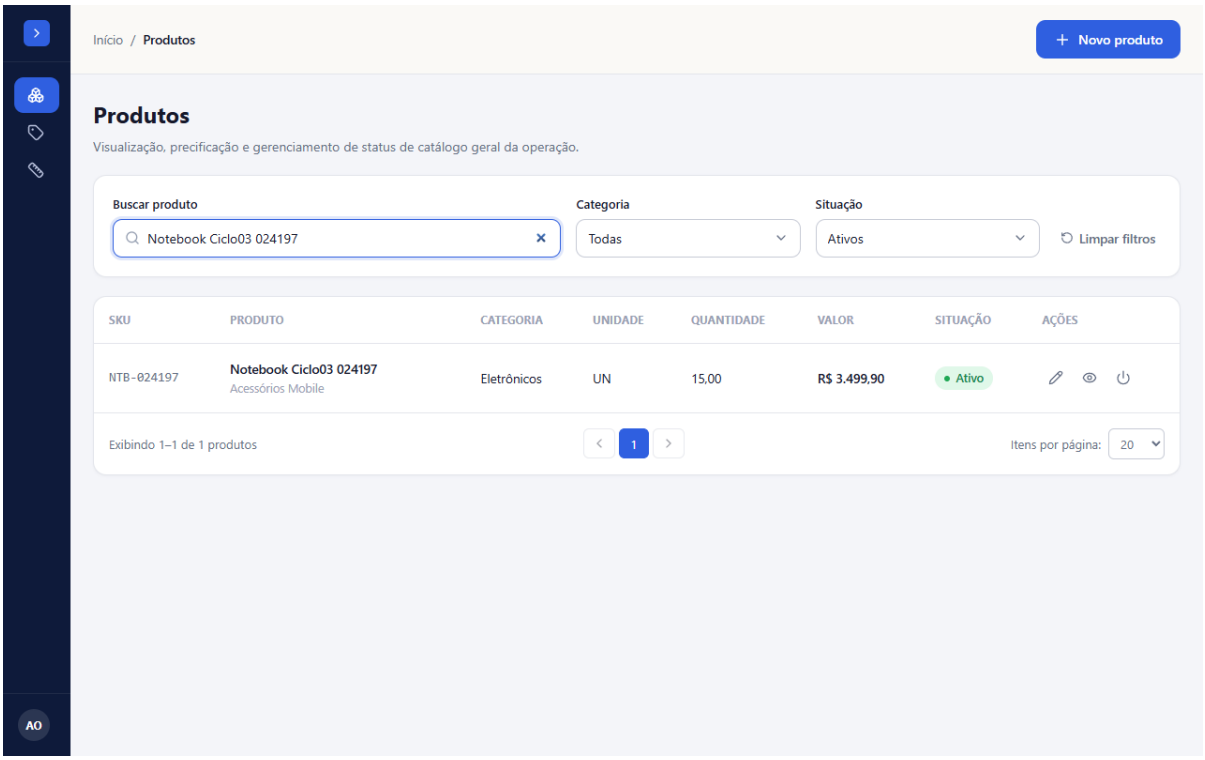


Figura 16 – TP-01: Listagem novo

TP-01 · Código executado no cadastro de produto

frontend/src/paginas/CadastroProduto.vue linhas 17–29 — Tela: envia e abre o detalhe

```vue
async function aoEnviar(valores) {
  enviando.value = true;
  erroServidor.value = null;
  try {
    const produto = await produtosApi.cadastrar(valores);
    await queryClient.invalidateQueries({ queryKey: ['produtos'] });
    router.push({ path: `/produtos/${produto.id}`, query: { criado: '1' } }); // RF01: confirmação com o identificador gerado
  } catch (erro) {
    erroServidor.value = erro instanceof ErroApi ? erro : new ErroApi({ erro: 'Não foi possível salvar o produto.', codigo: 'ERRO_INTERNO' });
  } finally {
    enviando.value = false;
  }
}
```

backend/src/servicos/produtos.servico.js linhas 30–56 — API: validações + transação

```backend
export async function cadastrar(dados, usuarioId) {
  await validarCategoriaEUnidade(dados.categoria_id, dados.unidade_medida_id);

  const skuExistente = await produtosRepo.buscarPorSku(dados.sku);
  if (skuExistente) throw new ErroAplicacao('SKU_DUPLICADO');

  if (dados.ean13) {
    const ean13Existente = await produtosRepo.buscarPorEan13(dados.ean13);
    if (ean13Existente) throw new ErroAplicacao('EAN13_DUPLICADO');
  }

  const nomeDuplicado = await produtosRepo.buscarPorNomeCategoria(dados.nome, dados.categoria_id);
  if (nomeDuplicado) throw new ErroAplicacao('NOME_DUPLICADO_NA_CATEGORIA');

  return executarEmTransacao(async (client) => {
    const { id } = await produtosRepo.inserir(client, dados);

    await precosServico.registrarHistoricoInicial(client, {
      produtoId: id,
      precoCusto: dados.preco_custo,
      precoVenda: dados.preco_venda,
      usuarioId,
    });

    return produtosRepo.buscarPorId(id, client);
  });
}
```

Figura 17 – TP-01: Codigo

TP-02 — Editar o preço de venda de um produto	Aprovado
Dados / passos utilizados	Tela /produtos/{id}/editar · preço de venda 3499,90 → 3299,90
Resultado esperado	Alteração salva e novo preço exibido no detalhe (R$ 3.299,90).
Resultado obtido	Detalhe do produto exibe R$ 3.299,90.

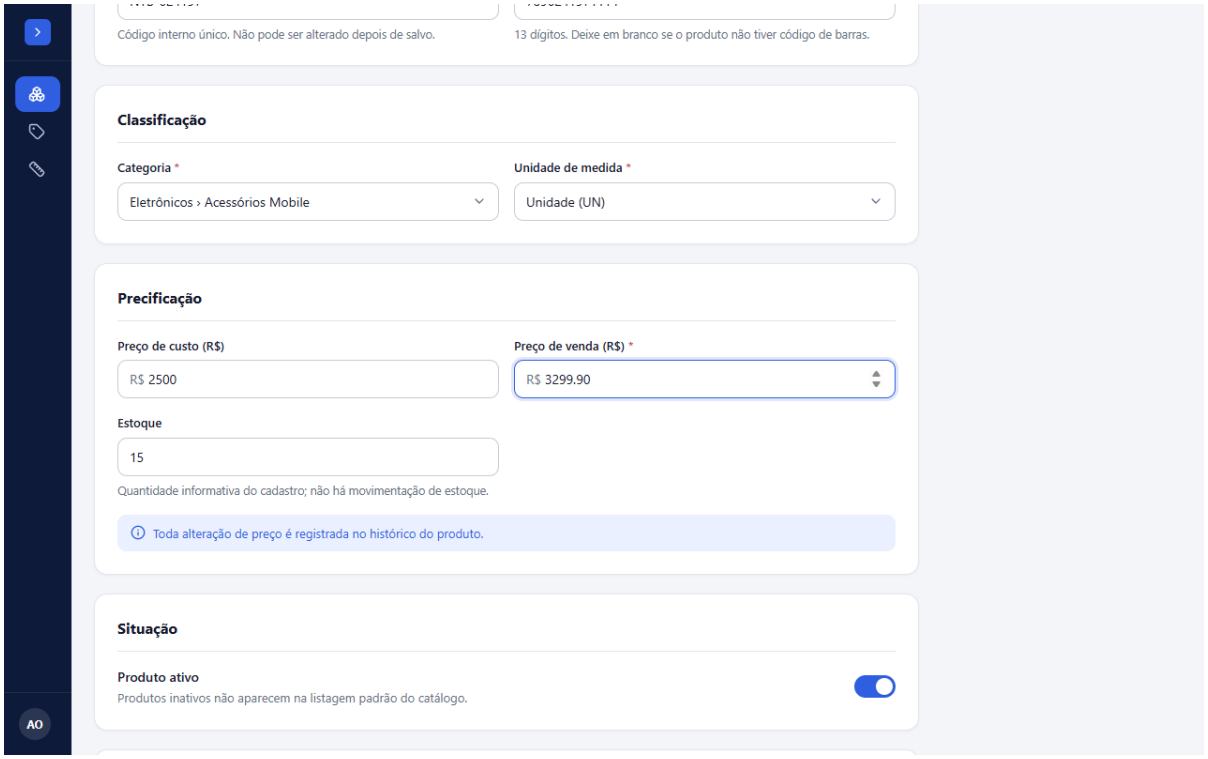


Figura 18 – TP-02: Edicao

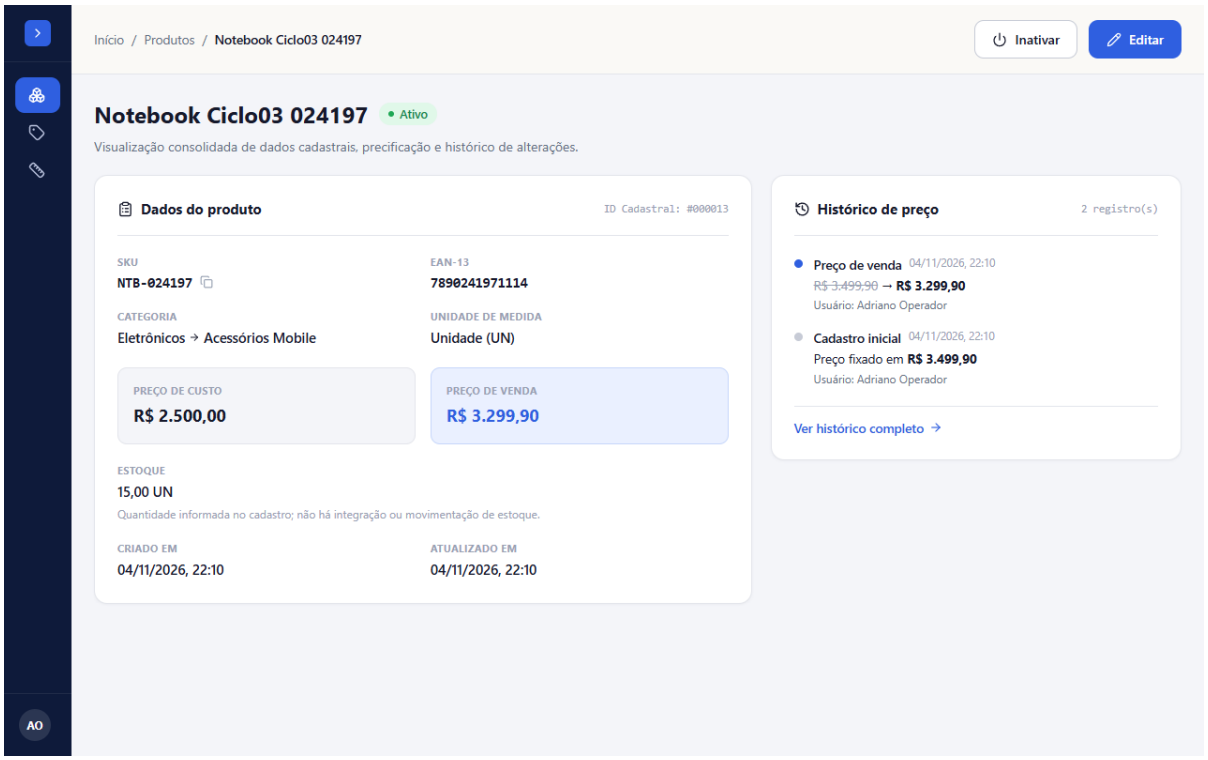


Figura 19 – TP-02: Detalhe atualizado

TP-02 · Código executado na edição de preço

frontend/src/paginas/EdicaoProduto.vue linhas 25–38

```
25 async function aoEnviar(valores) {
26   enviando.value = true;
27   erroServidor.value = null;
28   try {
29     await produtosApi.atualizar(props.id, valores);
30     await queryClient.invalidateQueries({ queryKey: ['produtos'] });
31     await queryClient.invalidateQueries({ queryKey: ['produto', props.id] });
32     router.push(`/produtos/${props.id}`);
33   } catch (erro) {
34     erroServidor.value = erro instanceof ErroApi ? erro : new ErroApi({ erro: 'Não foi possível salvar as
   alterações.', codigo: 'ERRO_INTERNO' });
35   } finally {
36     enviando.value = false;
37   }
38 }
```

backend/src/servicos/precos.servico.js linhas 27–55 API: grava valor anterior/novo

```
27 export async function registrarAlteracoesDePreco(client, { produtoAntigo, camposAtualizados, usuarioId, motivo
   }) {
28   if (
29     camposAtualizados.preco_venda !== undefined &&
30     Number(camposAtualizados.preco_venda) !== Number(produtoAntigo.preco_venda)
31   ) {
32     await historicoRepo.inserir(client, {
33       produto_id: produtoAntigo.id,
34       tipo_preco: 'venda',
35       valor_anterior: produtoAntigo.preco_venda,
36       valor_novo: camposAtualizados.preco_venda,
37       motivo: motivo ?? 'Alteração de preço de venda',
38       usuario_id: usuarioId,
39     });
40   }
41
42   if (
43     camposAtualizados.preco_custo !== undefined &&
44     Number(camposAtualizados.preco_custo) !== Number(produtoAntigo.preco_custo)
45   ) {
46     await historicoRepo.inserir(client, {
47       produto_id: produtoAntigo.id,
48       tipo_preco: 'custo',
49       valor_anterior: produtoAntigo.preco_custo,
50       valor_novo: camposAtualizados.preco_custo,
51       motivo: motivo ?? 'Alteração de preço de custo',
52       usuario_id: usuarioId,
53     });
54   }
55 }
```

Figura 20 – TP-02: Codigo

TP-03 — Consultar o histórico de preço do produto	Aprovado
Dados / passos utilizados	Tela /produtos/{id}/precos após o cadastro e a alteração de preço
Resultado esperado	Histórico listando o registro inicial e a alteração de venda (3.499,90 → 3.299,90).
Resultado obtido	Histórico apresenta o valor inicial (3.499,90) e o novo valor (3.299,90).

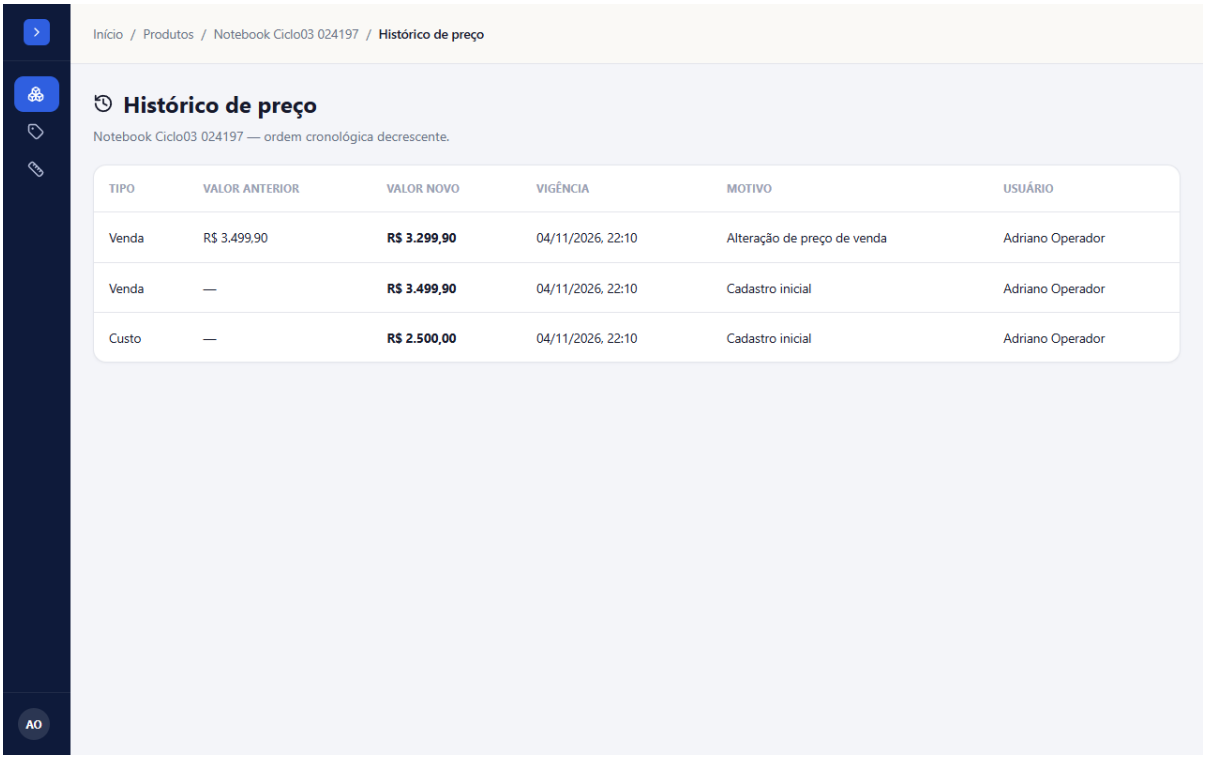


Figura 21 – TP-03: Historico

TP-03 · Código executado no histórico de preço

frontend/src/paginas/HistoricoPreco.vue linhas 19–22

```
19 const { data: resposta, isLoading, isError, refetch } = useQuery({
20   queryKey: ['produto', props.id, 'precos'],
21   queryFn: () => produtosApi.buscarHistorico(props.id),
22 });
```

backend/src/servicos/produtos.servico.js linhas 127–130

```
127 export async function listarHistoricoDePreco(id) {
128   await buscarPorId(id);
129   return precosServico.listarHistoricoDoProduto(id);
130 }
```

Figura 22 – TP-03: Codigo

TP-04 — Inativar produto pela listagem	**Aprovado**
Dados / passos utilizados	Listagem · busca pelo nome · ação "Inativar produto" · confirmar no modal
Resultado esperado	Produto some da listagem padrão (apenas ativos) e aparece com selo "Inativo" no filtro Inativos.
Resultado obtido	Produto removido dos ativos e exibido com selo "Inativo" no filtro Inativos.

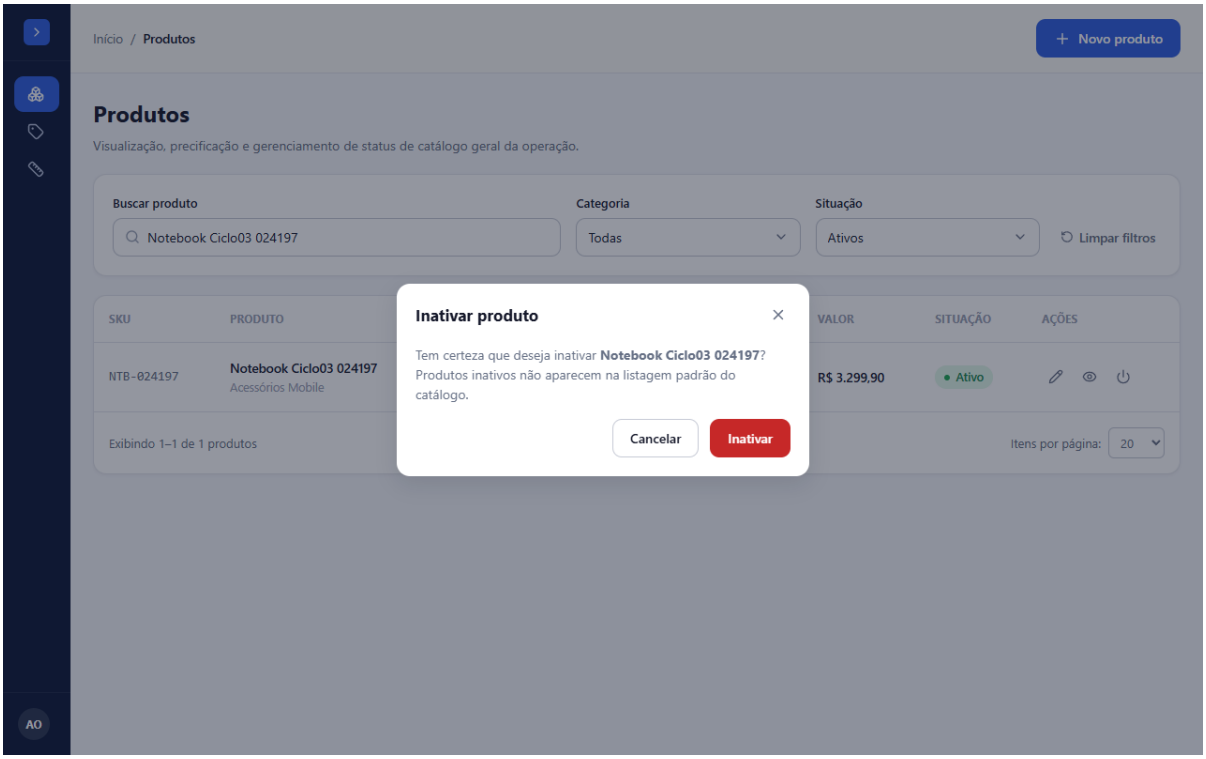


Figura 23 – TP-04: Modal inativar

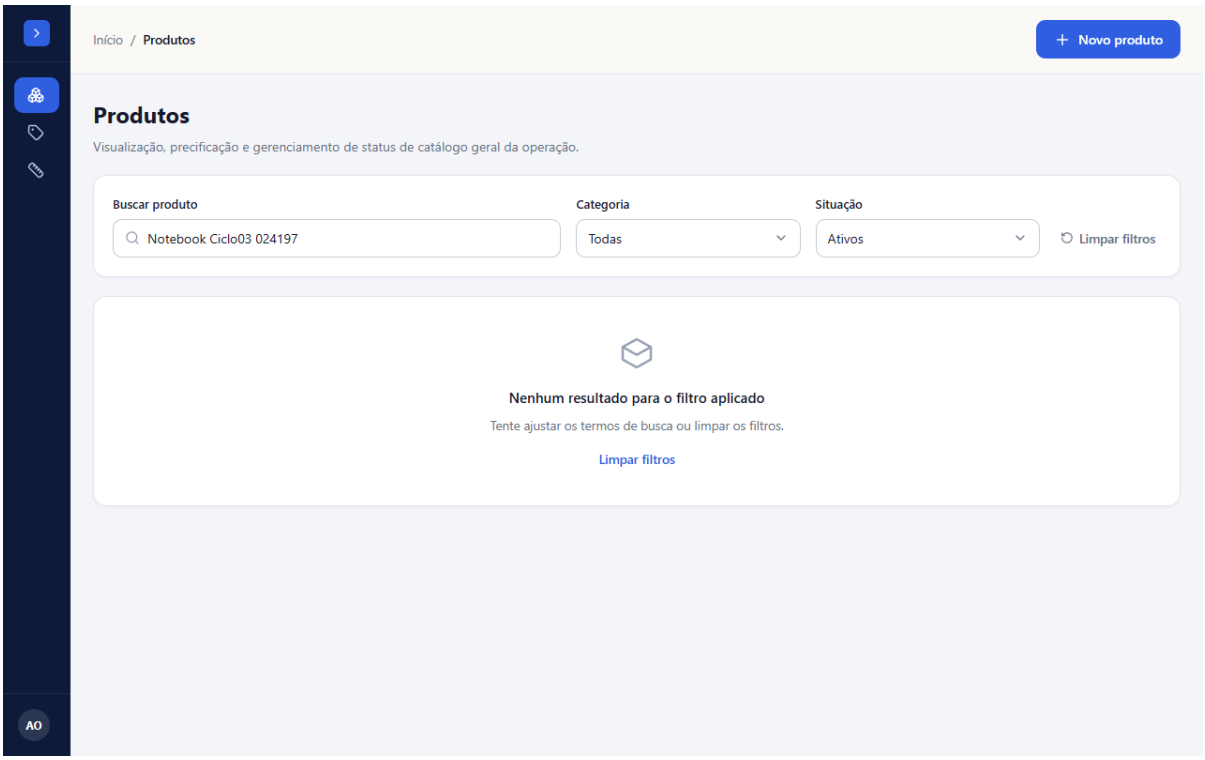


Figura 24 – TP-04: Some dos ativos

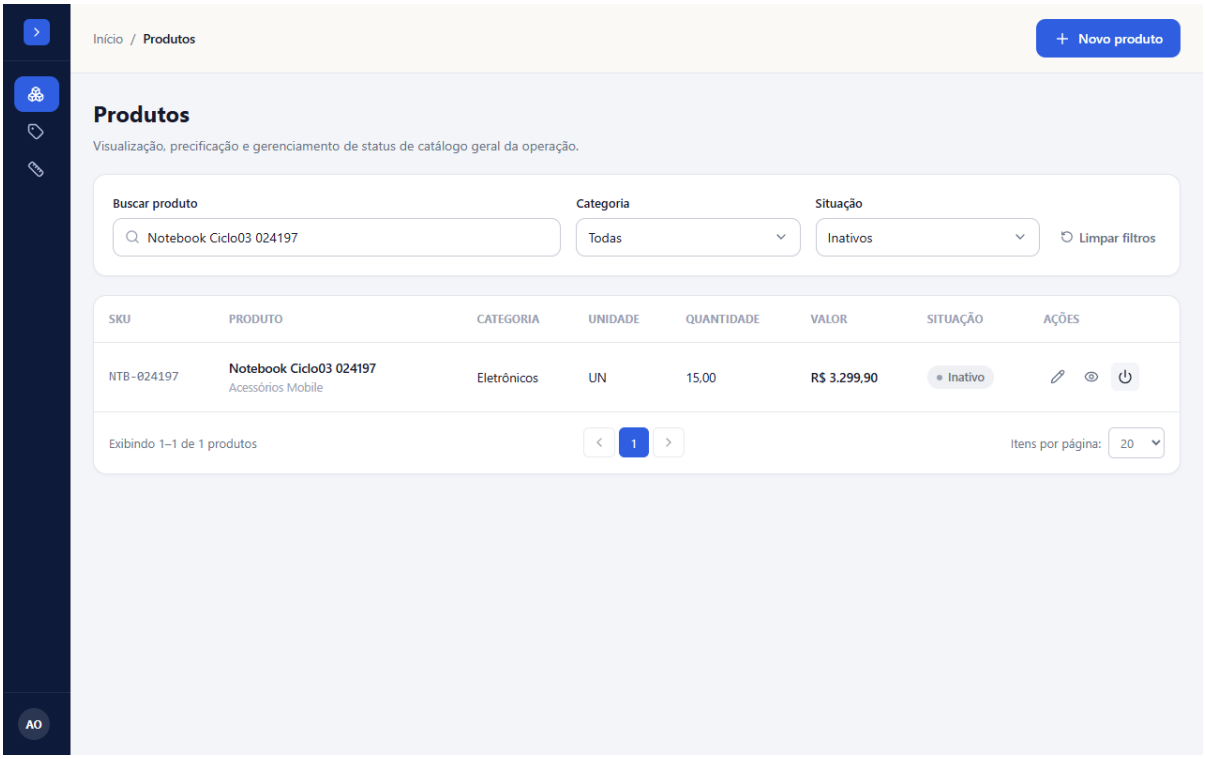


Figura 25 – TP-04: Filtro inativos

TP-04 · Código executado na inativação

frontend/src/paginas/ListagemProdutos.vue linhas 94–109 — Tela: modal de confirmação

```
94  async function confirmarAlternancia() {
95    if (!produtoParaAlternar.value) return;
96    alternandoSituacao.value = true;
97    erroAcao.value = '';
98    try {
99      const produto = produtoParaAlternar.value;
100     if (produto.situacao === 'ativo') await produtosApi.inativar(produto.id);
101     else await produtosApi.reativar(produto.id);
102     produtoParaAlternar.value = null;
103     await queryClient.invalidateQueries({ queryKey: ['produtos'] });
104   } catch (erro) {
105     erroAcao.value = erro instanceof ErroApi ? erro.message : 'Não foi possível concluir a operação.';
106   } finally {
107     alternandoSituacao.value = false;
108   }
109 }
```

backend/src/servicos/produtos.servico.js linhas 98–102 — API: inativação lógica

```
98  export async function inativar(id) {
99    const produto = await buscarPorId(id);
100   if (!produto.situacao || produto.situacao === 'inativo') throw new ErroAplicacao('PRODUTO_JA_INATIVO');
101   return produtosRepo.alterarSituacao(id, false);
102 }
```

Figura 26 – TP-04: Codigo

TP-05 — Reativar produto inativo	**Aprovado**
Dados / passos utilizados	Filtro Inativos · ação "Reativar produto" · confirmar
Resultado esperado	Produto volta a aparecer na listagem de ativos com selo "Ativo".

Resultado obtido	Produto reativado e visível na listagem de ativos.

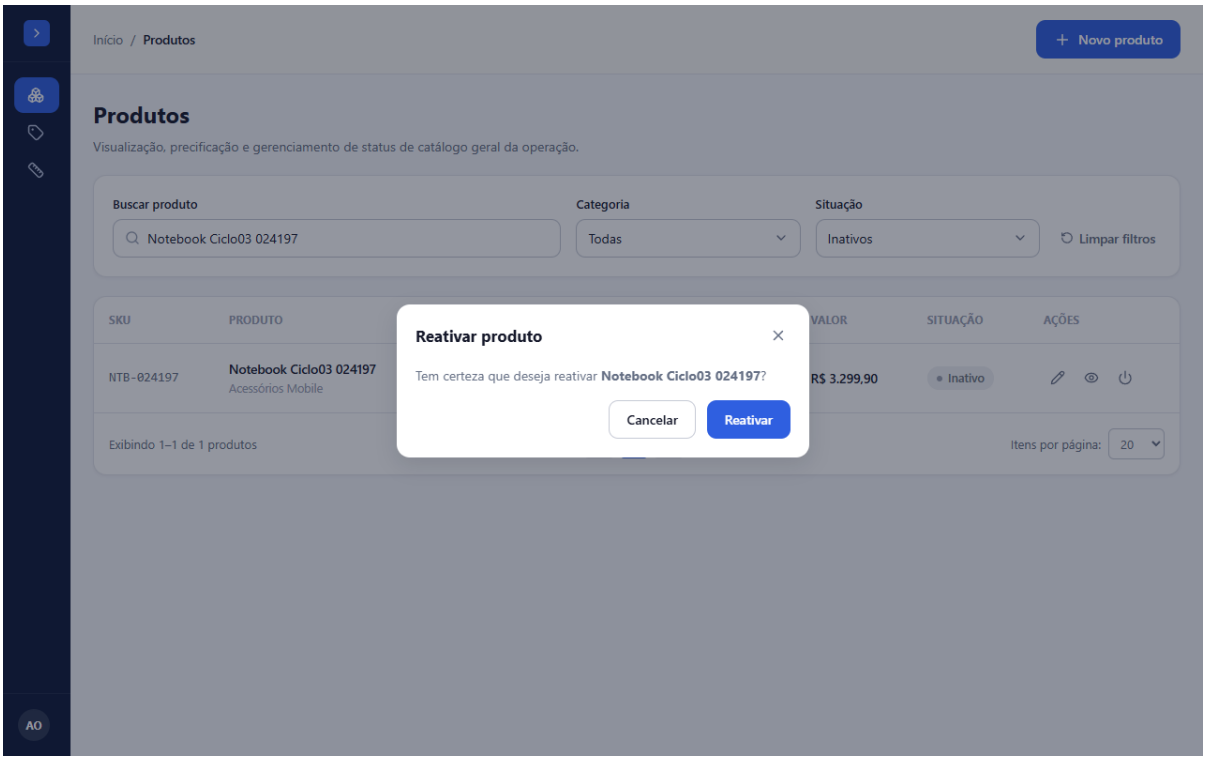


Figura 27 – TP-05: Modal reativar

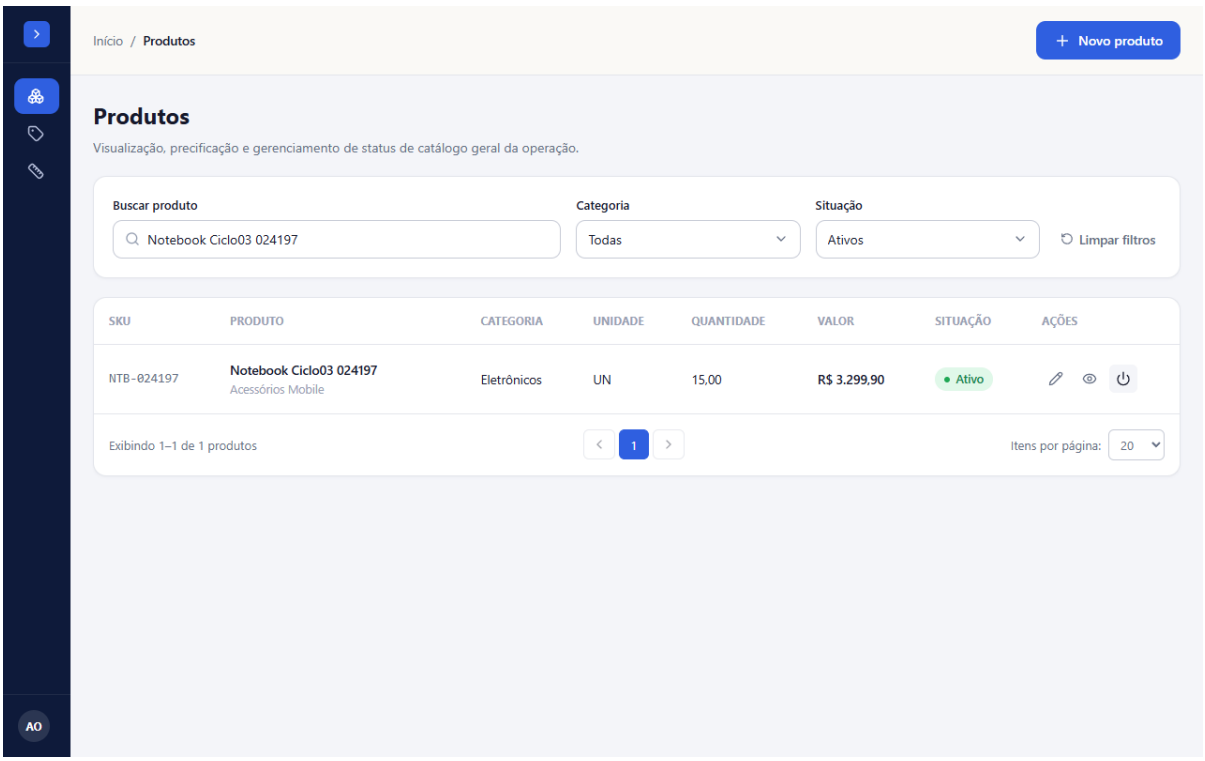


Figura 28 – TP-05: Reativado

TP-05 · Código executado na reativação

backend/src/servicos/produtos.servico.js linhas 105–109

```
105 export async function reativar(id) {
106   const produto = await buscarPorId(id);
107   if (produto.situacao === 'ativo') throw new ErroAplicacao('PRODUTO_JA_ATIVO');
108   return produtosRepo.alterarSituacao(id, true);
109 }
```

Figura 29 – TP-05: Codigo

TP-06 — Cadastrar categoria principal e subcategoria pela interface	Aprovado
Dados / passos utilizados	Tela /categorias · "Nova categoria" · nome "Categoria Ciclo03 024197" · depois adicionar a subcategoria "Sub 024197" no card
Resultado esperado	Categoria criada e subcategoria vinculada ao card da categoria principal.
Resultado obtido	Categoria e subcategoria criadas e exibidas na árvore.

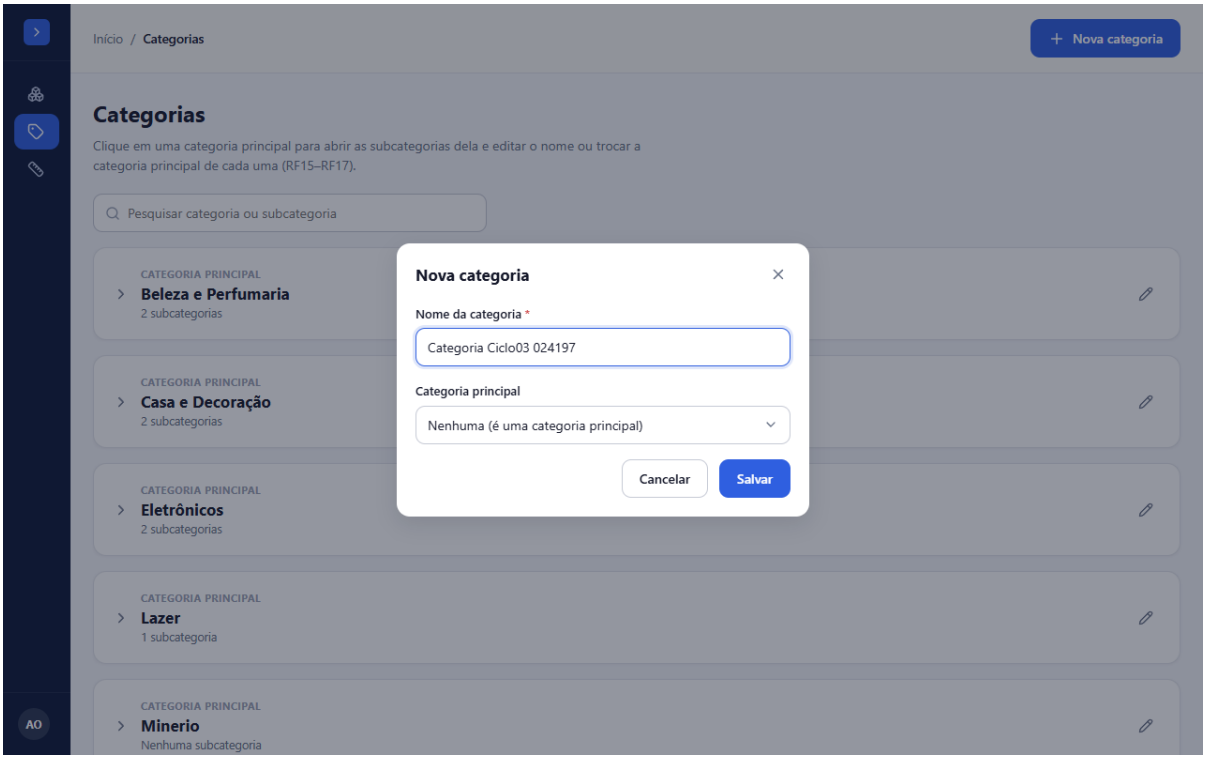


Figura 30 – TP-06: Modal categoria

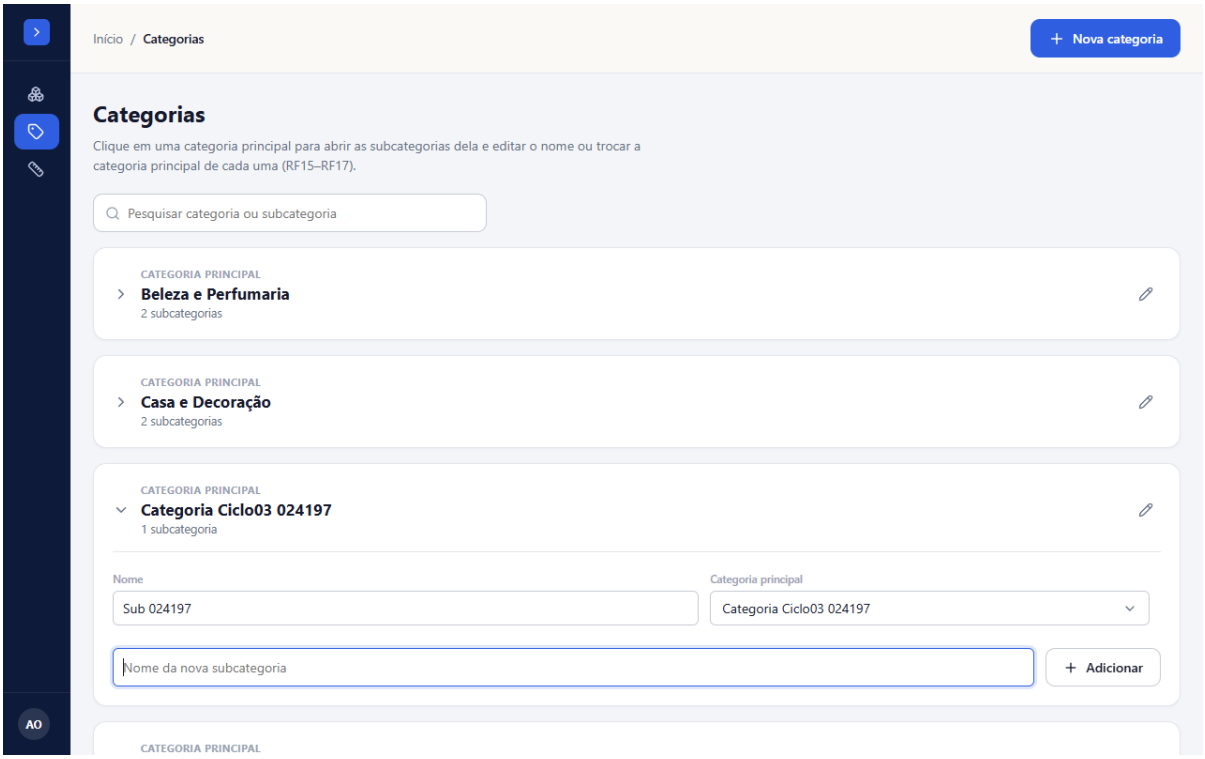


Figura 31 – TP-06: Categoria subcategoria

TP-06 · Código executado no cadastro de categoria/subcategoria

frontend/src/paginas/Categorias.vue linhas 73–86

```
73  async function adicionarSubcategoria(principal) {
74    const nomeNovo = (novasSubcategorias.value[principal.id] ?? '').trim();
75    if (!nomeNovo) {
76      erroNovaSubcategoria.value = { ...erroNovaSubcategoria.value, [principal.id]: 'Informe o nome da
    subcategoria' };
77      return;
78    }
79    adicionandoEm.value = principal.id;
80    erroNovaSubcategoria.value = { ...erroNovaSubcategoria.value, [principal.id]: '' };
81    try {
82      await categoriasApi.cadastrar({ nome: nomeNovo, categoria_pai_id: principal.id });
83      novasSubcategorias.value = { ...novasSubcategorias.value, [principal.id]: '' };
84      await queryClient.invalidateQueries({ queryKey: ['categorias'] });
85    } catch (erro) {
86      erroNovaSubcategoria.value = {
```

backend/src/servicos/categorias.servico.js linhas 36–43

```
36  export async function cadastrar({ nome, categoria_pai_id }) {
37    const existente = await categoriasRepo.buscarPorNome(nome);
38    if (existente) throw new ErroAplicacao('NOME_CATEGORIA_DUPLICADO');
39
40    if (categoria_pai_id) await buscarPorId(categoria_pai_id); // 404 se pai não existir
41
42    return categoriasRepo.inserir({ nome, categoria_pai_id });
43  }
```

Figura 32 – TP-06: Codigo

TP-07 — Cadastrar unidade de medida pela interface	**Aprovado**
Dados / passos utilizados	Tela /unidades-medida · "Nova unidade" · nome "Caixa C024197" · sigla "CX197"

Resultado esperado	Unidade criada e presente na lista.
Resultado obtido	Unidade criada e exibida na lista.

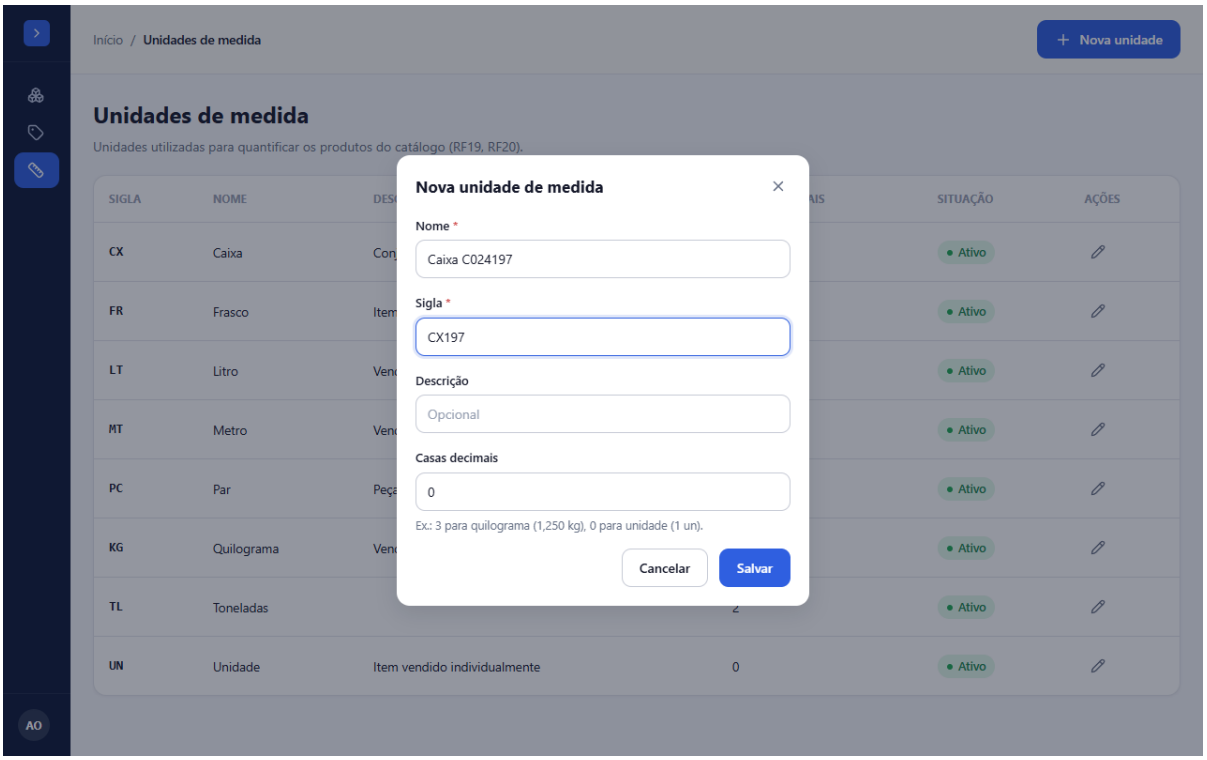


Figura 33 – TP-07: Modal unidade

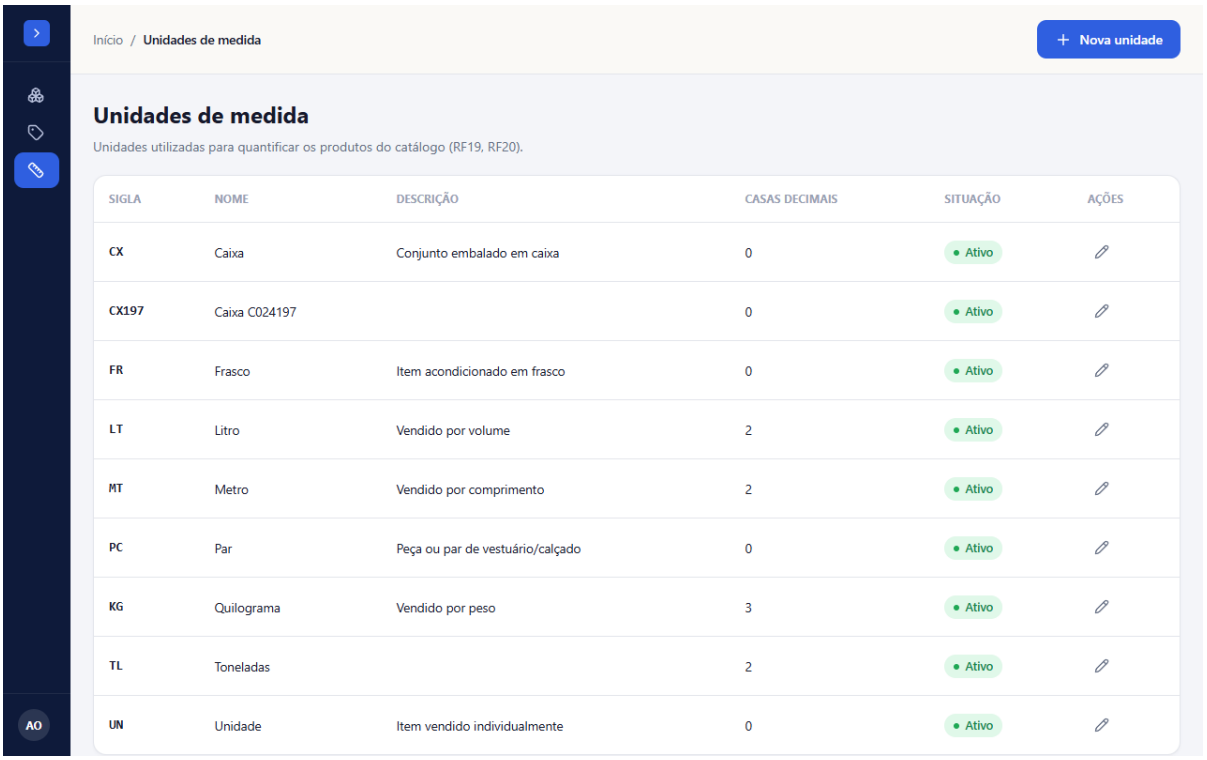


Figura 34 – TP-07: Unidade criada

TP-07 · Código executado no cadastro de unidade

backend/src/servicos/unidades.servico.js linhas 15–20

```
15 export async function cadastrar({ nome, sigla, descricao, casas_decimais }) {
16   const existente = await unidadesRepo.buscarPorSigla(sigla);
17   if (existente) throw new ErroAplicacao('SIGLA_UNIDADE_DUPLICADA');
18
19   return unidadesRepo.inserir({ nome, sigla, descricao, casas_decimais });
20 }
```

Figura 35 – TP-07: Codigo

TP-08 — Filtrar produtos por categoria	Aprovado
Dados / passos utilizados	Listagem · filtro Categoria = "Acessórios Mobile"
Resultado esperado	Somente produtos da categoria selecionada na tabela.
Resultado obtido	1 produto(s) listado(s), todos da categoria filtrada.

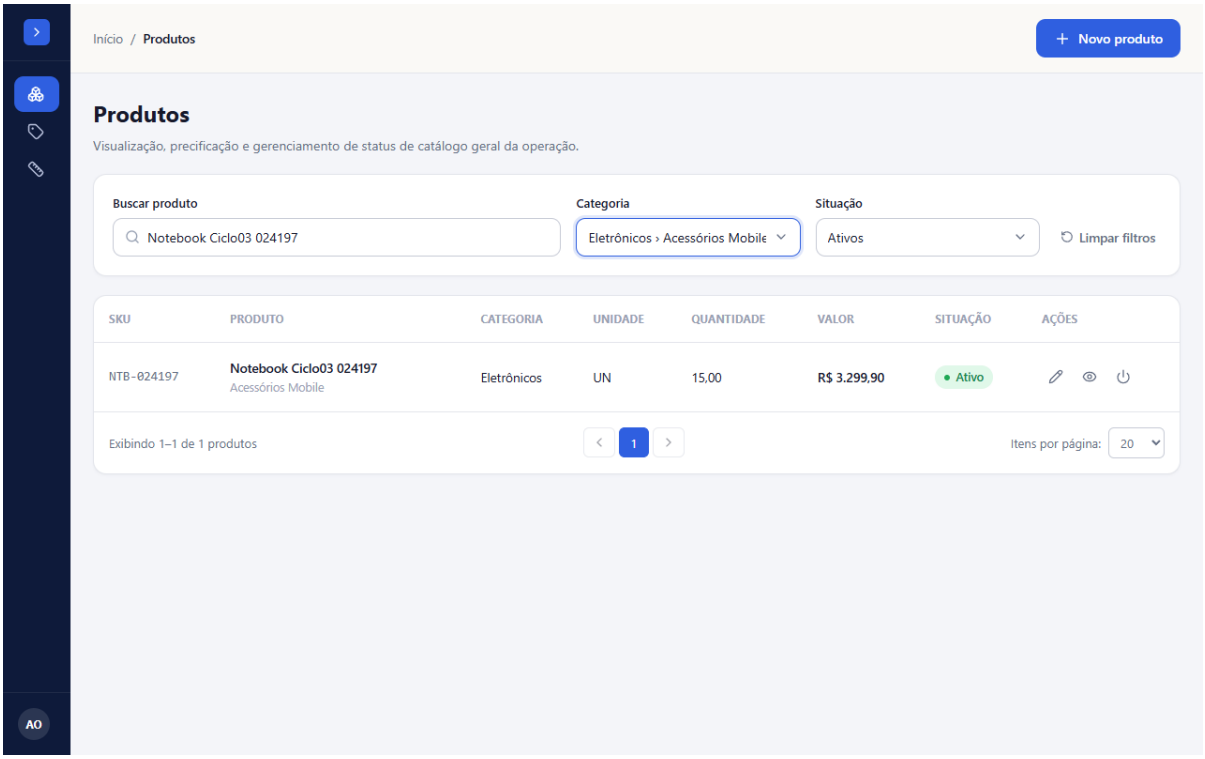


Figura 36 – TP-08: Filtro categoria

TP-08 · Código executado no filtro por categoria

backend/src/repositorios/produtos.repo.js linhas 38–59

```
38 function montarFiltro({ nome, sku, categoria_id, situacao }) {
39   const condicoes = [];
40   const parametros = [];
41
42   if (nome) {
43     parametros.push(`%${nome}%`);
44     condicoes.push(`p.nome ILIKE $${parametros.length}`);
45   }
46   if (sku) {
47     parametros.push(`%${sku}%`);
48     condicoes.push(`p.sku ILIKE $${parametros.length}`);
49   }
50   if (categoria_id) {
51     parametros.push(categoria_id);
52     condicoes.push(`p.categoria_id = $${parametros.length}`);
53   }
54   if (situacao === 'ativo') condicoes.push('p.ativo = TRUE');
55   if (situacao === 'inativo') condicoes.push('p.ativo = FALSE');
56
57   const where = condicoes.length ? `WHERE ${condicoes.join(' AND ')}` : '';
58   return { where, parametros };
59 }
```

Figura 37 – TP-08: Codigo

3. Testes negativos

O que é: Verificam se o sistema identifica e trata dados inválidos, campos obrigatórios vazios, formatos incorretos ou operações não permitidas.

Aplicação neste sistema: Foram enviados campos vazios, preços e estoque negativos, EAN-13 com dígito verificador errado, SKU, nome e e-mail duplicados, senha curta e acesso sem sessão.

TN-01 — Login com senha incorreta	Aprovado
Dados / passos utilizados	Tela /login · e-mail demo@sige.com.br · senha "errada123"
Resultado esperado	Login negado com mensagem genérica (sem revelar qual campo errou); usuário permanece em /login.
Resultado obtido	Mensagem "E-mail ou senha inválidos" exibida; permaneceu em /login.

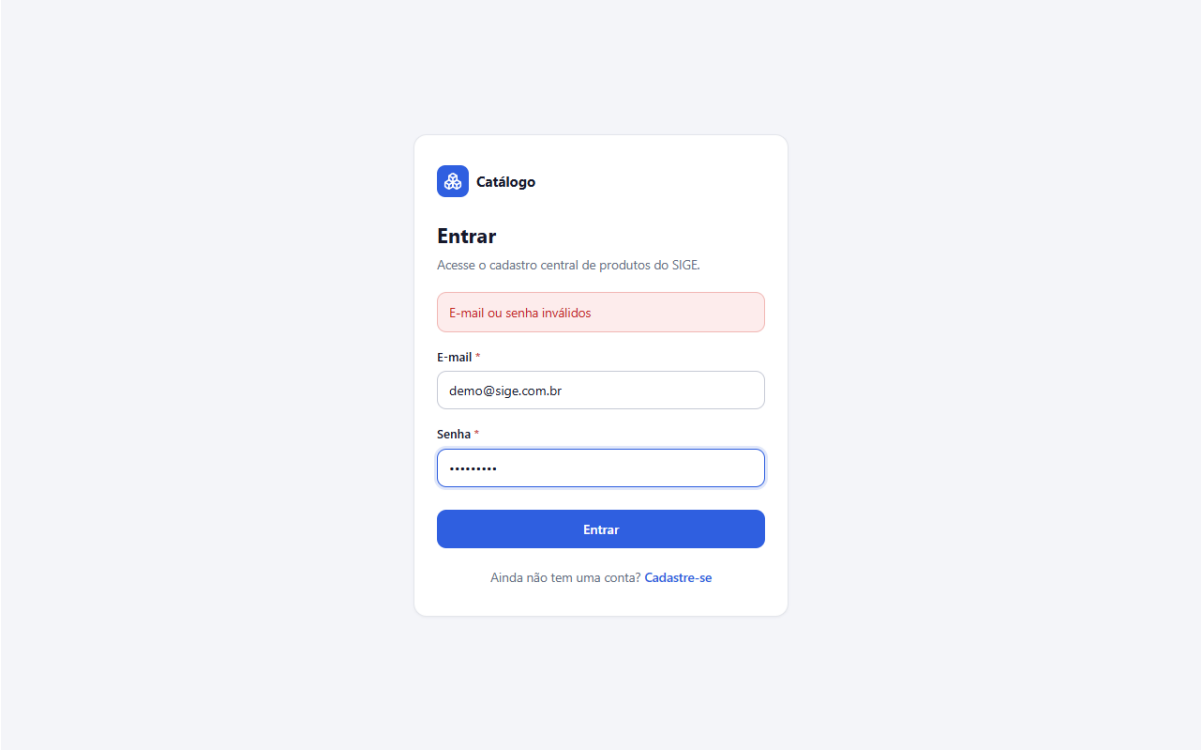


Figura 38 – TN-01: Login invalido

TN-01 · Código que gera a mensagem genérica de login

backend/src/servicos/auth.servico.js linhas 16–23

```
16 export async function login({ email, senha }) {
17   const usuario = await usuariosRepo.buscarPorEmailComSenha(email);
18   const ok = usuario ? await bcrypt.compare(senha, usuario.senha) : false;
19
20   if (!ok) throw new ErroAplicacao('CREDENCIAIS_INVALIDAS');
21
22   return { id: usuario.id, nome: usuario.nome, email: usuario.email };
23 }
```

backend/src/erros/codigos.js linhas 6–6

```
6   CREDENCIAIS_INVALIDAS: { status: 401, mensagem: 'E-mail ou senha inválidos' },
```

Figura 39 – TN-01: Codigo

TN-02 — Acesso a rota protegida sem sessão	**Aprovado**
Dados / passos utilizados	Sem login, acessar diretamente /produtos/novo
Resultado esperado	Redirecionamento para /login.
Resultado obtido	URL final: /login?redirecionarPara=/produtos/novo

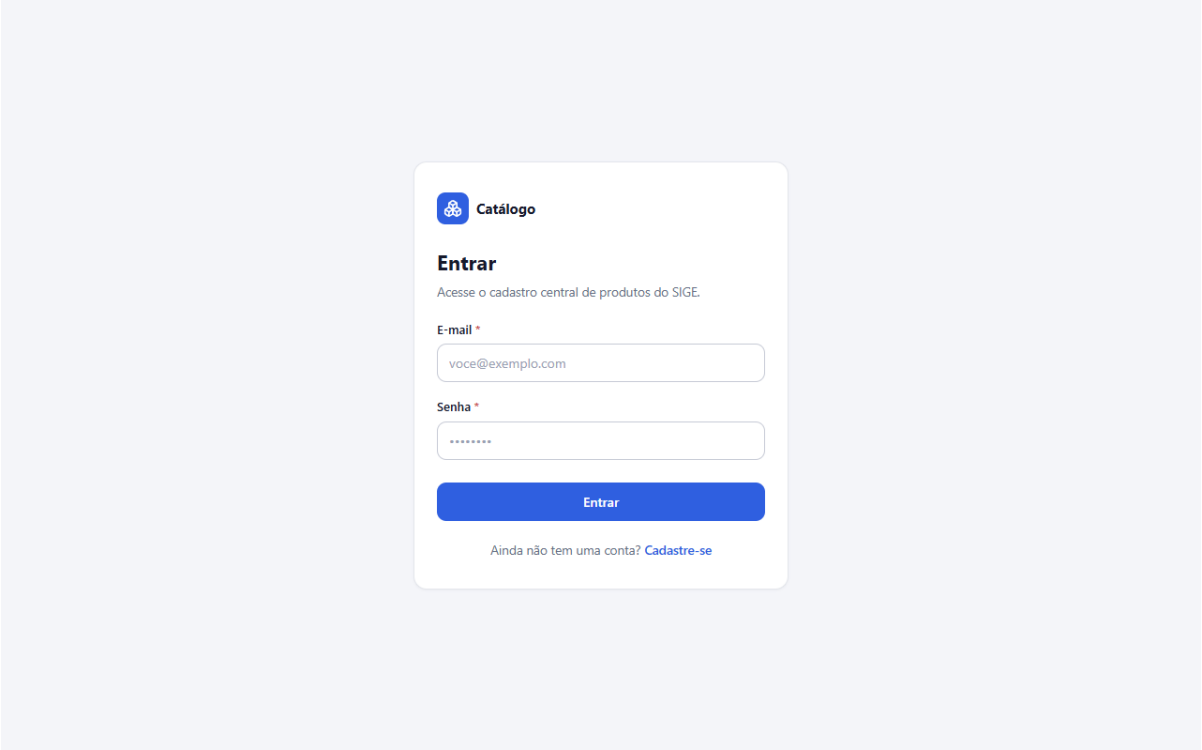


Figura 40 – TN-02: Sem sessao

TN-02 · Código que bloqueia rota sem sessão

frontend/src/rotas/index.js linhas 79–100

```
79  router.beforeEach(async (to) => {
80    const sessao = useSessaoStore();
81
82    if (to.meta.requerSessao) {
83      if (!sessao.estaAutenticado) {
84        return { name: 'login', query: { redirecionarPara: to.fullPath } };
85      }
86
87      try {
88        await authApi.eu();
89      } catch {
90        sessao.encerrarSessao();
91        return { name: 'login', query: { redirecionarPara: to.fullPath, sessaoExpirada: '1' } };
92      }
93    }
94
95    if (to.meta.publica && sessao.estaAutenticado) {
96      return { name: 'produtos-lista' };
97    }
98
99    return true;
100 });
```

Figura 41 – TN-02: Codigo

TN-03 — Cadastro de usuário com senha curta e e-mail inválido	**Aprovado**
Dados / passos utilizados	Tela /cadastro · nome "Ab" · e-mail "semarroba" · senha "123"
Resultado esperado	Mensagens de validação por campo e cadastro bloqueado.
Resultado obtido	Exibidas as mensagens "Informe seu nome", "E-mail inválido" e "A senha deve ter no mínimo 8 caracteres".

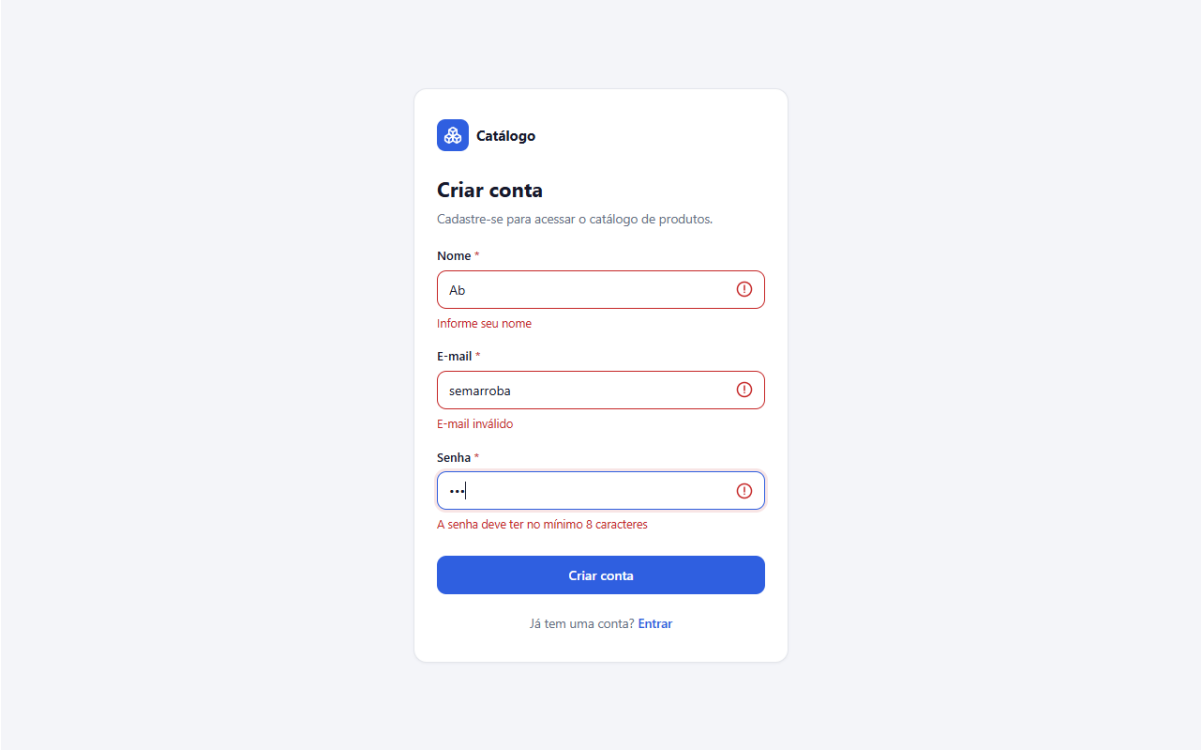


Figura 42 – TN-03: Cadastro invalido

TN-03 · Regras de validação do cadastro de usuário (Zod compartilhado)

shared/schemas/usuario.schema.js linhas 3–7

```
3  export const usuarioCadastroSchema = z.object({
4    nome: z.string().trim().min(3, 'Informe seu nome').max(60, 'Nome muito longo'),
5    email: z.string().trim().toLowerCase().email('E-mail inválido').max(100),
6    senha: z.string().min(8, 'A senha deve ter no mínimo 8 caracteres').max(72), // RN14
7  }).strict();
```

Figura 43 – TN-03: Codigo

TN-04 — Cadastrar produto com campos obrigatórios vazios	**Aprovado**
Dados / passos utilizados	Tela /produtos/novo · clicar em "Salvar produto" sem preencher nada
Resultado esperado	Mensagens de obrigatoriedade (nome, SKU, categoria, unidade, preço de venda) e nenhum produto criado.
Resultado obtido	Exibidas mensagens para nome, categoria, unidade de medida e demais campos obrigatórios; permaneceu em /produtos/novo.

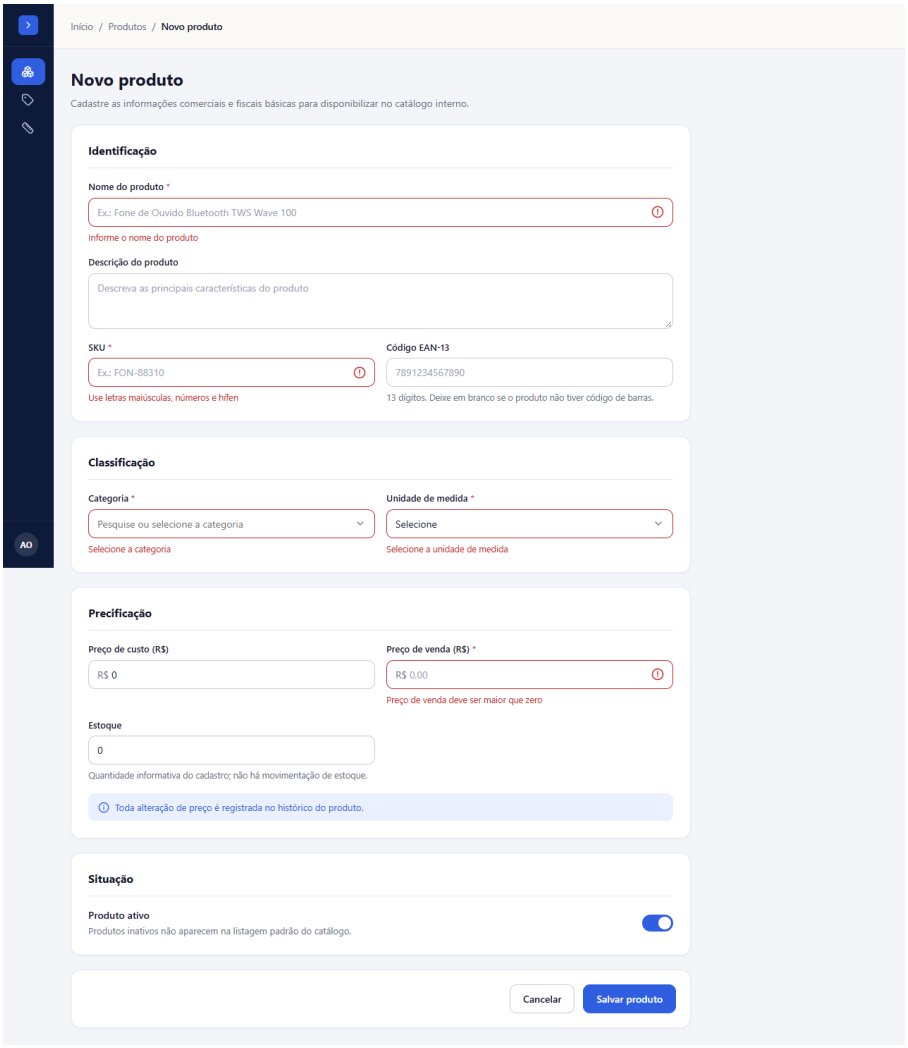


Figura 44 – TN-04: Obrigatorios

TN-04 · Regras de obrigatoriedade do produto (Zod compartilhado front/back)

shared/schemas/produto.schema.js linhas 4–22

```
4  export const produtoCriacaoSchema = z.object({
5    sku: z.string().trim().max(15).regex(/^[A-Z0-9-]+$/, 'Use letras maiúsculas, números e hífen'),
6    nome: z.string().trim().min(1, 'Informe o nome do produto').max(60), // RF06
7    descricao: z.string().trim().max(2000).optional().nullable(),
8    // Campo em branco chega como '' do formulário — sem o preprocess, '' cairia em .length(13)
9    // e falharia a validação mesmo sendo opcional (RF22: "deixe em branco se não tiver código").
10   ean13: z.preprocess(
11     (valor) => (typeof valor === 'string' && valor.trim() === '' ? null : valor),
12     z.string().length(13).regex(/^\d{13}$/)
13       .refine(ean13Valido, 'Dígito verificador do EAN-13 inválido') // RF22 / RN04
14       .nullable().optional()
15   ),
16   categoria_id: z.coerce.number().int().positive('Selecione a categoria'), // RF07 / RN11
17   unidade_medida_id: z.coerce.number().int().positive('Selecione a unidade de medida'),
18   preco_custo: z.coerce.number().min(0, 'Preço de custo não pode ser negativo').default(0), // RN02
19   preco_venda: z.coerce.number().gt(0, 'Preço de venda deve ser maior que zero'), // RF08 / RN01
20   estoque: z.coerce.number().min(0, 'Estoque não pode ser negativo').default(0),
21   ativo: z.boolean().default(true),
22 }).strict();
```

Figura 45 – TN-04: Codigo

TN-05 — Preço de venda zero/negativo e estoque negativo	Aprovado

Dados / passos utilizados	Tela /produtos/novo · preço de venda = -10 · estoque = -5 · preço de custo = -1
Resultado esperado	Mensagens "Preço de venda deve ser maior que zero", "Estoque não pode ser negativo" e "Preço de custo não pode ser negativo".
Resultado obtido	As três mensagens de validação foram exibidas e o envio foi bloqueado.

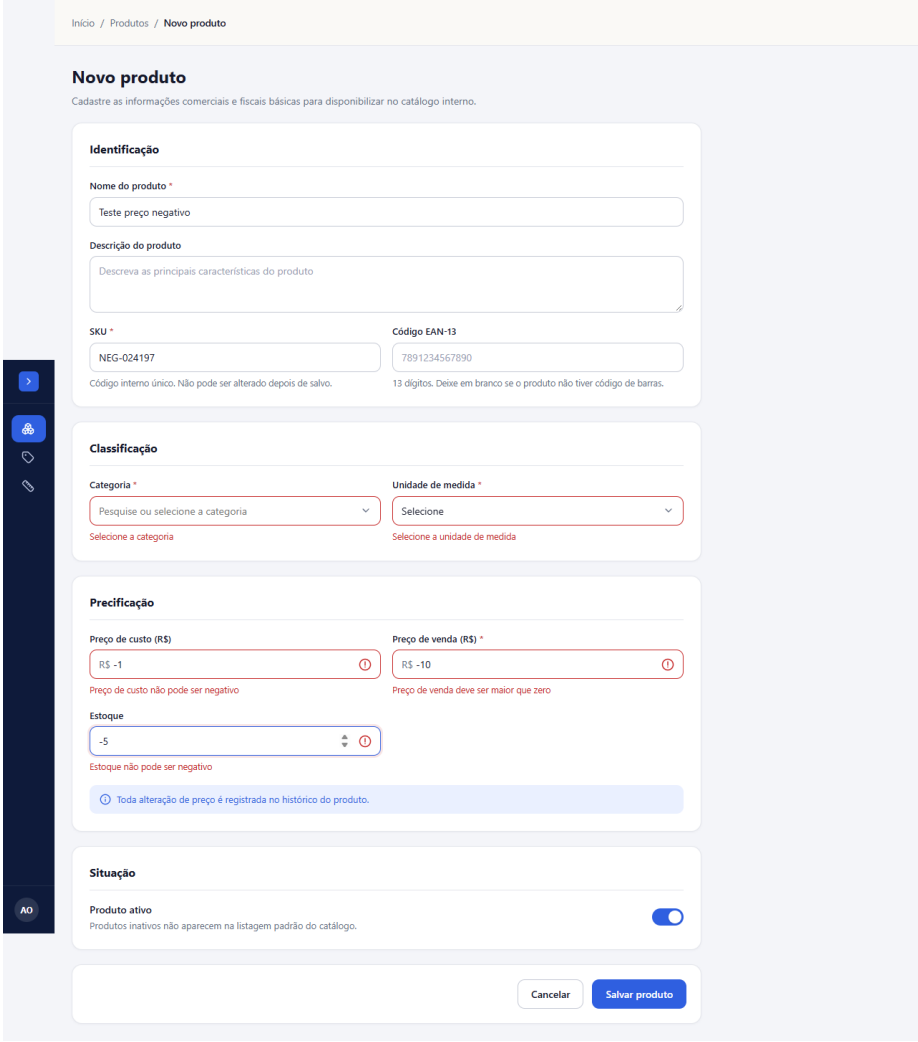


Figura 46 – TN-05: Precos negativos

TN-05 · Regras de preço e estoque (Zod)

shared/schemas/produto.schema.js linhas 4–22

```
4  export const produtoCriacaoSchema = z.object({
5    sku: z.string().trim().max(15).regex(/^[A-Z0-9-]+$/, 'Use letras maiúsculas, números e hífen'),
6    nome: z.string().trim().min(1, 'Informe o nome do produto').max(60), // RF06
7    descricao: z.string().trim().max(2000).optional().nullable(),
8    // Campo em branco chega como '' do formulário — sem o preprocess, '' cairia em .length(13)
9    // e falharia a validação mesmo sendo opcional (RF22: "deixe em branco se não tiver código").
10   ean13: z.preprocess(
11     (valor) => (typeof valor === 'string' && valor.trim() === '' ? null : valor),
12     z.string().length(13).regex(/^\d{13}$/)
13       .refine(ean13Valido, 'Dígito verificador do EAN-13 inválido') // RF22 / RN04
14       .nullable().optional()
15   ),
16   categoria_id: z.coerce.number().int().positive('Selecione a categoria'), // RF07 / RN11
17   unidade_medida_id: z.coerce.number().int().positive('Selecione a unidade de medida'),
18   preco_custo: z.coerce.number().min(0, 'Preço de custo não pode ser negativo').default(0), // RN02
19   preco_venda: z.coerce.number().gt(0, 'Preço de venda deve ser maior que zero'), // RF08 / RN01
20   estoque: z.coerce.number().min(0, 'Estoque não pode ser negativo').default(0),
21   ativo: z.boolean().default(true),
22 }).strict();
```

Figura 47 – TN-05: Codigo

TN-06 — Cadastrar produto com EAN-13 de dígito verificador inválido	**Aprovado**
Dados / passos utilizados	Tela /produtos/novo · EAN-13 = 7890241971115
Resultado esperado	Mensagem "Dígito verificador do EAN-13 inválido" e envio bloqueado.
Resultado obtido	Mensagem de dígito verificador inválido exibida.

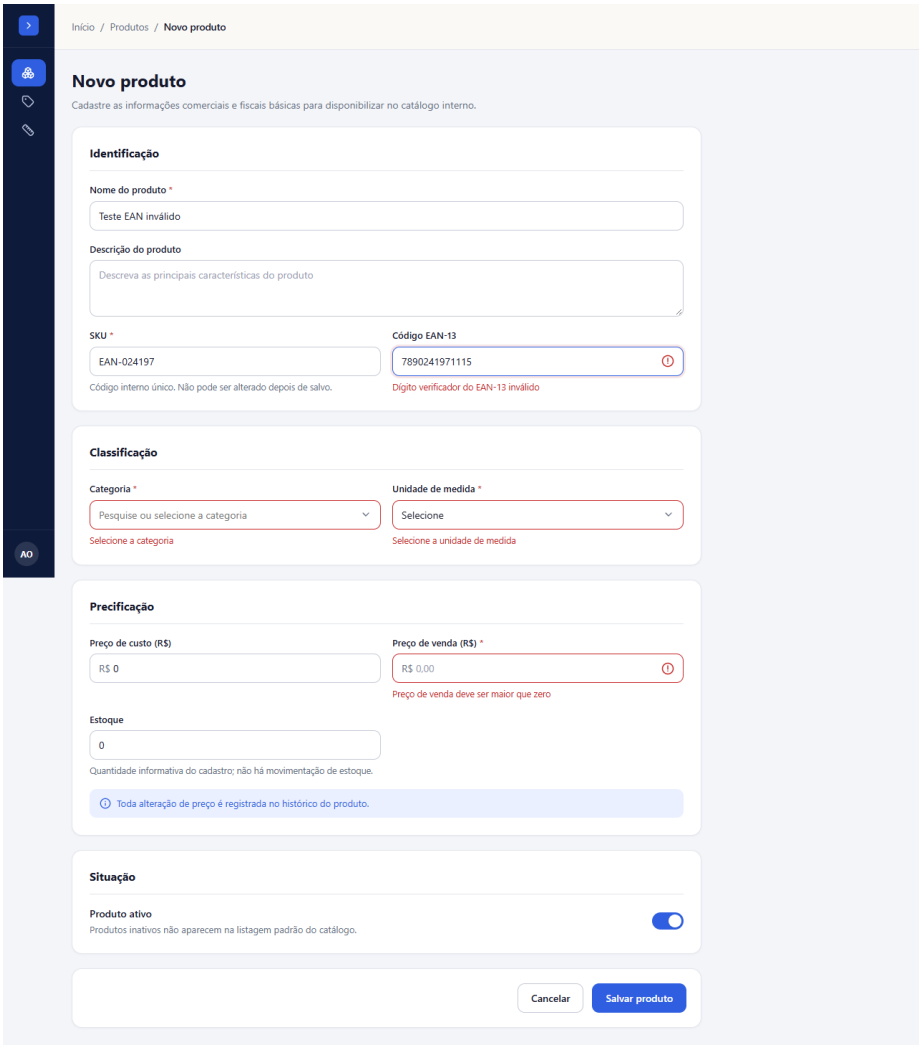


Figura 48 – TN-06: Ean invalido

TN-06 · Validação do dígito verificador do EAN-13

shared/util/ean13.js linhas 2–13

```
2  export function ean13Valido(codigo) {
3    if (typeof codigo !== 'string' || !/^\d{13}$/.test(codigo)) return false;
4
5    const digitos = codigo.slice(0, 12).split('').map(Number);
6    const soma = digitos.reduce((acc, digito, indice) => {
7      return acc + (indice % 2 === 0 ? digito : digito * 3);
8    }, 0);
9    const resto = soma % 10;
10   const dvEsperado = resto === 0 ? 0 : 10 - resto;
11
12   return dvEsperado === Number(codigo[12]);
13 }
```

shared/schemas/produto.schema.js linhas 10–15

```
10   ean13: z.preprocess(
11     (valor) => (typeof valor === 'string' && valor.trim() === '' ? null : valor),
12     z.string().length(13).regex(/^\d{13}$/)
13       .refine(ean13Valido, 'Dígito verificador do EAN-13 inválido') // RF22 / RN04
14       .nullable().optional()
15   ),
```

Figura 49 – TN-06: Codigo

TN-07 — Cadastrar produto com SKU já existente	Aprovado
Dados / passos utilizados	Tela /produtos/novo · SKU NTB-024197 (já usado no TP-01) · demais campos válidos
Resultado esperado	Mensagem "Já existe um produto com este SKU" no campo SKU; produto não duplicado.
Resultado obtido	Mensagem "Já existe um produto com este SKU" exibida.

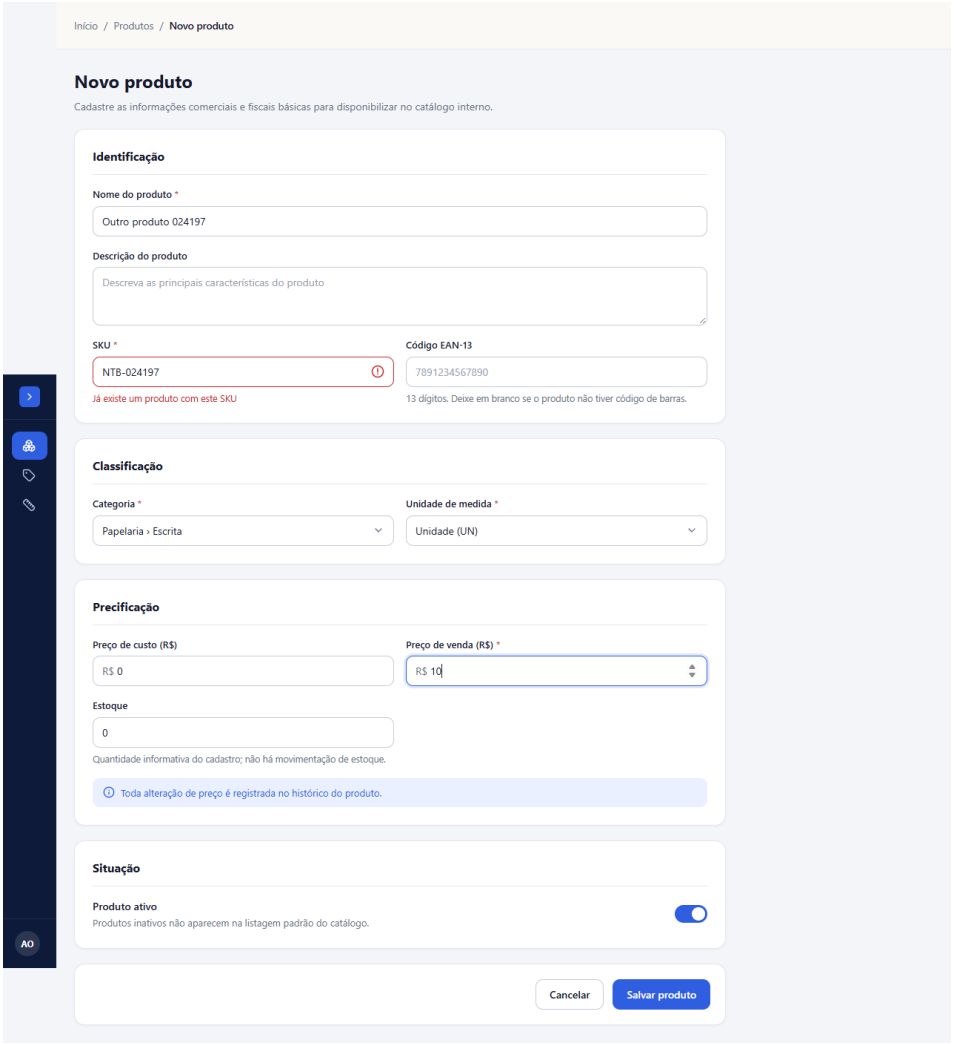


Figura 50 – TN-07: Sku duplicado

TN-07 · Código que detecta e exibe o SKU duplicado

backend/src/servicos/produtos.servico.js linhas 33–34

```
33   const skuExistente = await produtosRepo.buscarPorSku(dados.sku);
34   if (skuExistente) throw new ErroAplicacao('SKU_DUPLICADO');
```

frontend/src/componentes/FormularioProduto.vue linhas 84–87 — Tela: mensagem no campo SKU

```
84 const mensagemErroSku = computed(() => {
85   if (!props.erroServidor) return '';
86   return props.erroServidor.codigo === 'SKU_DUPLICADO' ? props.erroServidor.message : '';
87 });
```

Figura 51 – TN-07: Codigo

TN-08 — Cadastrar categoria sem nome	Aprovado

Dados / passos utilizados	Tela /categorias · "Nova categoria" · salvar com nome vazio
Resultado esperado	Mensagem "Informe o nome da categoria" e categoria não criada.
Resultado obtido	Mensagem de validação exibida; modal permaneceu aberto.

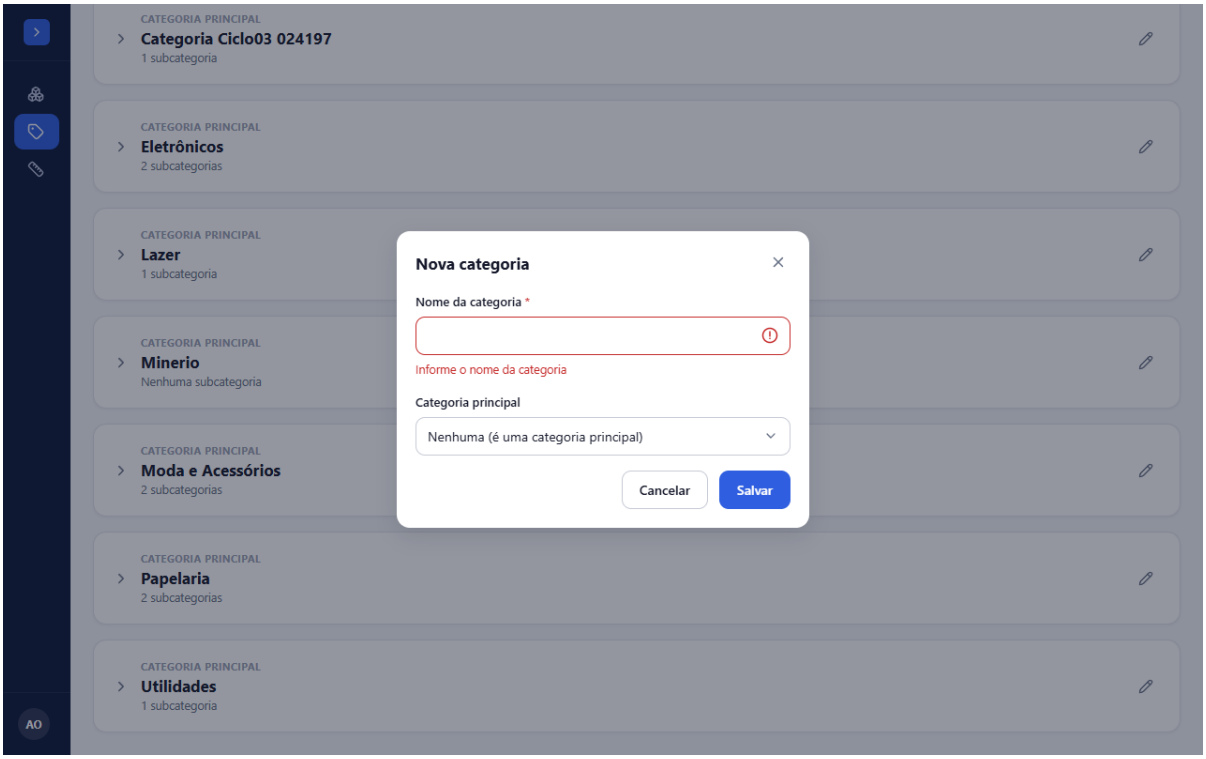


Figura 52 – TN-08: Categoria vazia

TN-08 · Validação do nome da categoria

shared/schemas/categoria.schema.js linhas 3–11

```
3  export const categoriaCriacaoSchema = z.object({
4    nome: z.string().trim().min(1, 'Informe o nome da categoria').max(60),
5    // Selects HTML representam "nenhuma opção" como string vazia; sem o preprocess,
6    // z.coerce.number() converte '' em 0, que falha em .positive() de forma silenciosa no front.
7    categoria_pai_id: z.preprocess(
8      (valor) => (valor === '' || valor === undefined ? null : valor),
9      z.coerce.number().int().positive().nullable()
10   ),
11 }).strict();
```

Figura 53 – TN-08: Codigo

TN-09 — Cadastrar categoria com nome duplicado	**Aprovado**
Dados / passos utilizados	POST /api/categorias {"nome":"Eletrônicos"} (nome já existente)
Resultado esperado	HTTP 409 com código NOME_CATEGORIA_DUPLICADO.
Resultado obtido	HTTP 409 — Já existe uma categoria com este nome (NOME_CATEGORIA_DUPLICADO)

TN-09 · Categoria com nome duplicado

REQUISIÇÃO

```
POST http://localhost:3000/api/categorias

Content-Type: application/json
x-usuario-id: 1

{
  "nome": "Eletrônicos"
}
```

RESPOSTA

```
HTTP 409 Conflict  ·  83 ms

{
  "erro": "Já existe uma categoria com este nome",
  "codigo": "NOME_CATEGORIA_DUPLICADO",
  "detalhes": []
}
```

Figura 54 – TN-09: Categoria duplicada

TN-10 — Cadastrar usuário com e-mail já existente	Aprovado
Dados / passos utilizados	POST /api/auth/cadastro com e-mail demo@sige.com.br
Resultado esperado	HTTP 409 EMAIL_JA_CADASTRADO.
Resultado obtido	HTTP 409 — E-mail já cadastrado (EMAIL_JA_CADASTRADO)

TN-10 · E-mail já cadastrado

REQUISIÇÃO

```
POST http://localhost:3000/api/auth/cadastro

Content-Type: application/json

{
  "nome": "Duplicado Teste",
  "email": "demo@sige.com.br",
  "senha": "senha12345"
}
```

RESPOSTA

```
HTTP 409 Conflict  ·  20 ms

{
  "erro": "E-mail já cadastrado",
  "codigo": "EMAIL_JA_CADASTRADO",
  "detalhes": []
}
```

Figura 55 – TN-10: Email duplicado

TN-11 — Nome de produto duplicado na mesma categoria	Aprovado
Dados / passos utilizados	POST /api/produtos com nome "Notebook Ciclo03 024197" na categoria 8 e SKU novo
Resultado esperado	HTTP 409 NOME_DUPLICADO_NA_CATEGORIA.
Resultado obtido	HTTP 409 — Já existe um produto com este nome nesta categoria (NOME_DUPLICADO_NA_CATEGORIA)

TN-11 · Nome duplicado na categoria

REQUISIÇÃO

POST http://localhost:3000/api/produtos

```
Content-Type: application/json
x-usuario-id: 1

{
  "sku": "DUP-024197",
  "nome": "Notebook Ciclo03 024197",
  "categoria_id": 8,
  "unidade_medida_id": 1,
  "preco_venda": 10
}
```

RESPOSTA

HTTP 409 Conflict · 30 ms

```
{
  "erro": "Já existe um produto com este nome nesta categoria",
  "codigo": "NOME_DUPLICADO_NA_CATEGORIA",
  "detalhes": []
}
```

Figura 56 – TN-11: Nome duplicado

4. Testes de API

O que é: Verificam se os endpoints da API recebem, processam e retornam informações corretamente.

Aplicação neste sistema: Foi montada uma coleção Postman com o CRUD completo (Autenticação, Produtos, Categorias e Unidades de medida), com requisições positivas e negativas e scripts de teste pm.test em cada uma. A coleção foi executada com o Newman, o executor oficial de linha de comando do Postman, contra a API real (http://localhost:3000). Cada print mostra método, URL, cabeçalhos, corpo enviado, status, tempo, corpo da resposta e o resultado dos testes (PASS/FAIL). A coleção está em docs/postman/ e pode ser importada no Postman (File → Import) e reexecutada pelo Collection Runner.

TA-01 — [Autenticação] Login válido (POST)	**Aprovado**
Dados / passos utilizados	Postman · POST http://localhost:3000/api/auth/login · corpo: { "email": "demo@sige.com.br", "senha": "demo1234" }
Resultado esperado	Status code é 200; Retorna o id do usuário; Não expõe a senha; Tempo de resposta < 1000 ms.
Resultado obtido	HTTP 200 OK em 102 ms. Testes do Postman: 4/4 PASS.

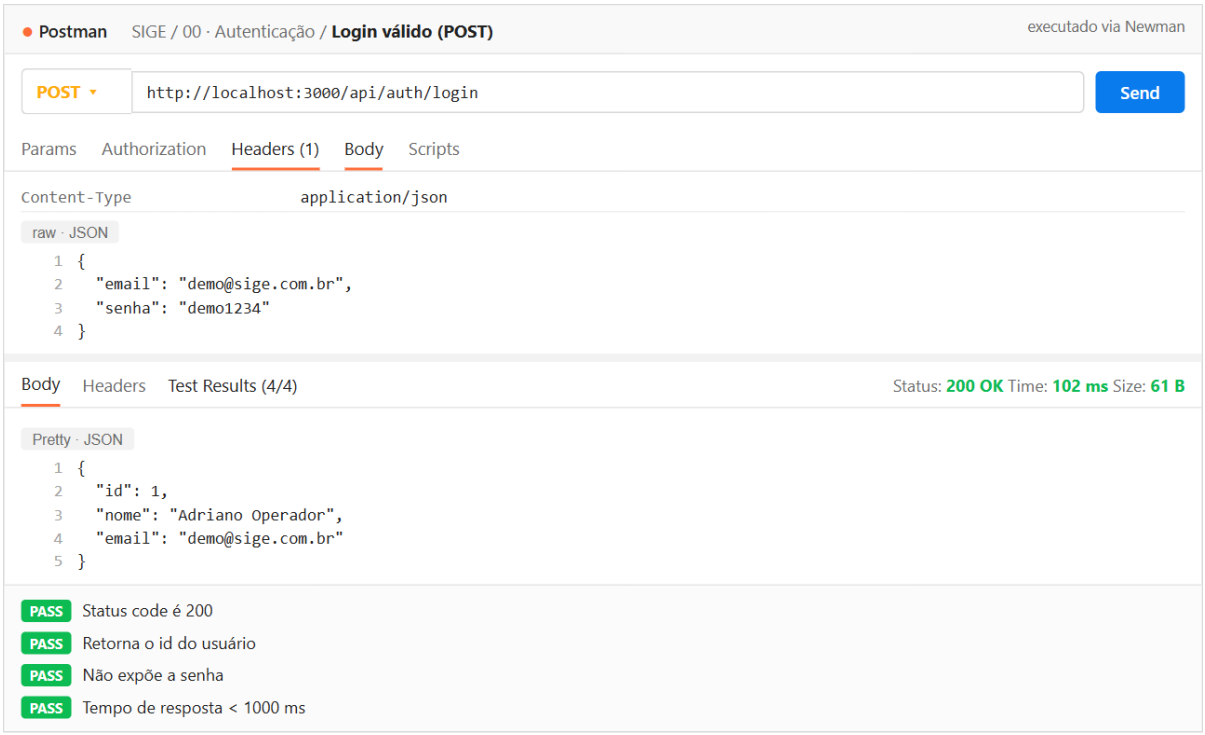


Figura 57 – TA-01: Evidência

TA-01 · Rota da API que atendeu a requisição

backend/src/rotas/auth.rotas.js linhas 28–47

```
28  fastify.post('/api/auth/login', {
29    schema: {
30      tags: TAG,
31      summary: 'Autentica o usuário e devolve seus dados',
32      description: 'Use o `id` retornado no header `x-usuario-id` (botão **Authorize**) para chamar as rotas de
    escrita.',
33      body: deZod(usuarioLoginSchema),
34      response: {
35        200: { description: 'Credenciais válidas', ...usuario },
36        401: respostasErro[401],
37        422: respostasErro[422],
38      },
39    },
40  }, async (request, reply) => {
41    const dados = usuarioLoginSchema.parse(request.body);
42    const usuario = await authServico.login(dados);
43    return reply.status(200).send(usuario);
44  });
45
46  // RF32 — revalidação de sessão ao recarregar a página
47  fastify.get('/api/auth/eu', {
```

Figura 58 – TA-01: Codigo

TA-02 — [Autenticação] Login com senha errada (negativo)	**Aprovado**
Dados / passos utilizados	Postman · POST http://localhost:3000/api/auth/login · corpo: { "email": "demo@sige.com.br", "senha": "senhaErrada" }
Resultado esperado	Status code é 401; Código de erro é CREDENCIAIS_INVALIDAS.
Resultado obtido	HTTP 401 Unauthorized em 90 ms. Testes do Postman: 2/2 PASS.

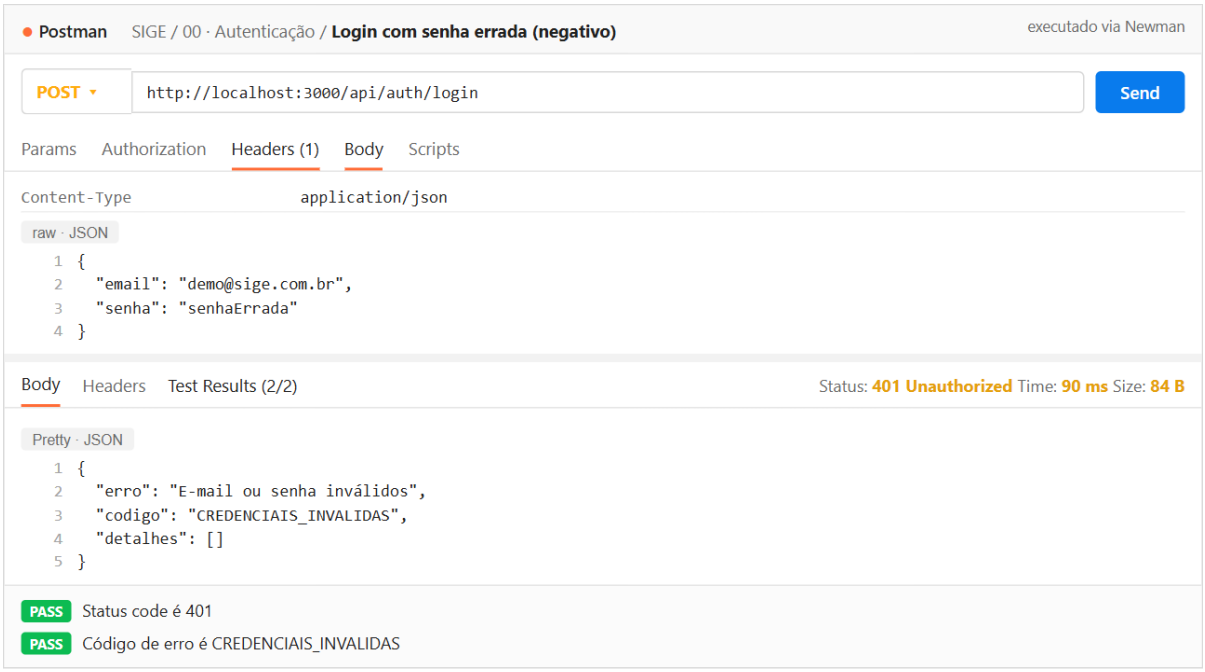


Figura 59 – TA-02: Evidência

TA-03 — [Autenticação] Cadastrar usuário (POST)	Aprovado
Dados / passos utilizados	Postman · POST http://localhost:3000/api/auth/cadastro · corpo: { "nome": "Usuário Postman", "email": "postman.155908@teste.com", "senha": "senha1234" }
Resultado esperado	Status code é 201; E-mail gravado em minúsculas.
Resultado obtido	HTTP 201 Created em 81 ms. Testes do Postman: 2/2 PASS.

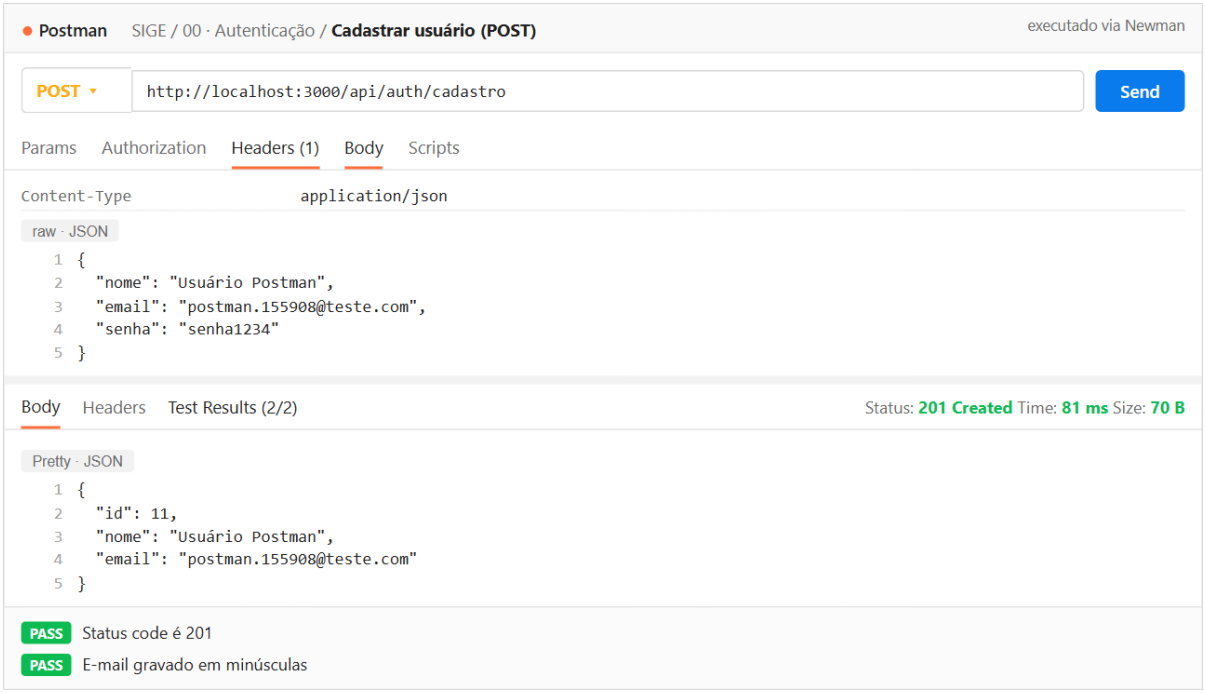


Figura 60 – TA-03: Evidência

TA-04 — [Autenticação] Cadastrar usuário com e-mail repetido (negativo)	Aprovado

Dados / passos utilizados	Postman · POST http://localhost:3000/api/auth/cadastro · corpo: { "nome": "Usuário Postman", "email": "postman.155908@teste.com", "senha": "senha1234" }
Resultado esperado	Status code é 409; Código de erro é EMAIL_JA_CADASTRADO.
Resultado obtido	HTTP 409 Conflict em 7 ms. Testes do Postman: 2/2 PASS.

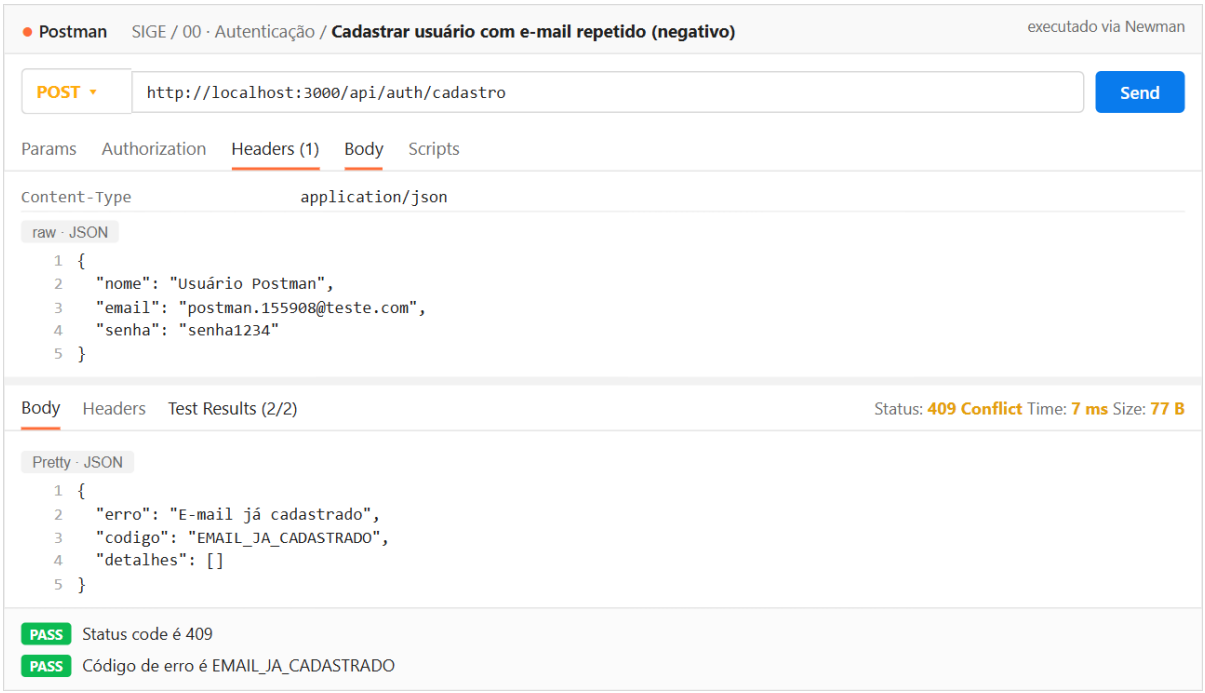


Figura 61 – TA-04: Evidência

TA-05 — [Autenticação] Cadastrar usuário com senha curta (negativo)	**Aprovado**
Dados / passos utilizados	Postman · POST http://localhost:3000/api/auth/cadastro · corpo: { "nome": "Senha Curta", "email": "curta.155908@teste.com", "senha": "123" }
Resultado esperado	Status code é 422; Código de erro é VALIDACAO.
Resultado obtido	HTTP 422 Unprocessable Entity em 11 ms. Testes do Postman: 2/2 PASS.

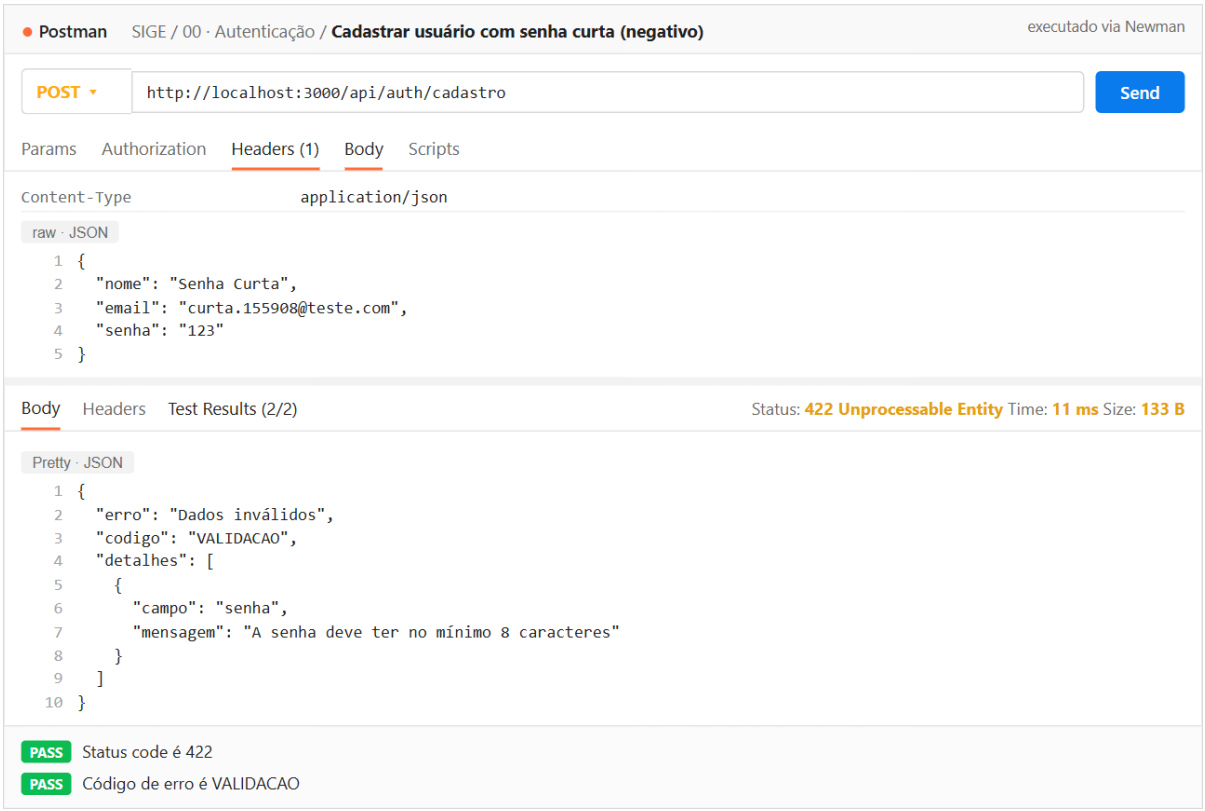


Figura 62 – TA-05: Evidência

TA-06 — [Produtos (CRUD)] CREATE · Cadastrar produto (POST)	Aprovado
Dados / passos utilizados	Postman · POST http://localhost:3000/api/produtos · x-usuario-id: 1 · corpo: { "sku": "PMN-155908", "nome": "Teclado Postman 155908", "descricao": "Criado pela coleção Postman", "ean13": "7891559085555", "categoria_id": 8, "unidade_medida_id": 1, "preco_custo": 80, "preco_venda": 149.9, "estoque": 25 }
Resultado esperado	Status code é 201; Retorna o produto com id e situação ativo; Preço de venda gravado.
Resultado obtido	HTTP 201 Created em 20 ms. Testes do Postman: 3/3 PASS.

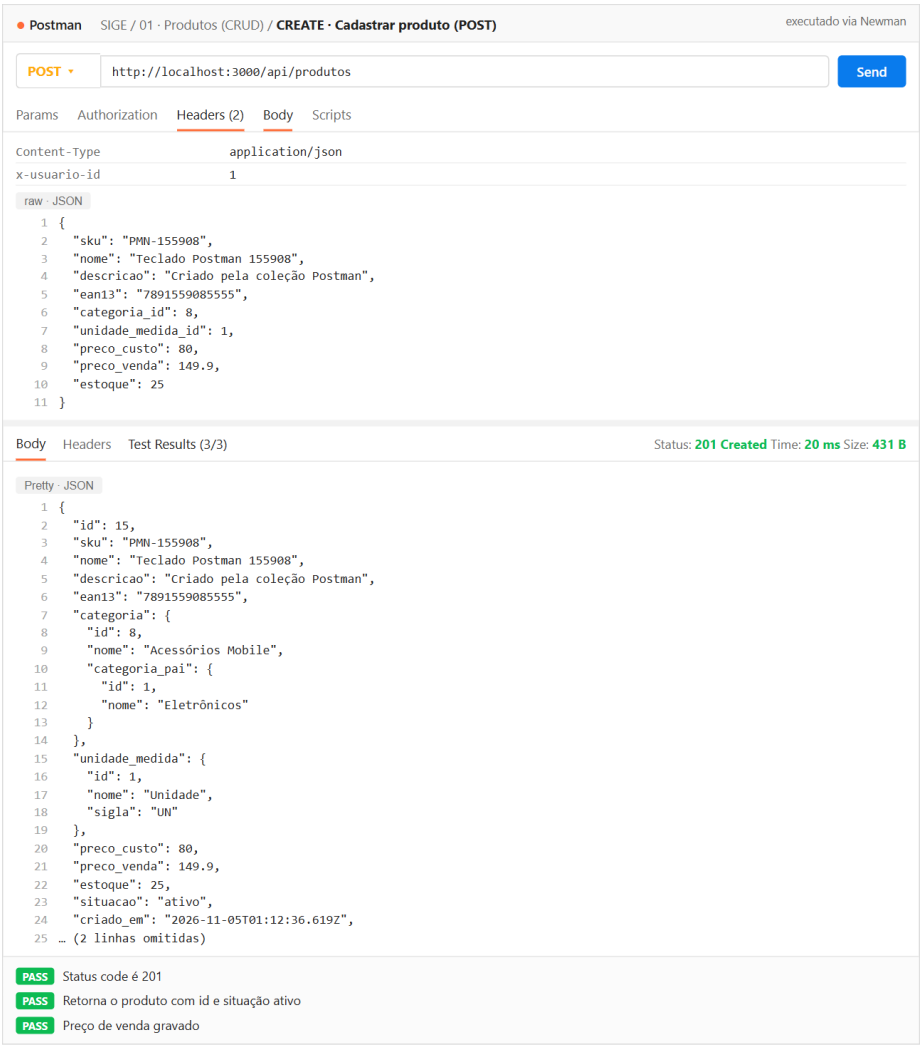


Figura 63 – TA-06: Evidência

TA-06 · Rota da API que atendeu a requisição

backend/src/rotas/produtos.rotas.js linhas 83–103

```
83  fastify.post('/api/produtos', {
84    schema: {
85      tags: TAG,
86      summary: 'Cadastra um produto',
87      security: SEGURANCA_USUARIO,
88      body: deZod(produtoCriacaoSchema),
89      response: {
90        201: { description: 'Produto criado', ...produto },
91        401: respostasErro[401],
92        409: respostasErro[409],
93        422: respostasErro[422],
94      },
95    },
96  }, async (request, reply) => {
97    const usuarioId = await exigirUsuarioAutenticado(request);
98    const dados = produtoCriacaoSchema.parse(request.body);
99    const produto = await produtosServico.cadastrar(dados, usuarioId);
100   return reply.status(201).send(produto);
101 });
102
103 // RF02, RF13
```

Figura 64 – TA-06: Codigo

TA-07 — [Produtos (CRUD)] CREATE · Sem autenticação (negativo)	Aprovado
Dados / passos utilizados	Postman · POST http://localhost:3000/api/produtos · corpo: { "sku": "SEMAUTH-1", "nome": "Sem auth", "categoria_id": 8, "unidade_medida_id": 1, "preco_venda": 10 }
Resultado esperado	Status code é 401; Código de erro é NAO_AUTENTICADO.
Resultado obtido	HTTP 401 Unauthorized em 8 ms. Testes do Postman: 2/2 PASS.

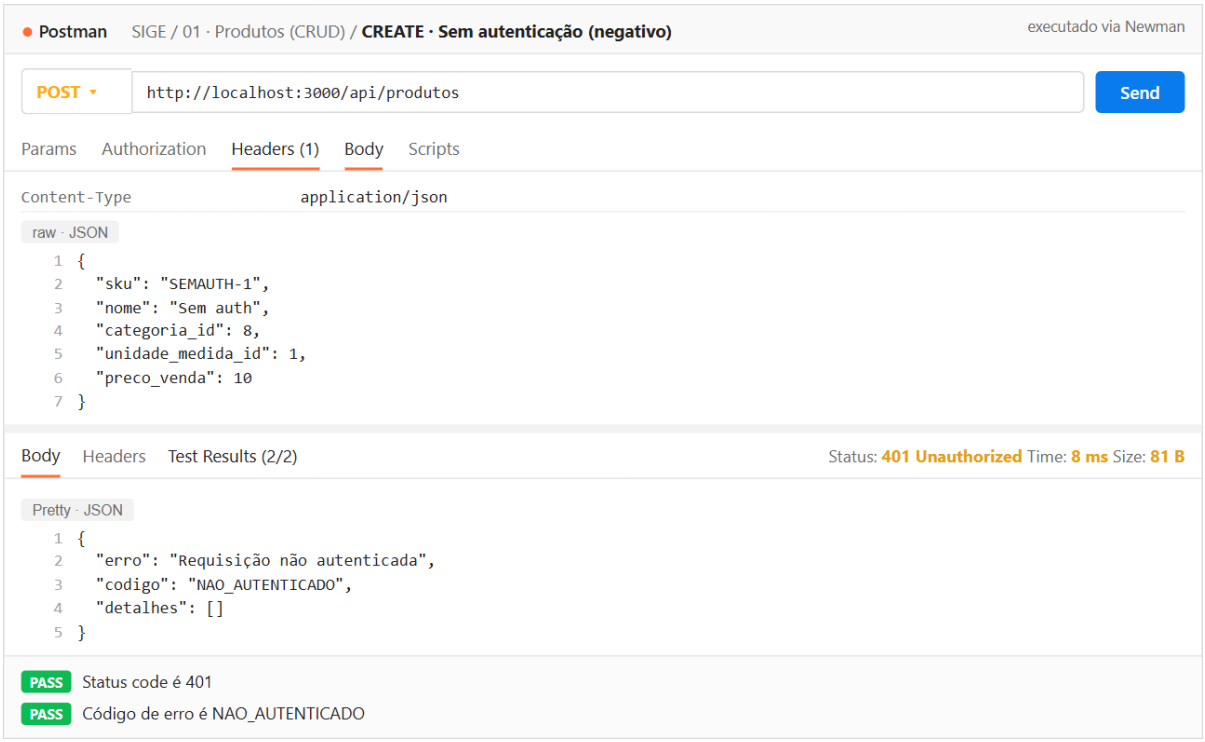


Figura 65 – TA-07: Evidência

TA-08 — [Produtos (CRUD)] CREATE · Dados inválidos (negativo)	Aprovado
Dados / passos utilizados	Postman · POST http://localhost:3000/api/produtos · x-usuario-id: 1 · corpo: { "sku": "minúsculo", "nome": "", "categoria_id": 8, "unidade_medida_id": 1, "preco_venda": -5 }
Resultado esperado	Status code é 422; Código de erro é VALIDACAO; Detalha os campos inválidos.
Resultado obtido	HTTP 422 Unprocessable Entity em 10 ms. Testes do Postman: 3/3 PASS.

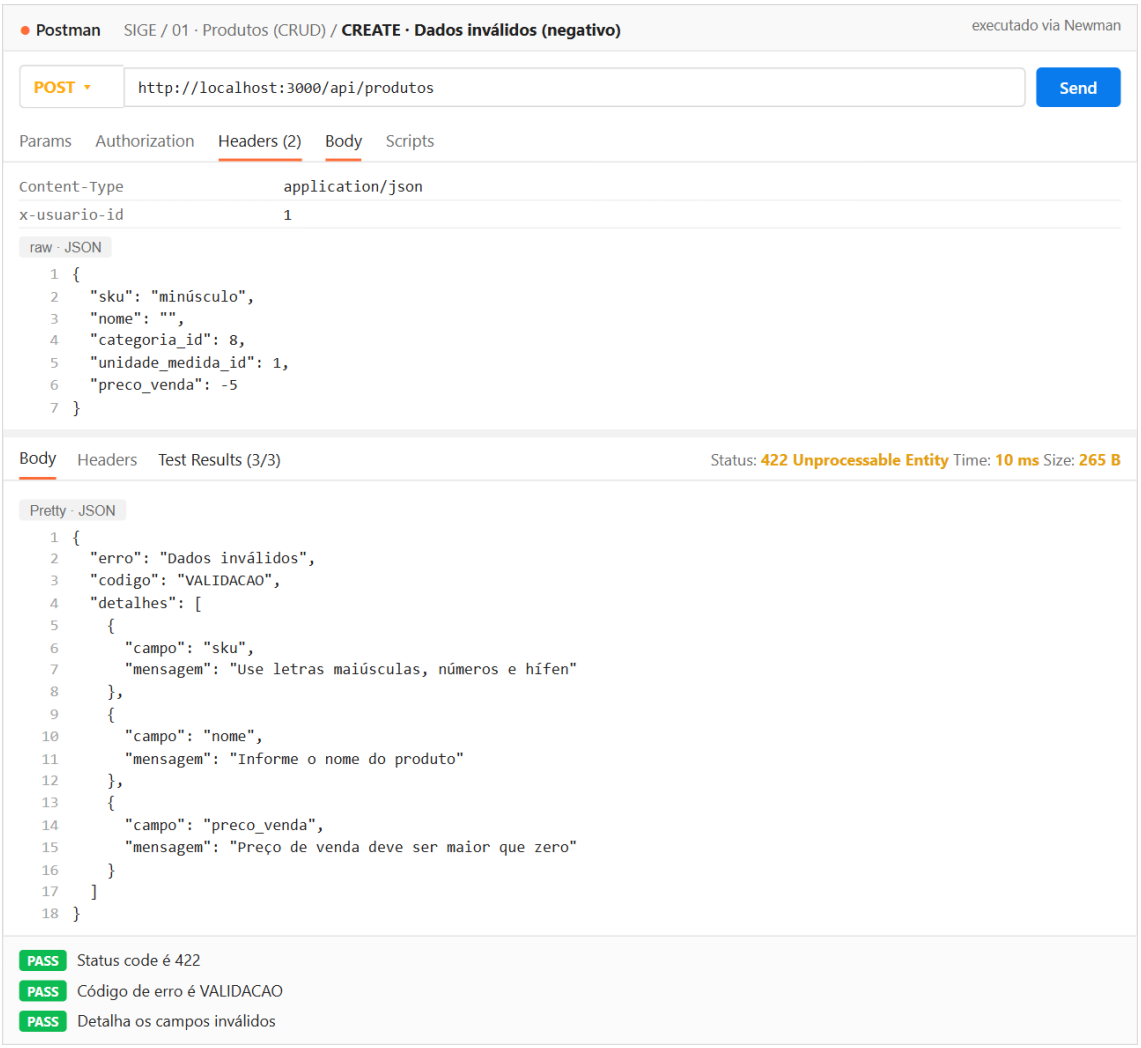


Figura 66 – TA-08: Evidência

TA-09 — [Produtos (CRUD)] CREATE · SKU duplicado (negativo)	Aprovado
Dados / passos utilizados	Postman · POST http://localhost:3000/api/produtos · x-usuario-id: 1 · corpo: { "sku": "PMN-155908", "nome": "Outro nome 155908", "categoria_id": 8, "unidade_medida_id": 1, "preco_venda": 10 }
Resultado esperado	Status code é 409; Código de erro é SKU_DUPLICADO.
Resultado obtido	HTTP 409 Conflict em 10 ms. Testes do Postman: 2/2 PASS.

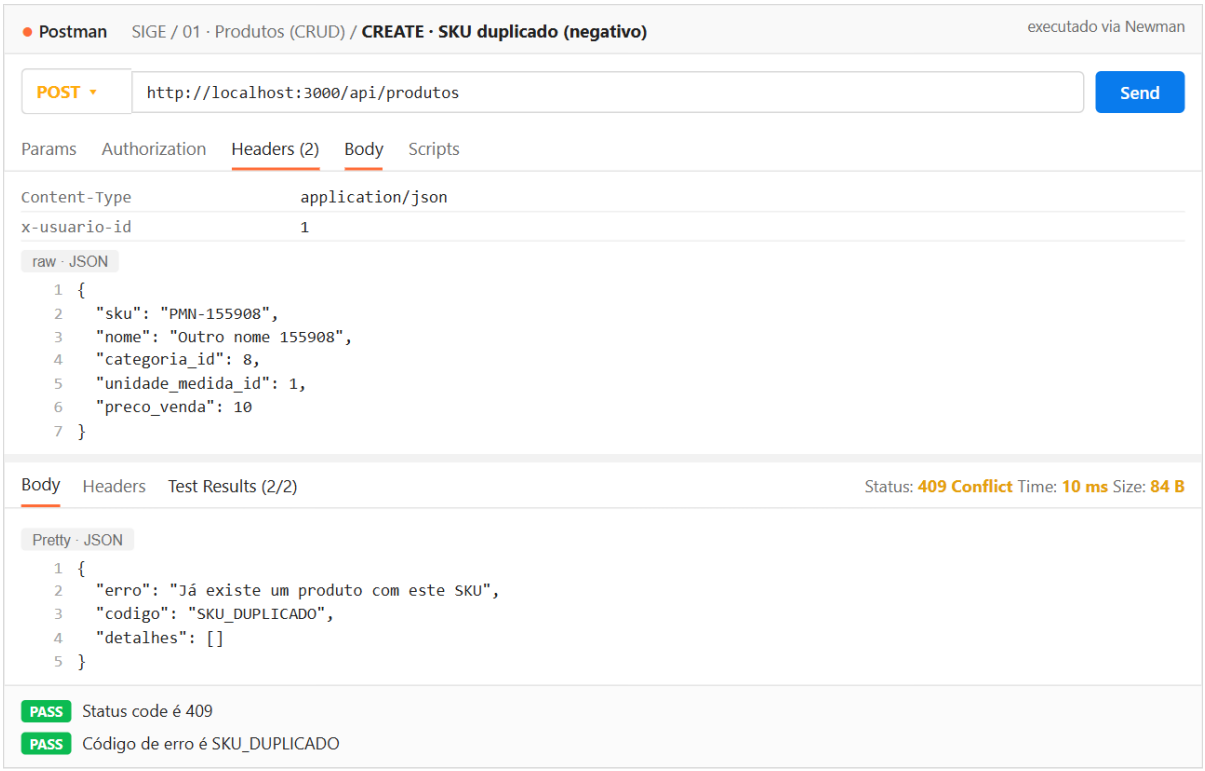


Figura 67 – TA-09: Evidência

TA-10 — [Produtos (CRUD)] CREATE · EAN-13 com dígito inválido (negativo)	**Aprovado**
Dados / passos utilizados	Postman · POST http://localhost:3000/api/produtos · x-usuario-id: 1 · corpo: { "sku": "EAN-155908", "nome": "EAN errado 155908", "ean13": "7891234567890", "categoria_id": 8, "unidade_medida_id": 1, "preco_venda": 10 }
Resultado esperado	Status code é 422; Mensagem do dígito verificador.
Resultado obtido	HTTP 422 Unprocessable Entity em 9 ms. Testes do Postman: 2/2 PASS.

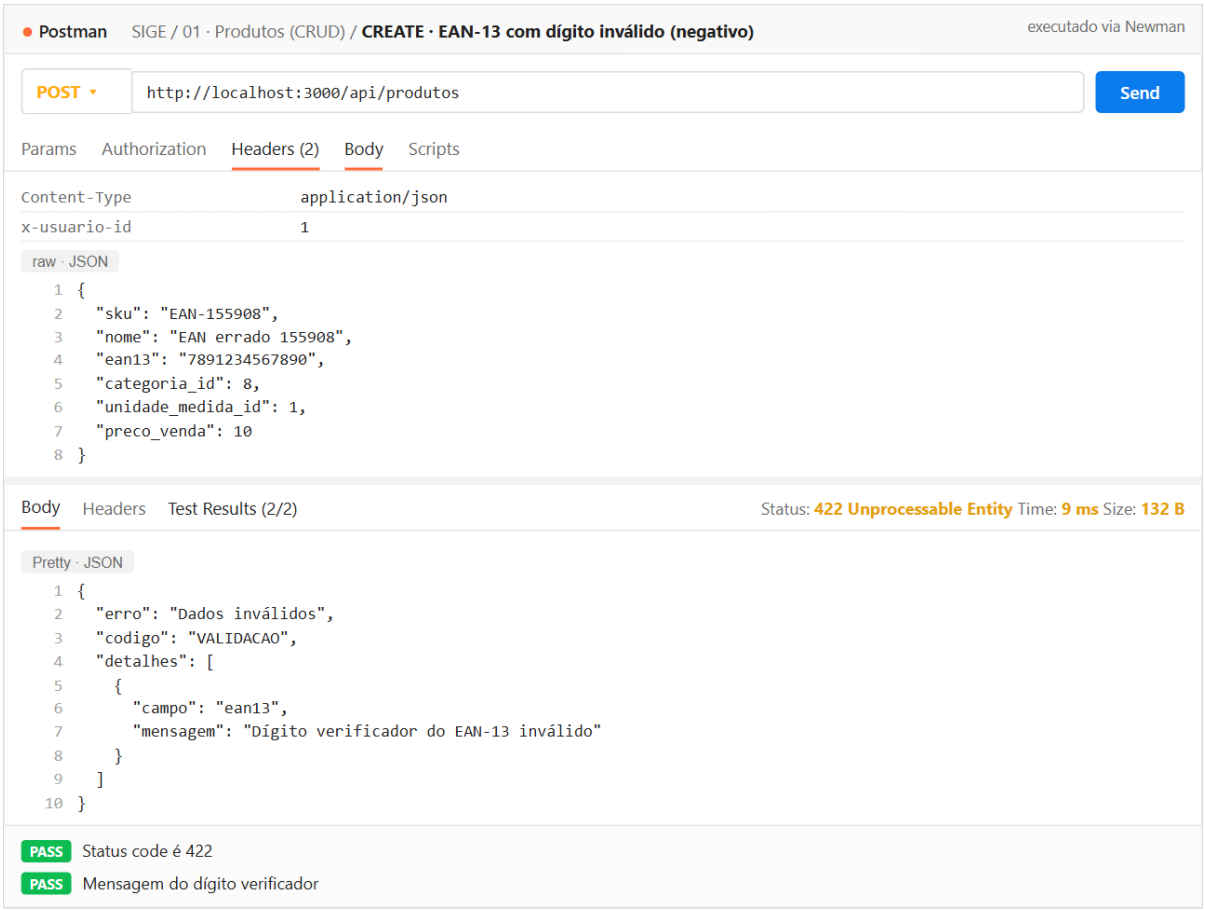


Figura 68 – TA-10: Evidência

TA-11 — [Produtos (CRUD)] READ · Listar produtos paginado (GET)	Aprovado
Dados / passos utilizados	Postman · GET http://localhost:3000/api/produtos?pagina=1&limite=5
Resultado esperado	Status code é 200; Estrutura dados + paginacao; Tempo de resposta < 1000 ms.
Resultado obtido	HTTP 200 OK em 10 ms. Testes do Postman: 3/3 PASS.

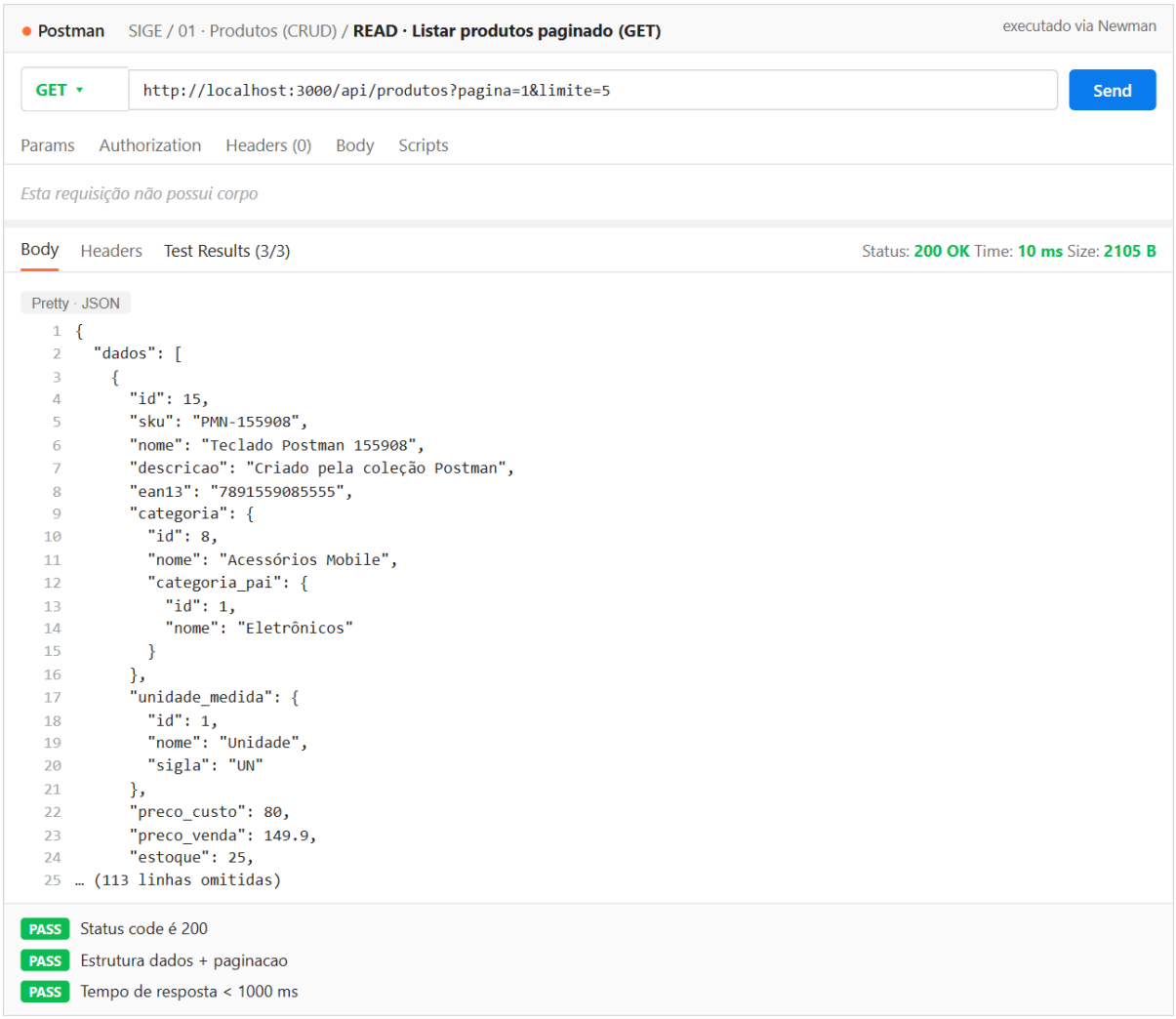


Figura 69 – TA-11: Evidência

TA-12 — [Produtos (CRUD)] READ · Pesquisar por SKU (GET)	**Aprovado**
Dados / passos utilizados	Postman · GET http://localhost:3000/api/produtos?sku=PMN-155908
Resultado esperado	Status code é 200; Encontra exatamente o produto criado.
Resultado obtido	HTTP 200 OK em 12 ms. Testes do Postman: 2/2 PASS.

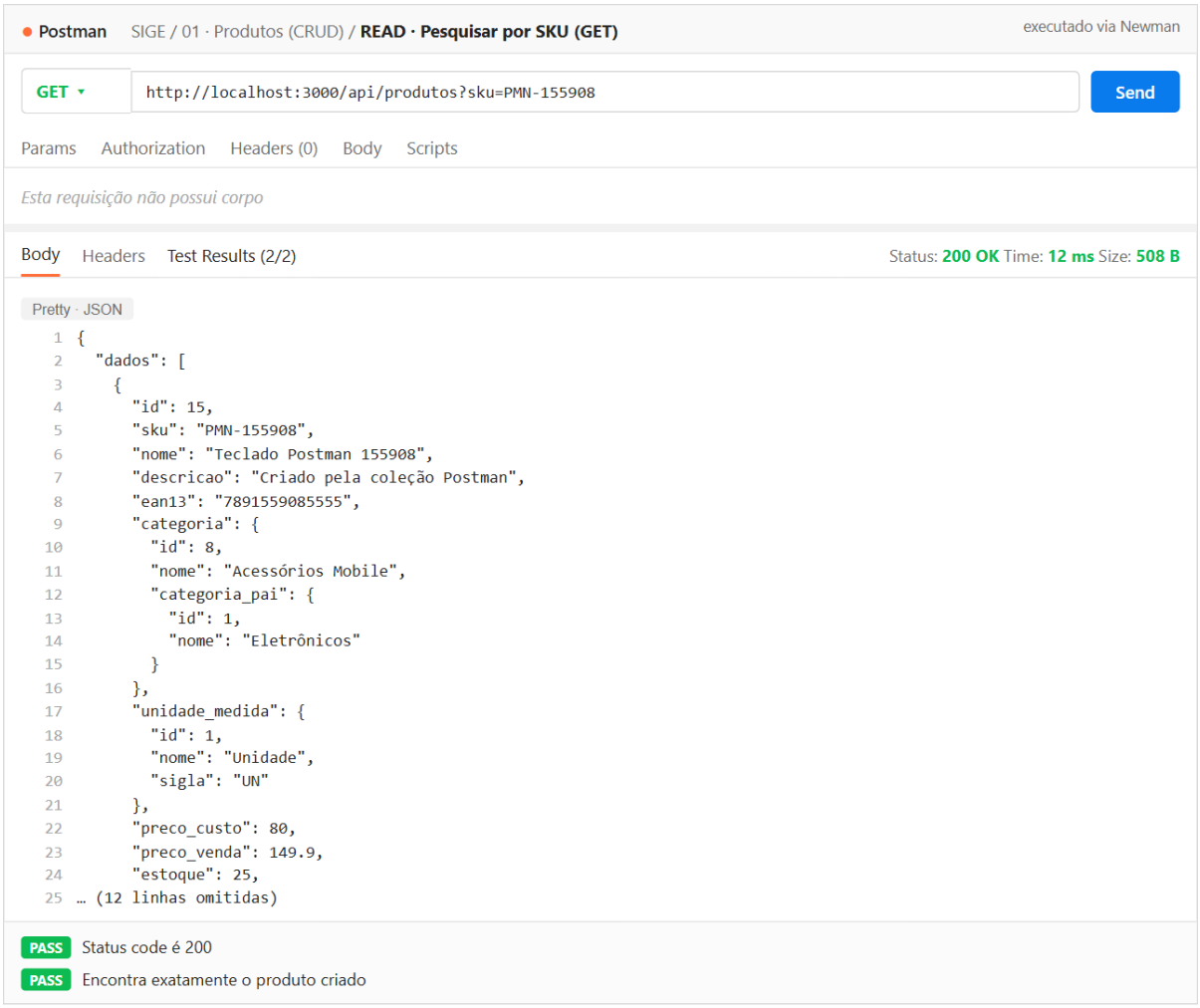


Figura 70 – TA-12: Evidência

TA-13 — [Produtos (CRUD)] READ · Buscar por ID (GET)	**Aprovado**
Dados / passos utilizados	Postman · GET http://localhost:3000/api/produtos/15
Resultado esperado	Status code é 200; ID confere.
Resultado obtido	HTTP 200 OK em 8 ms. Testes do Postman: 2/2 PASS.

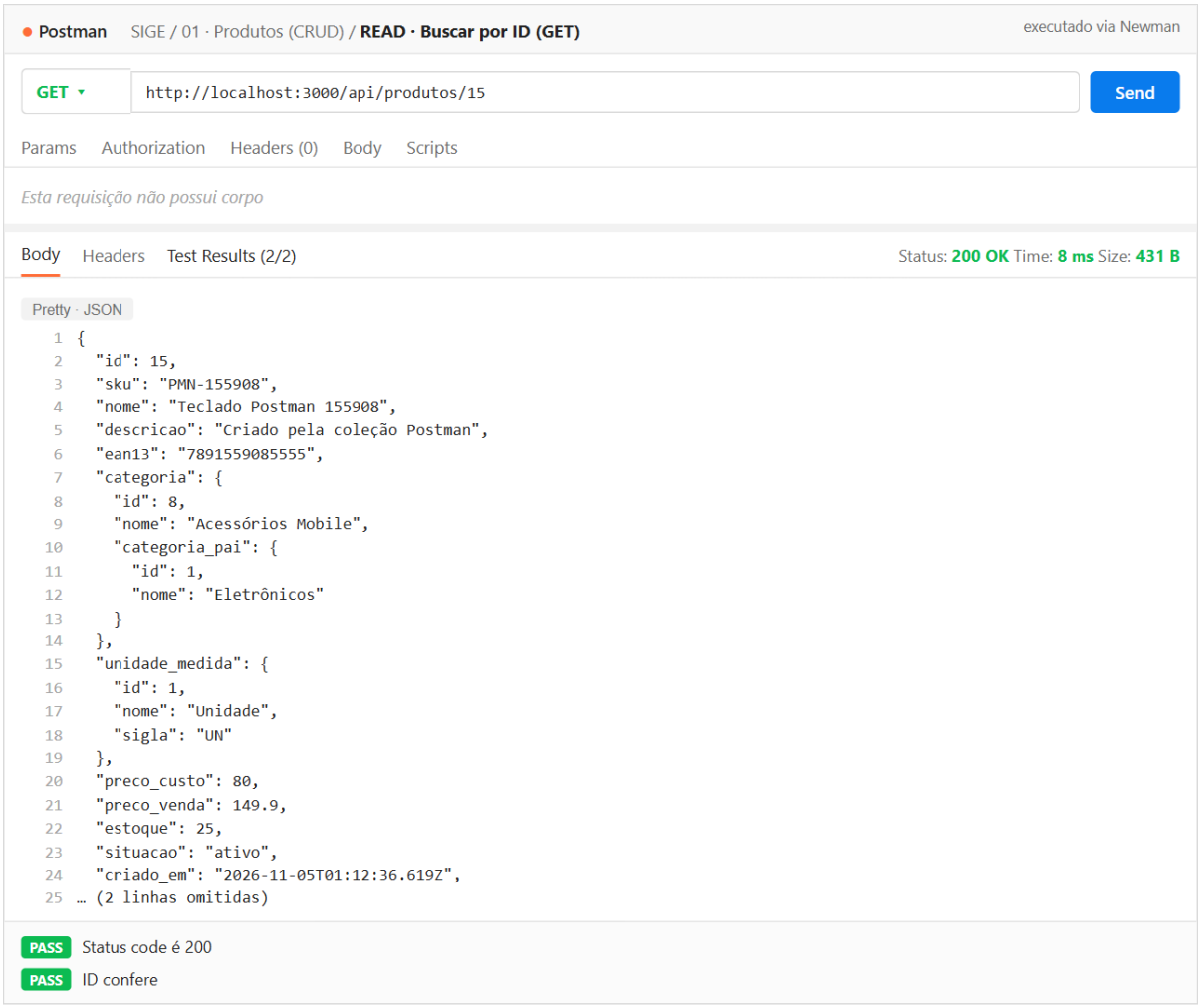


Figura 71 – TA-13: Evidência

TA-14 — [Produtos (CRUD)] READ · ID inexistente (negativo)	**Aprovado**
Dados / passos utilizados	Postman · GET http://localhost:3000/api/produtos/999999
Resultado esperado	Status code é 404; Código de erro é PRODUTO_NAO_ENCONTRADO.
Resultado obtido	HTTP 404 Not Found em 25 ms. Testes do Postman: 2/2 PASS.



Figura 72 – TA-14: Evidência

TA-15 — [Produtos (CRUD)] READ · ID inválido (negativo)	Aprovado
Dados / passos utilizados	Postman · GET http://localhost:3000/api/produtos/abc
Resultado esperado	Status code é 400.
Resultado obtido	HTTP 400 Bad Request em 4 ms. Testes do Postman: 1/1 PASS.



Figura 73 – TA-15: Evidência

TA-16 — [Produtos (CRUD)] UPDATE · Alterar preço e estoque (PUT)	Aprovado
Dados / passos utilizados	Postman · PUT http://localhost:3000/api/produtos/15 · x-usuario-id: 1 · corpo: { "preco_venda": 139.9, "estoque": 30 }
Resultado esperado	Status code é 200; Novo preço retornado; SKU não mudou.
Resultado obtido	HTTP 200 OK em 16 ms. Testes do Postman: 3/3 PASS.

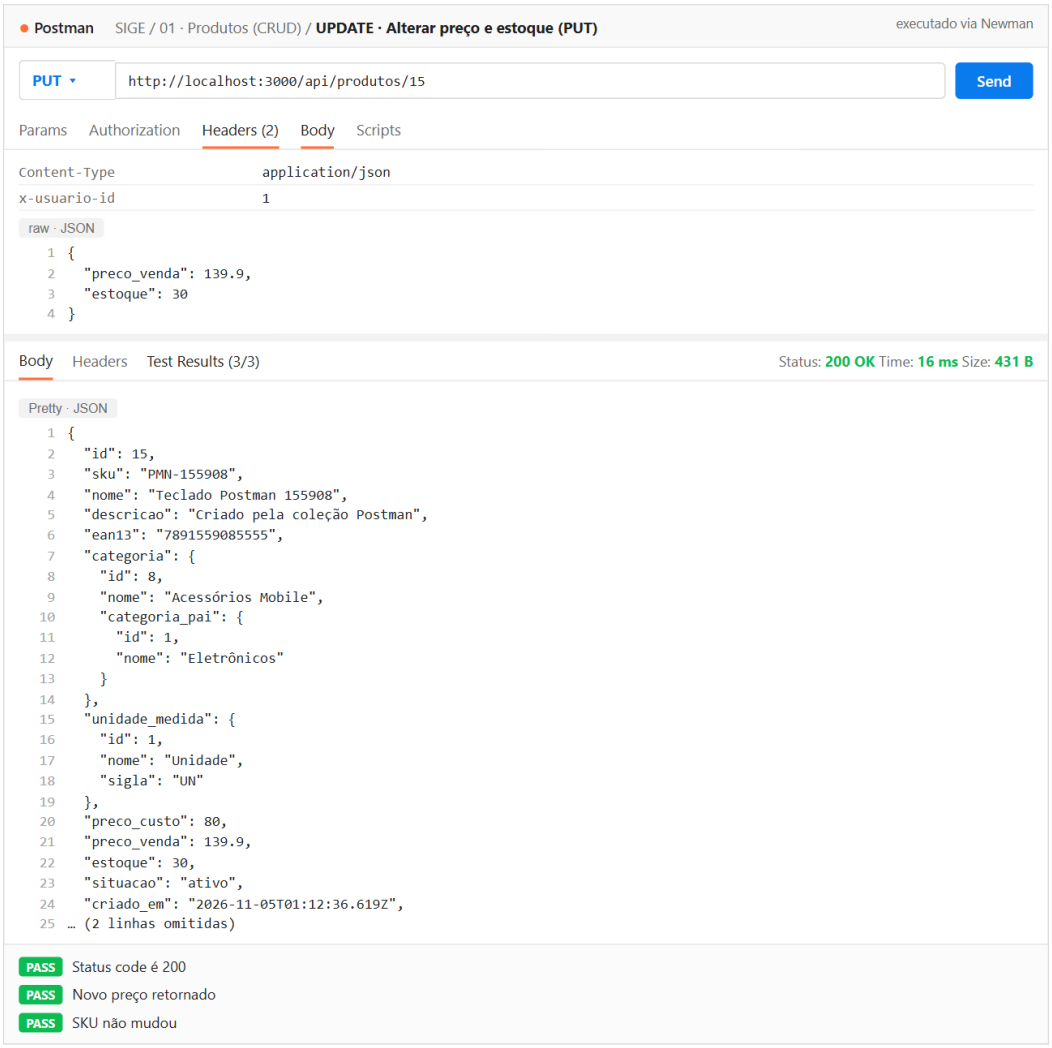


Figura 74 – TA-16: Evidência

TA-17 — [Produtos (CRUD)] UPDATE · Tentar alterar SKU (negativo)	**Aprovado**
Dados / passos utilizados	Postman · PUT http://localhost:3000/api/produtos/15 · x-usuario-id: 1 · corpo: { "sku": "NOVO-SKU" }
Resultado esperado	Status 4xx (SKU imutável).
Resultado obtido	HTTP 422 Unprocessable Entity em 8 ms. Testes do Postman: 1/1 PASS.

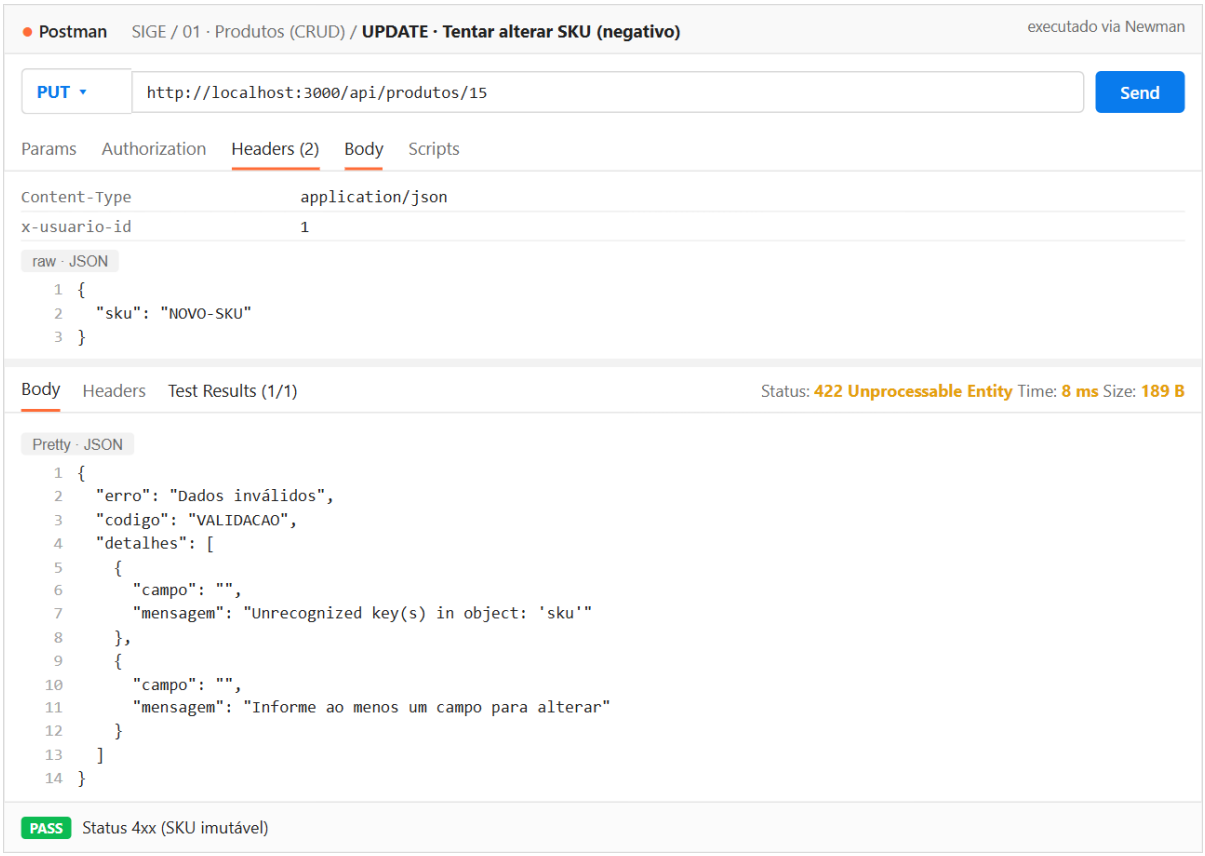


Figura 75 – TA-17: Evidência

TA-18 — [Produtos (CRUD)] UPDATE · Preço de venda zero (negativo)	Aprovado
Dados / passos utilizados	Postman · PUT http://localhost:3000/api/produtos/15 · x-usuario-id: 1 · corpo: { "preco_venda": 0 }
Resultado esperado	Status code é 422; Código de erro é VALIDACAO.
Resultado obtido	HTTP 422 Unprocessable Entity em 7 ms. Testes do Postman: 2/2 PASS.

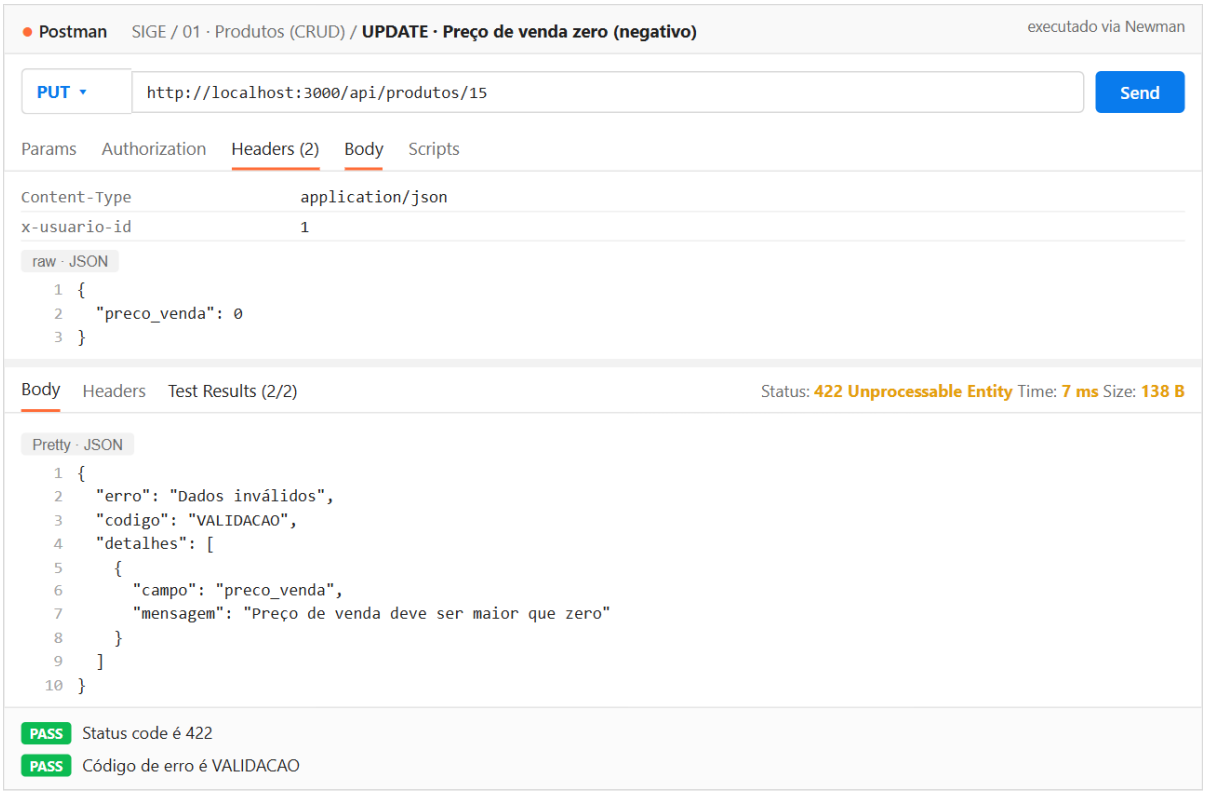


Figura 76 – TA-18: Evidência

TA-19 — [Produtos (CRUD)] READ · Histórico de preço (GET)	**Aprovado**
Dados / passos utilizados	Postman · GET http://localhost:3000/api/produtos/15/precos
Resultado esperado	Status code é 200; 3 registros: venda inicial, custo inicial e alteração; Alteração preserva valor anterior.
Resultado obtido	HTTP 200 OK em 10 ms. Testes do Postman: 3/3 PASS.

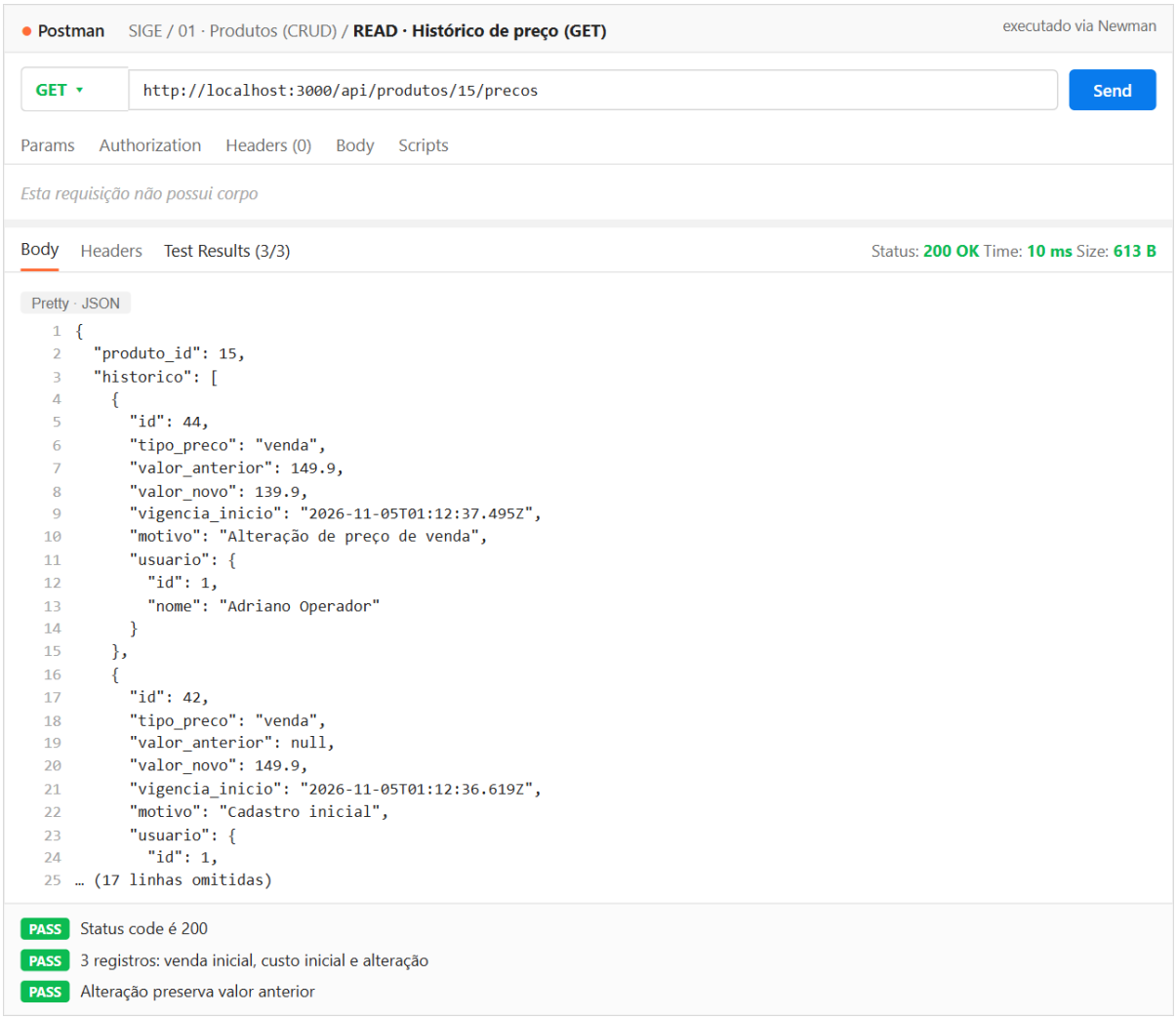


Figura 77 – TA-19: Evidência

TA-20 — [Produtos (CRUD)] DELETE lógico · Inativar (PATCH)	**Aprovado**
Dados / passos utilizados	Postman · PATCH http://localhost:3000/api/produtos/15/inativar · x-usuario-id: 1
Resultado esperado	Status code é 200; Situação inativo.
Resultado obtido	HTTP 200 OK em 15 ms. Testes do Postman: 2/2 PASS.

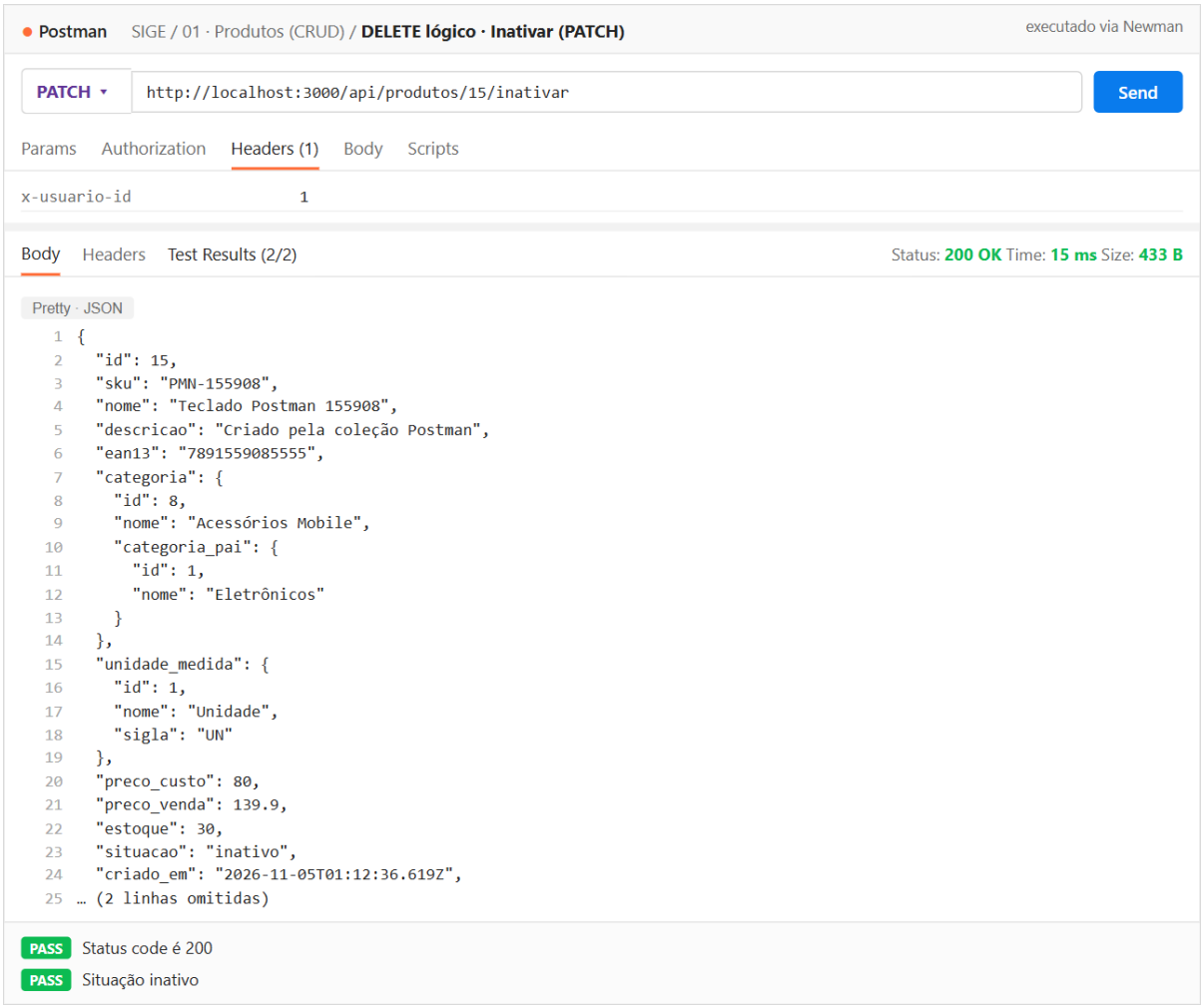


Figura 78 – TA-20: Evidência

TA-21 — [Produtos (CRUD)] DELETE lógico · Inativar de novo (negativo)	**Aprovado**
Dados / passos utilizados	Postman · PATCH http://localhost:3000/api/produtos/15/inativar · x-usuario-id: 1
Resultado esperado	Status code é 409; Código de erro é PRODUTO_JA_INATIVO.
Resultado obtido	HTTP 409 Conflict em 11 ms. Testes do Postman: 2/2 PASS.



Figura 79 – TA-21: Evidência

TA-22 — [Produtos (CRUD)] READ · Listagem padrão oculta inativos (GET)	Aprovado
Dados / passos utilizados	Postman · GET http://localhost:3000/api/produtos?sku=PMN-155908
Resultado esperado	Status code é 200; Produto inativo não aparece.
Resultado obtido	HTTP 200 OK em 7 ms. Testes do Postman: 2/2 PASS.

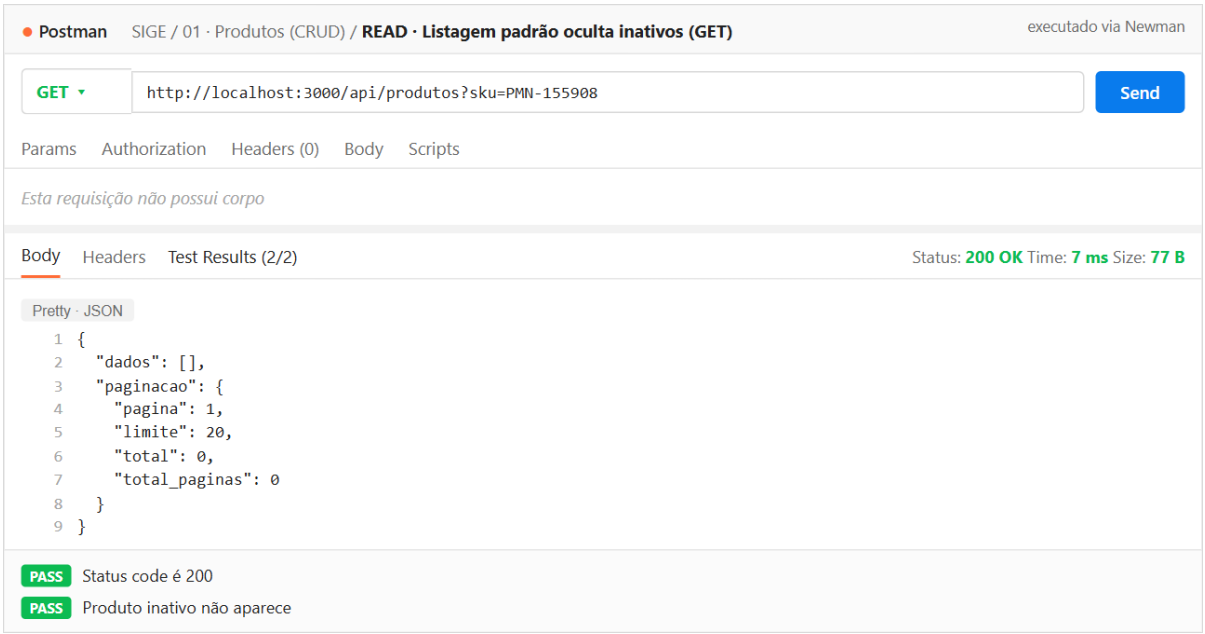


Figura 80 – TA-22: Evidência

TA-23 — [Produtos (CRUD)] UPDATE · Reativar (PATCH)	Aprovado
Dados / passos utilizados	Postman · PATCH http://localhost:3000/api/produtos/15/reativar · x-usuario-id: 1
Resultado esperado	Status code é 200; Situação ativo.
Resultado obtido	HTTP 200 OK em 13 ms. Testes do Postman: 2/2 PASS.

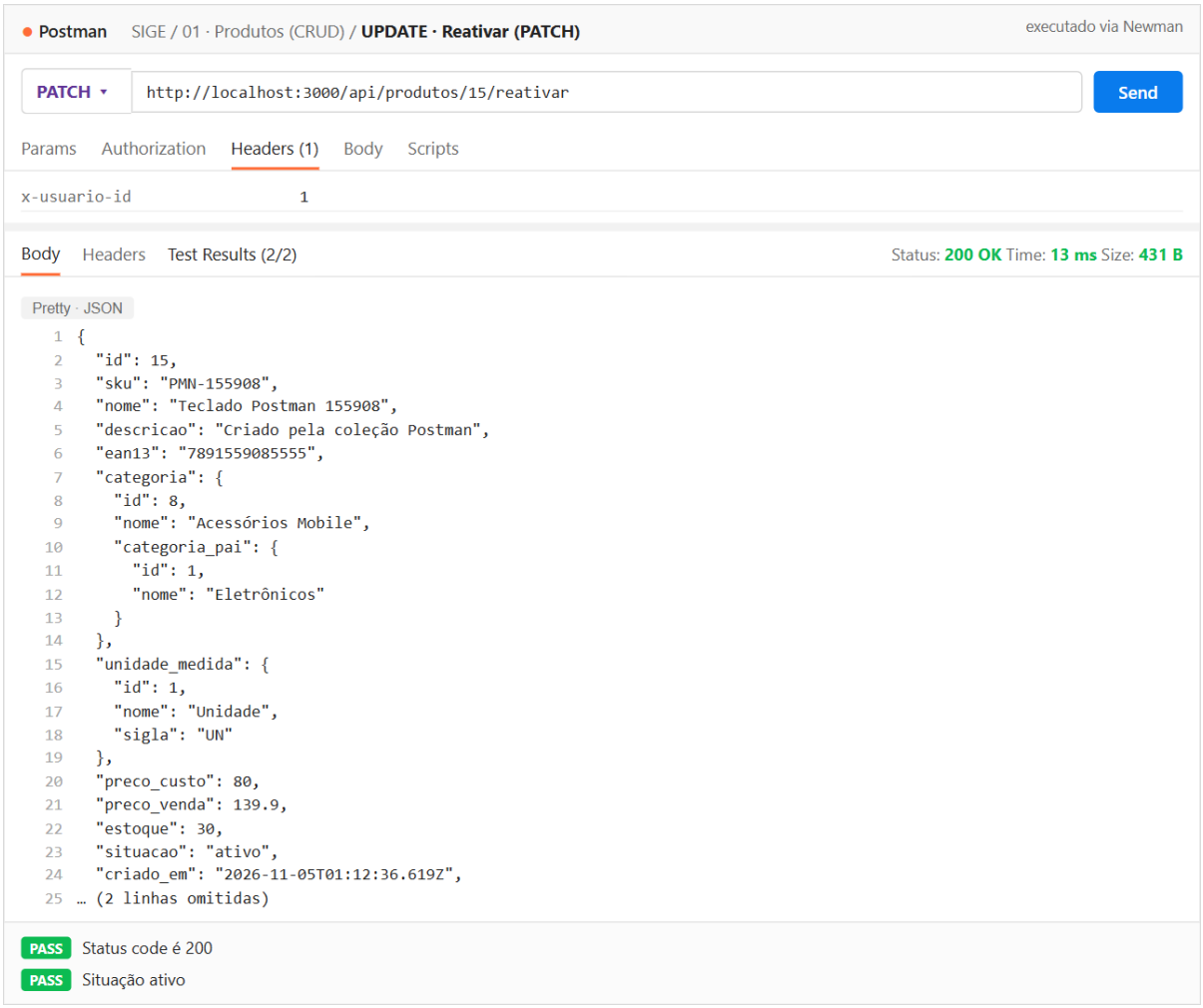


Figura 81 – TA-23: Evidência

TA-24 — [Produtos (CRUD)] DELETE · Exclusão física bloqueada (negativo)	**Aprovado**
Dados / passos utilizados	Postman · DELETE http://localhost:3000/api/produtos/15 · x-usuario-id: 1
Resultado esperado	Status code é 409; Código de erro é EXCLUSAO_NAO_PERMITIDA.
Resultado obtido	HTTP 409 Conflict em 12 ms. Testes do Postman: 2/2 PASS.



Figura 82 – TA-24: Evidência

TA-25 — [Categorias (CRUD)] CREATE · Cadastrar categoria (POST)	Aprovado
Dados / passos utilizados	Postman · POST http://localhost:3000/api/categorias · x-usuario-id: 1 · corpo: { "nome": "Categoria Postman 155908", "categoria_pai_id": null }
Resultado esperado	Status code é 201; Retorna id.
Resultado obtido	HTTP 201 Created em 11 ms. Testes do Postman: 2/2 PASS.

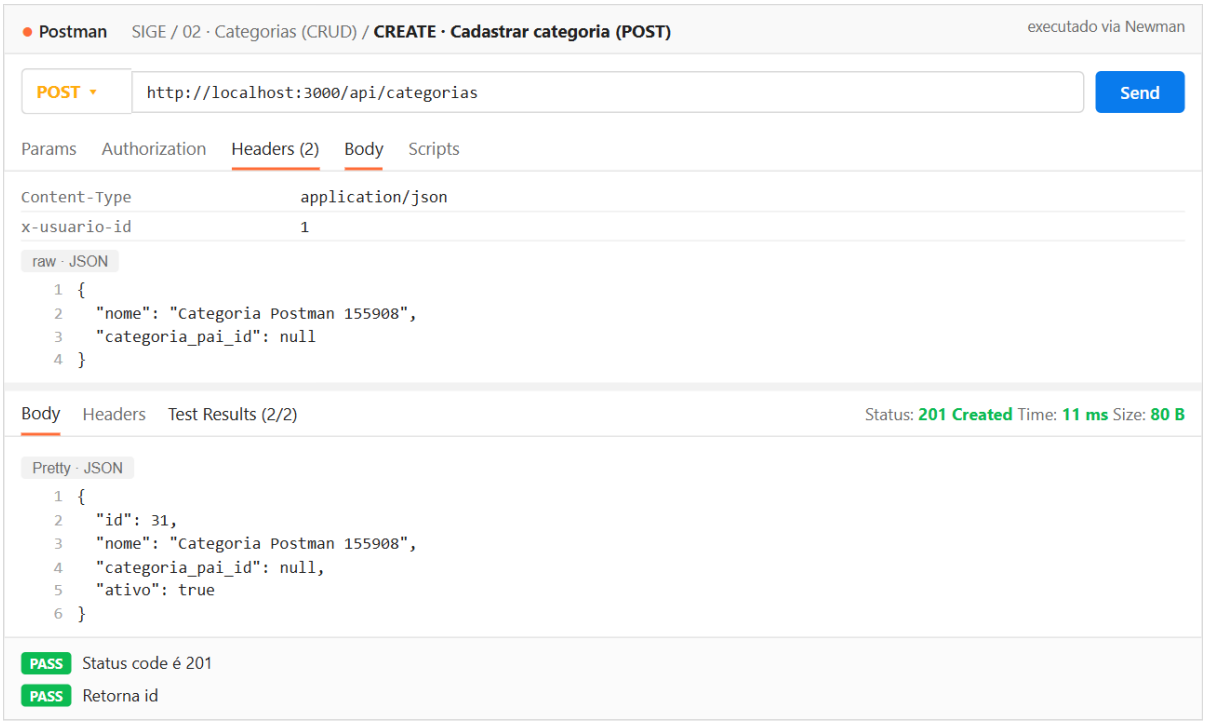


Figura 83 – TA-25: Evidência

TA-25 · Rota da API que atendeu a requisição

backend/src/rotas/categorias.rotas.js linhas 44–63

```
44  fastify.post('/api/categorias', {
45    schema: {
46      tags: TAG,
47      summary: 'Cadastra uma categoria',
48      security: SEGURANCA_USUARIO,
49      body: deZod(categoriaCriacaoSchema),
50      response: {
51        201: { description: 'Categoria criada', ...categoria },
52        401: respostasErro[401],
53        409: respostasErro[409],
54        422: respostasErro[422],
55      },
56    },
57  }, async (request, reply) => {
58    await exigirUsuarioAutenticado(request);
59    const dados = categoriaCriacaoSchema.parse(request.body);
60    const categoria = await categoriasServico.cadastrar(dados);
61    return reply.status(201).send(categoria);
62  });
63
```

Figura 84 – TA-25: Codigo

TA-26 — [Categorias (CRUD)] CREATE · Nome duplicado (negativo)	Aprovado
Dados / passos utilizados	Postman · POST http://localhost:3000/api/categorias · x-usuario-id: 1 · corpo: { "nome": "Categoria Postman 155908" }
Resultado esperado	Status code é 409; Código de erro é NOME_CATEGORIA_DUPLICADO.
Resultado obtido	HTTP 409 Conflict em 9 ms. Testes do Postman: 2/2 PASS.

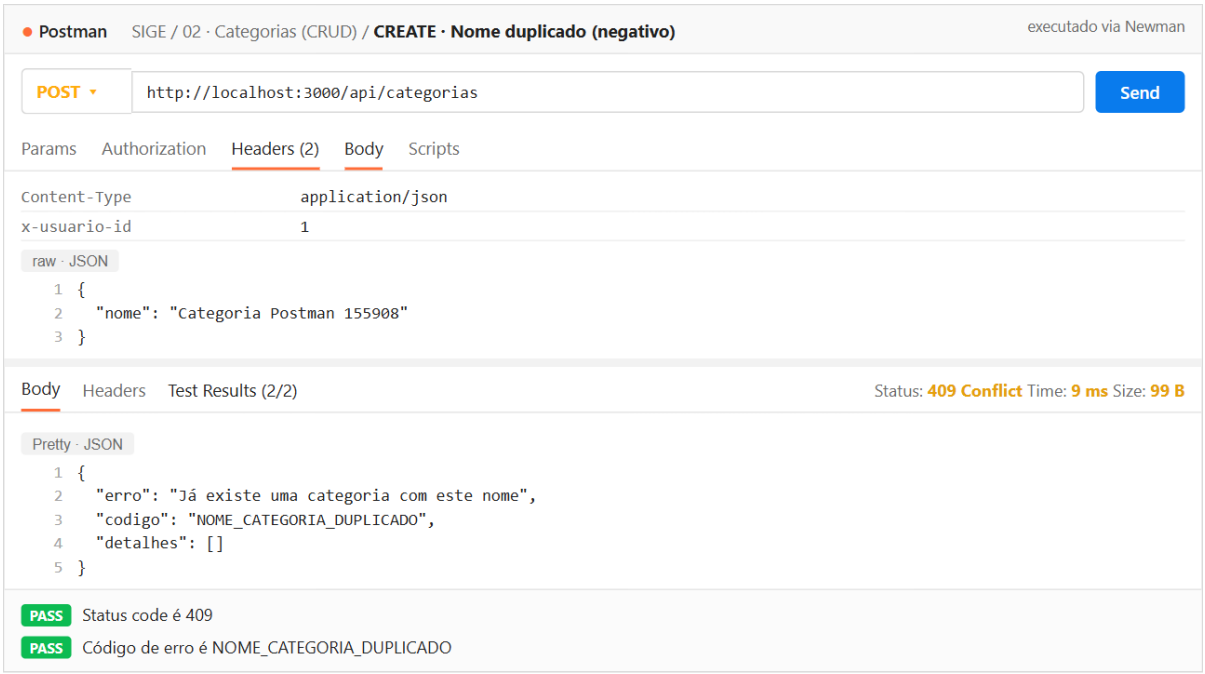


Figura 85 – TA-26: Evidência

TA-27 — [Categorias (CRUD)] CREATE · Sem nome (negativo)	Aprovado

Dados / passos utilizados	Postman · POST http://localhost:3000/api/categorias · x-usuario-id: 1 · corpo: { "nome": "" }
Resultado esperado	Status code é 422.
Resultado obtido	HTTP 422 Unprocessable Entity em 8 ms. Testes do Postman: 1/1 PASS.

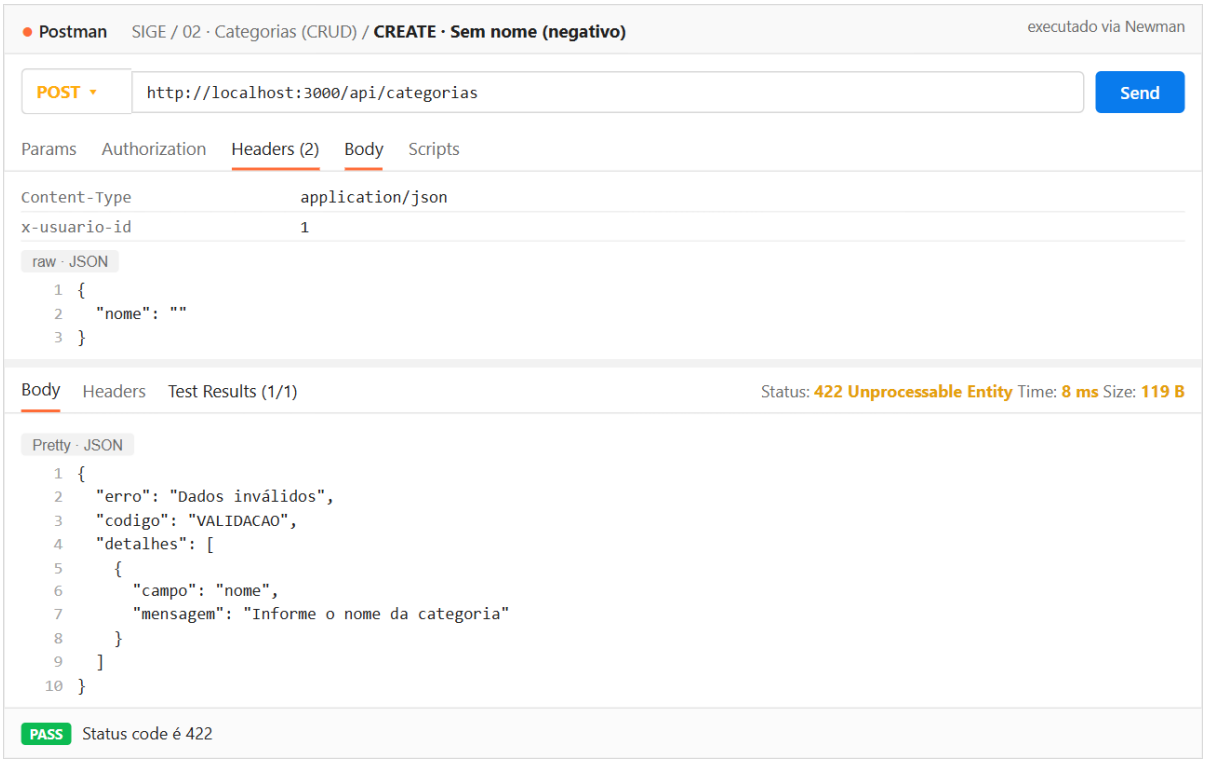


Figura 86 – TA-27: Evidência

TA-28 — [Categorias (CRUD)] READ · Listar em árvore (GET)	**Aprovado**
Dados / passos utilizados	Postman · GET http://localhost:3000/api/categorias?formato=arvore
Resultado esperado	Status code é 200; Categoria criada aparece.
Resultado obtido	HTTP 200 OK em 25 ms. Testes do Postman: 2/2 PASS.

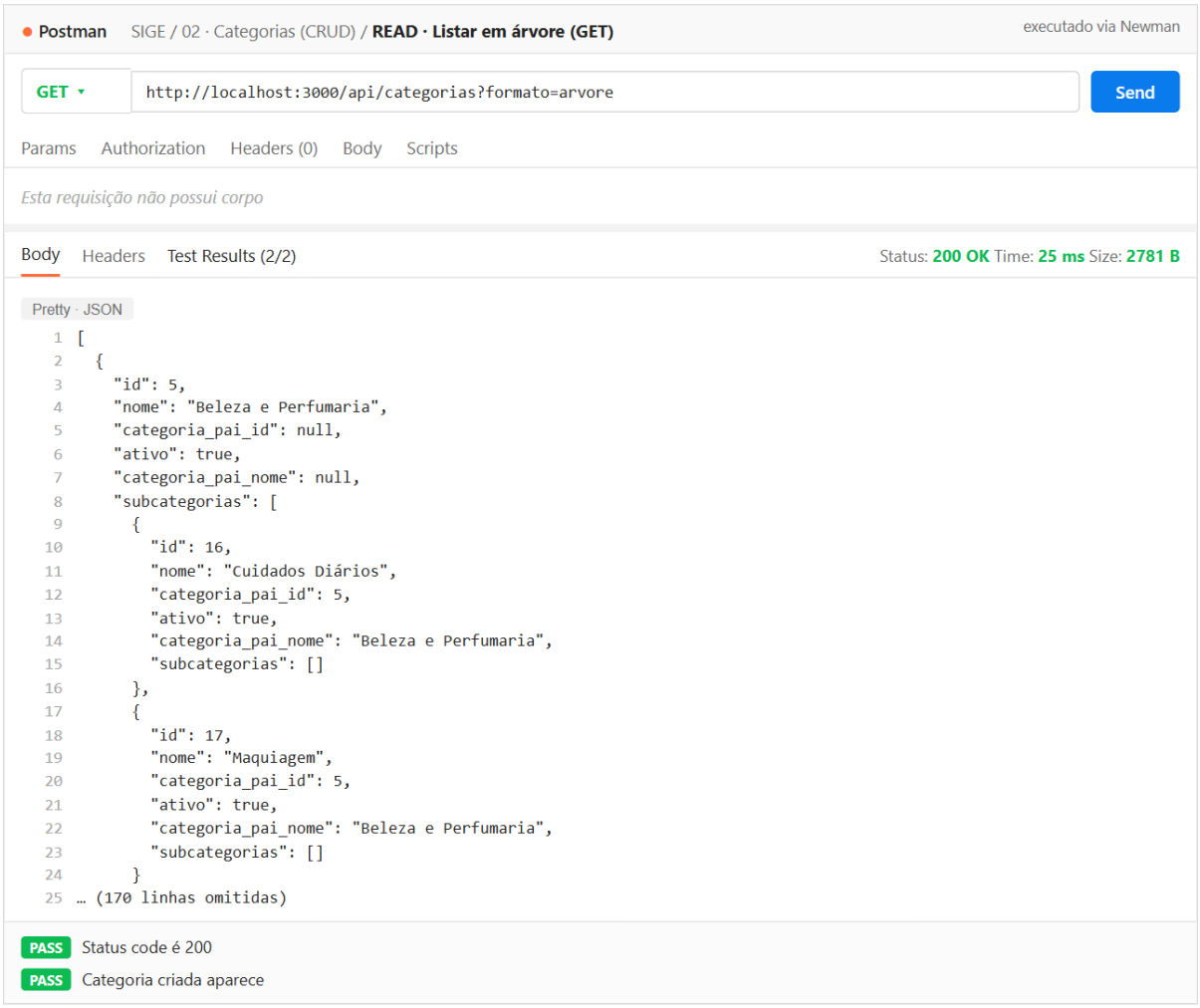


Figura 87 – TA-28: Evidência

TA-29 — [Categorias (CRUD)] UPDATE · Renomear categoria (PUT)	Aprovado
Dados / passos utilizados	Postman · PUT http://localhost:3000/api/categorias/31 · x-usuario-id: 1 · corpo: { "nome": "Categoria Postman 155908 (editada)" }
Resultado esperado	Status code é 200; Nome atualizado.
Resultado obtido	HTTP 200 OK em 12 ms. Testes do Postman: 2/2 PASS.

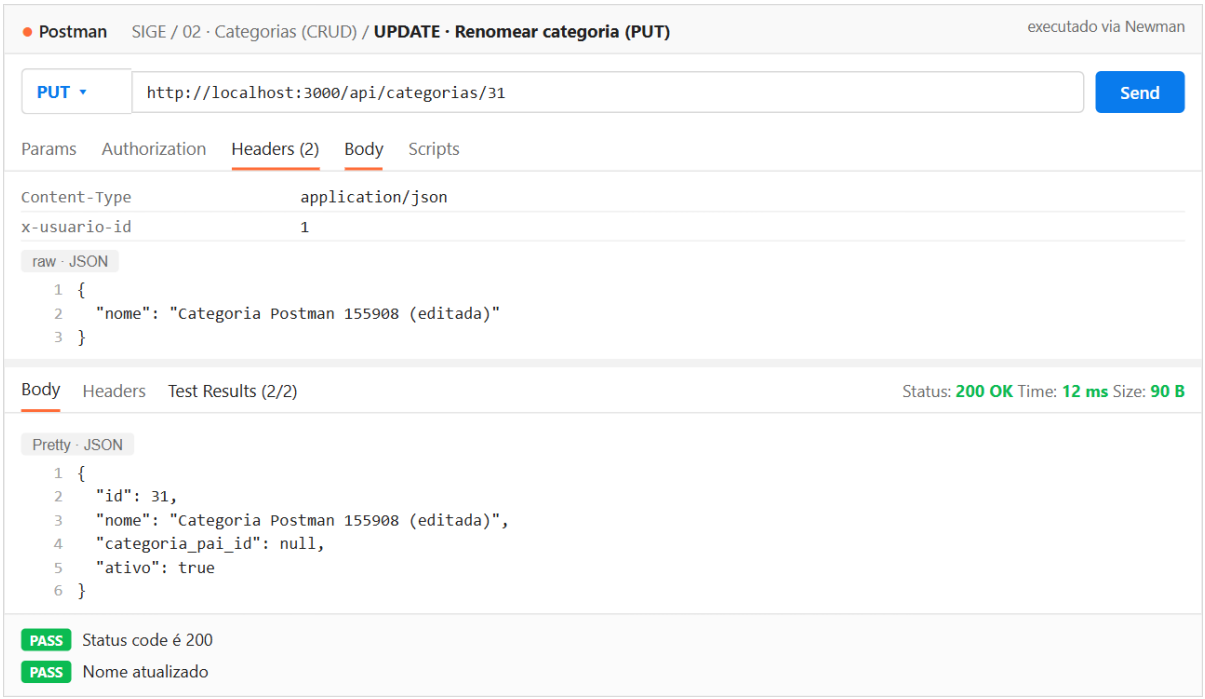


Figura 88 – TA-29: Evidência

TA-30 — [Categorias (CRUD)] UPDATE · Categoria inexistente (negativo)	Aprovado
Dados / passos utilizados	Postman · PUT http://localhost:3000/api/categorias/999999 · x-usuario-id: 1 · corpo: { "nome": "X 155908" }
Resultado esperado	Status code é 404; Código de erro é CATEGORIA_NAO_ENCONTRADA.
Resultado obtido	HTTP 404 Not Found em 9 ms. Testes do Postman: 2/2 PASS.

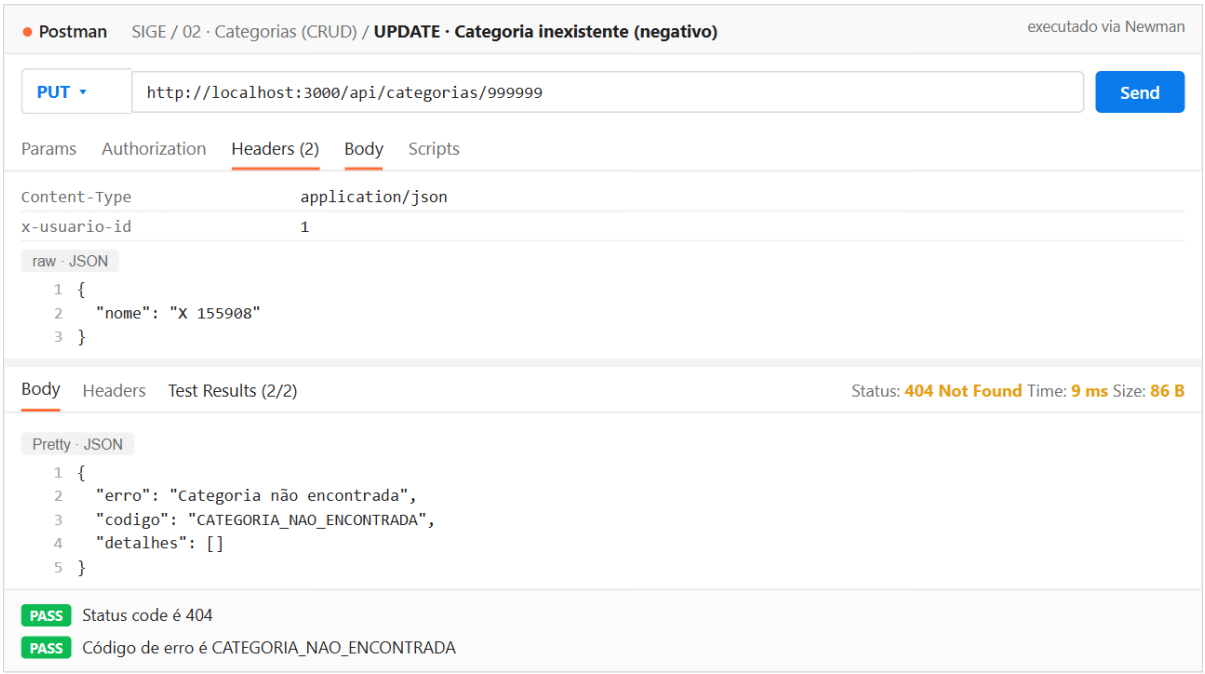


Figura 89 – TA-30: Evidência

TA-31 — [Unidades de medida (CRUD)] CREATE · Cadastrar unidade (POST)	Aprovado

Dados / passos utilizados	Postman · POST http://localhost:3000/api/unidades-medida · x-usuario-id: 1 · corpo: { "nome": "Pacote 155908", "sigla": "P155908", "casas_decimais": 0 }
Resultado esperado	Status code é 201.
Resultado obtido	HTTP 201 Created em 12 ms. Testes do Postman: 1/1 PASS.

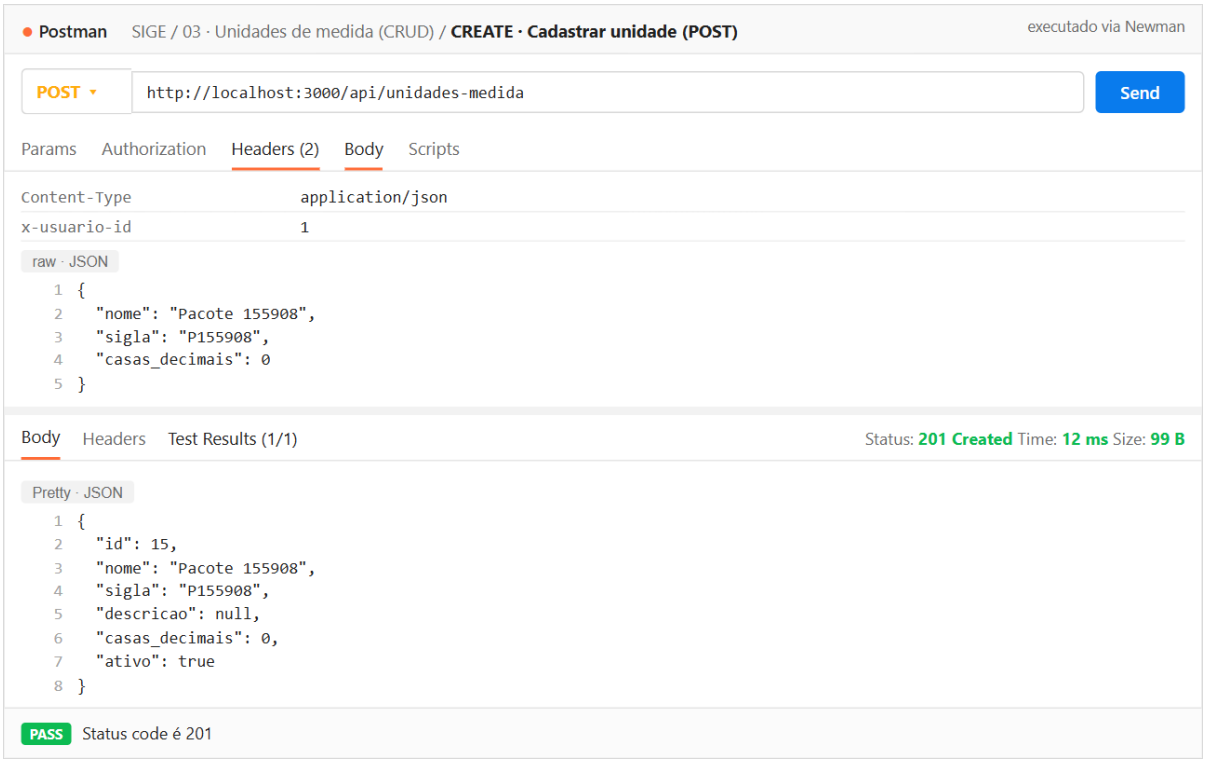


Figura 90 – TA-31: Evidência

TA-31 · Rota da API que atendeu a requisição

backend/src/rotas/unidades-medida.rotas.js linhas 38–57

```
38  fastify.post('/api/unidades-medida', {
39    schema: {
40      tags: TAG,
41      summary: 'Cadastra uma unidade de medida',
42      security: SEGURANCA_USUARIO,
43      body: deZod(unidadeMedidaCriacaoSchema),
44      response: {
45        201: { description: 'Unidade criada', ...unidadeMedida },
46        401: respostasErro[401],
47        409: respostasErro[409],
48        422: respostasErro[422],
49      },
50    },
51  }, async (request, reply) => {
52    await exigirUsuarioAutenticado(request);
53    const dados = unidadeMedidaCriacaoSchema.parse(request.body);
54    const unidade = await unidadesServico.cadastrar(dados);
55    return reply.status(201).send(unidade);
56  });
57
```

Figura 91 – TA-31: Codigo

TA-32 — [Unidades de medida (CRUD)] CREATE · Sigla duplicada (negativo)	Aprovado

Dados / passos utilizados	Postman · POST http://localhost:3000/api/unidades-medida · x-usuario-id: 1 · corpo: { "nome": "Outra 155908", "sigla": "P155908" }
Resultado esperado	Status code é 409; Código de erro é SIGLA_UNIDADE_DUPLICADA.
Resultado obtido	HTTP 409 Conflict em 9 ms. Testes do Postman: 2/2 PASS.



Figura 92 – TA-32: Evidência

TA-33 — [Unidades de medida (CRUD)] READ · Listar unidades (GET)	**Aprovado**
Dados / passos utilizados	Postman · GET http://localhost:3000/api/unidades-medida
Resultado esperado	Status code é 200; Lista não vazia.
Resultado obtido	HTTP 200 OK em 6 ms. Testes do Postman: 2/2 PASS.

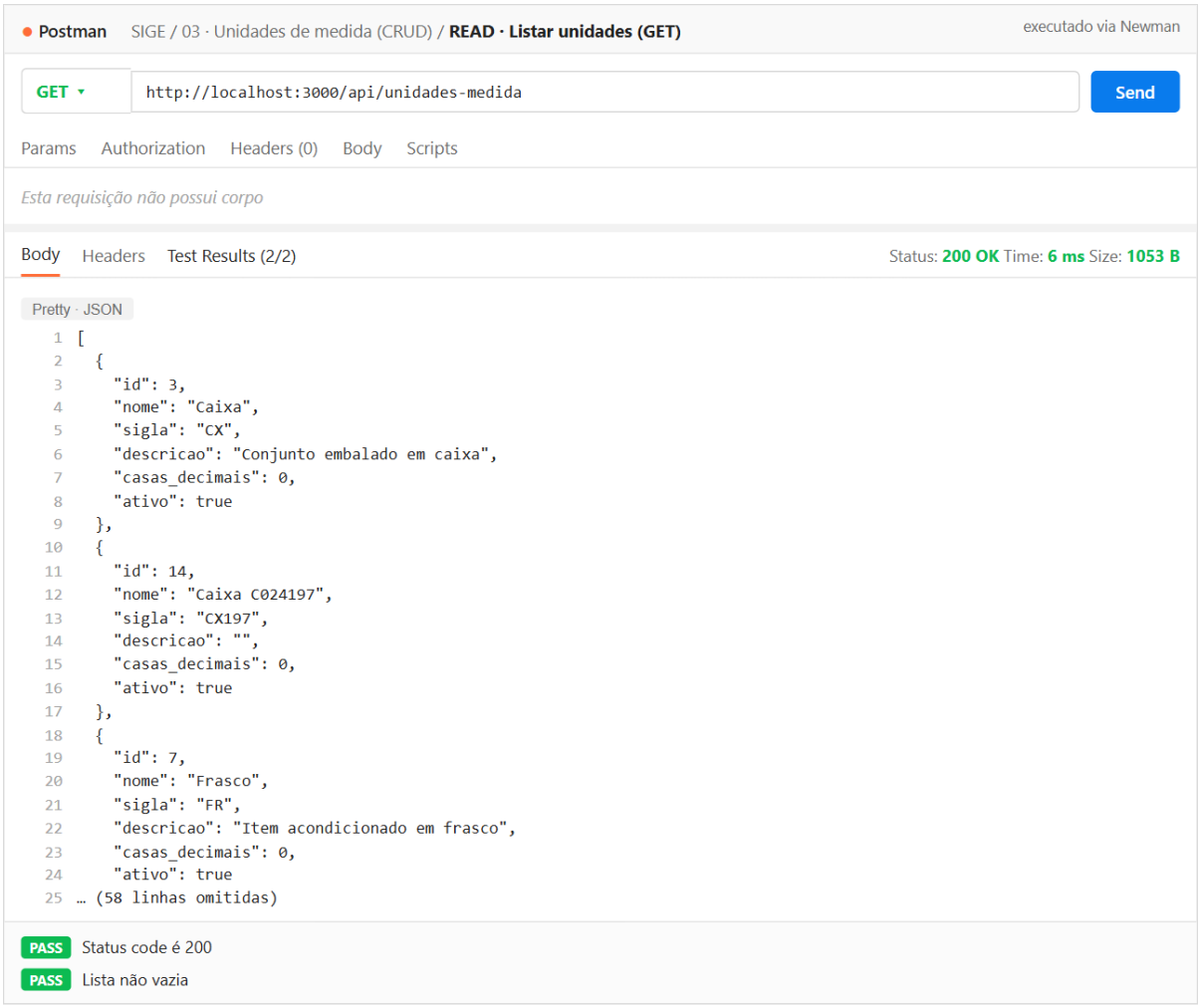


Figura 93 – TA-33: Evidência

TA-34 — [Unidades de medida (CRUD)] UPDATE · Alterar unidade (PUT)	Aprovado
Dados / passos utilizados	Postman · PUT http://localhost:3000/api/unidades-medida/15 · x-usuario-id: 1 · corpo: { "descricao": "Atualizada pelo Postman", "casas_decimais": 2 }
Resultado esperado	Status code é 200; Casas decimais = 2.
Resultado obtido	HTTP 200 OK em 32 ms. Testes do Postman: 2/2 PASS.

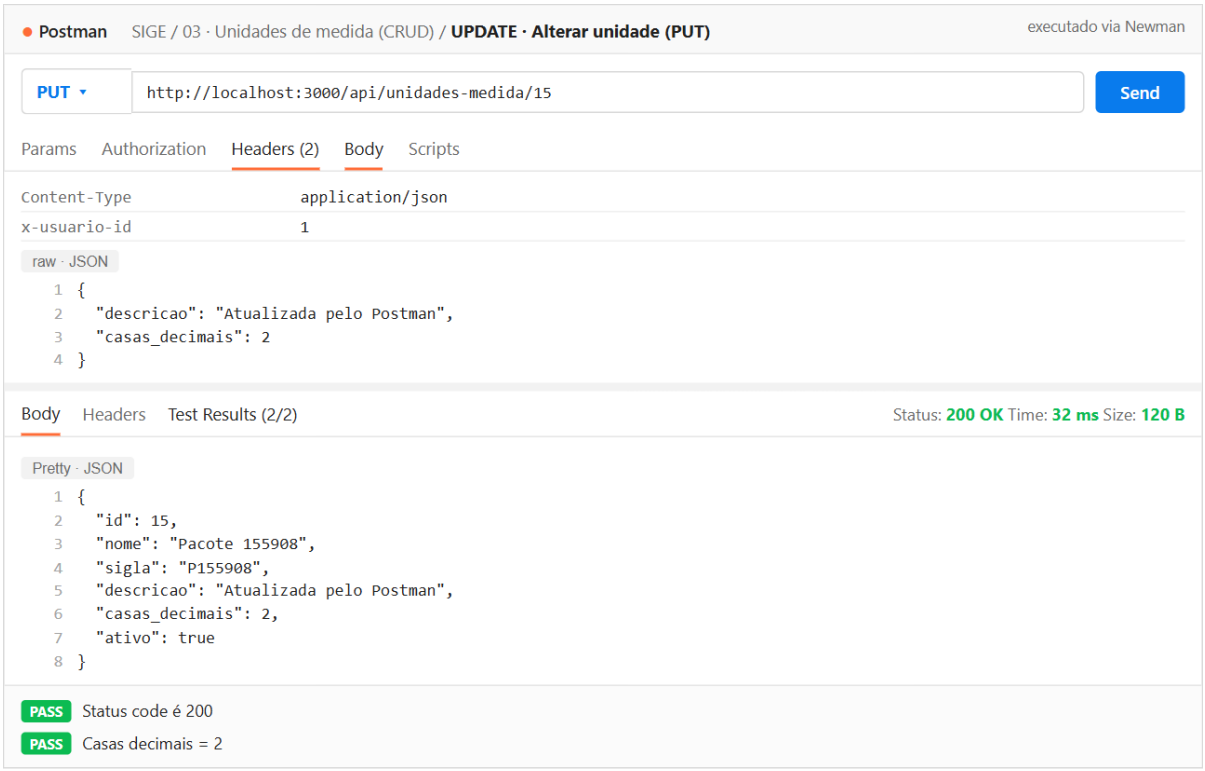


Figura 94 – TA-34: Evidência

TA-35 — [Unidades de medida (CRUD)] UPDATE · Casas decimais acima do limite (negativo)	**Aprovado**
Dados / passos utilizados	Postman · PUT http://localhost:3000/api/unidades-medida/15 · x-usuario-id: 1 · corpo: { "casas_decimais": 9 }
Resultado esperado	Status code é 422.
Resultado obtido	HTTP 422 Unprocessable Entity em 9 ms. Testes do Postman: 1/1 PASS.



Figura 95 – TA-35: Evidência

TA-36 — Documentação OpenAPI (Swagger) disponível	Aprovado
Dados / passos utilizados	Abrir http://localhost:3000/api/docs no navegador
Resultado esperado	Página de documentação interativa listando os endpoints.
Resultado obtido	Swagger UI exibido com os endpoints de Produtos, Categorias, Unidades e Autenticação.

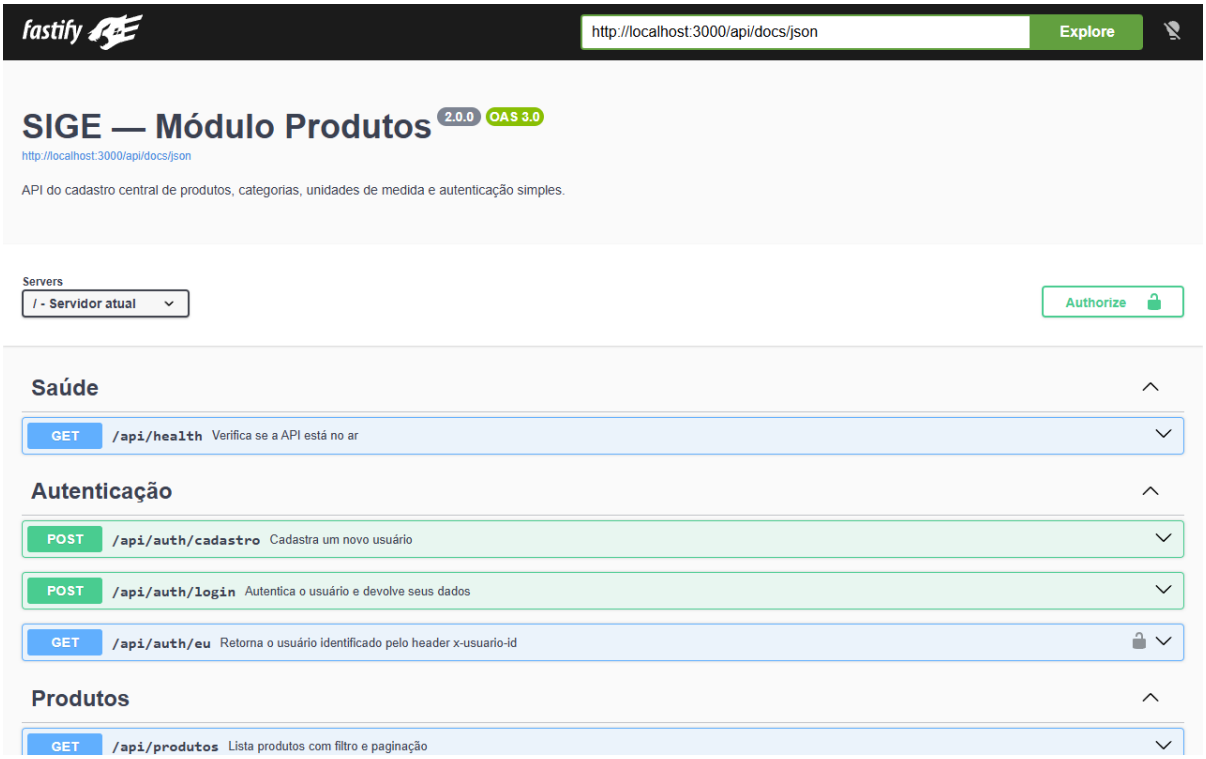


Figura 96 – TA-36: Swagger

5. Testes de banco de dados

O que é: Verificam se os dados são inseridos, consultados, atualizados e excluídos corretamente, além de conferir a consistência e os relacionamentos entre as tabelas.

Aplicação neste sistema: Consultas SQL foram executadas diretamente no PostgreSQL do sistema (psql dentro do container). Os testes de restrição foram feitos dentro de transação com ROLLBACK, sem alterar dados.

TB-01 — Produto cadastrado pela interface foi gravado corretamente	**Aprovado**
Dados / passos utilizados	SELECT id, sku, nome, ean13, categoria_id, unidade_medida_id, preco_custo, preco_venda, estoque, ativo FROM produto WHERE sku = 'NTB-024197'
Resultado esperado	Uma linha com os valores digitados no TP-01 (custo 2500, venda 3299,90 após edição, estoque 15).
Resultado obtido	Registro encontrado com os valores esperados (preco_venda=3299.90, preco_custo=2500.00, estoque=15.000).

TB-01 · Produto gravado

PSQL · SIGE_PRODUTOS (CONTAINER APP-PRODUCT-POSTGRES-1)

```
SELECT id, sku, nome, ean13, categoria_id, unidade_medida_id, preco_custo, preco_venda, estoque, ativo FROM produto WHERE sku = 'NTB-024197';
 id |    sku     |          nome           |     ean13     | categoria_id | unidade_medida_id | preco_custo | preco_venda | estoque | ativo
----+------------+-------------------------+---------------+--------------+-------------------+-------------+-------------+---------+-------
 13 | NTB-024197 | Notebook Ciclo03 024197 | 7890241971114 |            8 |                 1 |     2500.00 |     3299.90 |  15.000 | t
(1 row)
```

Figura 97 – TB-01: Select produto

TB-02 — Histórico de preço gravado no cadastro e na alteração	Aprovado
Dados / passos utilizados	SELECT tipo_preco, valor_anterior, valor_novo, motivo, usuario_id FROM produto_preco_historico WHERE produto_id = {id}
Resultado esperado	3 linhas: venda inicial, custo inicial e alteração de venda (3499,90 → 3299,90).
Resultado obtido	3 registros de histórico, incluindo valor_anterior=3499.90 e valor_novo=3299.90.

TB-02 · Histórico de preço

```
PSQL · SIGE_PRODUTOS (CONTAINER APP-PRODUCT-POSTGRES-1)
SELECT id, tipo_preco, valor_anterior, valor_novo, motivo, usuario_id FROM produto_preco_historico WHERE produto_
id = 13 ORDER BY id;
 id | tipo_preco | valor_anterior | valor_novo |          motivo            | usuario_id
----+------------+----------------+------------+----------------------------+------------
 36 | venda      |                |    3499.90 | Cadastro inicial           |          1
 37 | custo      |                |    2500.00 | Cadastro inicial           |          1
 38 | venda      |        3499.90 |    3299.90 | Alteração de preço de venda |          1
(3 rows)
```

Figura 98 – TB-02: Historico

TB-03 — Senha do usuário armazenada como hash bcrypt	Aprovado
Dados / passos utilizados	SELECT id, nome, email, left(senha,7) AS prefixo_hash, length(senha) FROM usuario WHERE email = '{email}'
Resultado esperado	Coluna senha com hash bcrypt (prefixo $2b$10$, 60 caracteres) e não "senha1234".
Resultado obtido	Senha armazenada como hash bcrypt de 60 caracteres; comparação com texto puro = false.

TB-03 · Hash da senha

```
PSQL · SIGE_PRODUTOS (CONTAINER APP-PRODUCT-POSTGRES-1)
SELECT id, nome, email, left(senha,7) AS prefixo_hash, length(senha) AS tamanho, senha = 'senha1234' AS texto_pur
o FROM usuario WHERE email = 'ciclo03.024197@teste.com';
 id |       nome        |          email           | prefixo_hash | tamanho | texto_puro
----+-------------------+--------------------------+--------------+---------+------------
 10 | Testador Ciclo 03 | ciclo03.024197@teste.com | $2b$10$      |      60 | f
(1 row)
```

Figura 99 – TB-03: Hash senha

TB-04 — Restrição CHECK impede preço de venda menor ou igual a zero	Aprovado
Dados / passos utilizados	INSERT direto em produto com preco_venda = 0 (dentro de transação com ROLLBACK)
Resultado esperado	Erro "violates check constraint ck_produto_preco_venda"; nenhuma linha inserida.
Resultado obtido	Banco rejeitou o INSERT: violates check constraint "ck_produto_preco_venda".

TB-04 · CHECK de preço

```
PSQL · SIGE_PRODUTOS (CONTAINER APP-PRODUCT-POSTGRES-1)
BEGIN;
BEGIN
INSERT INTO produto (sku, nome, categoria_id, unidade_medida_id, preco_venda) VALUES ('CHK-024197', 'Teste chec
k', 8, 1, 0);
ERROR:  new row for relation "produto" violates check constraint "ck_produto_preco_venda"
DETAIL:  Failing row contains (16, CHK-024197, Teste check, null, null, 8, 1, 0.00, 0.00, 0.000, t, 2026-11-05 0
1:12:48.152436+00, 2026-11-05 01:12:48.152436+00).
ROLLBACK;
ROLLBACK
```

Figura 100 – TB-04: Check

TB-04 · Restrições definidas na tabela produto

database/migrations/004_produto.sql linhas 2–26

```
CREATE TABLE IF NOT EXISTS produto (
  id                SERIAL PRIMARY KEY,
  sku               VARCHAR(15)   NOT NULL,
  nome              VARCHAR(60)   NOT NULL,
  descricao         TEXT          NULL,
  ean13             CHAR(13)      NULL,               -- P-05
  categoria_id      INTEGER       NOT NULL,
  unidade_medida_id INTEGER       NOT NULL,
  preco_custo       NUMERIC(10,2) NOT NULL DEFAULT 0,  -- P-04
  preco_venda       NUMERIC(10,2) NOT NULL,            -- P-04
  estoque           NUMERIC(10,3) NOT NULL DEFAULT 0,  -- campo informativo, sem movimentação
  ativo             BOOLEAN       NOT NULL DEFAULT TRUE,
  criado_em         TIMESTAMPTZ   NOT NULL DEFAULT NOW(),
  atualizado_em     TIMESTAMPTZ   NOT NULL DEFAULT NOW(),
  CONSTRAINT uk_produto_sku            UNIQUE (sku),                 -- RN03 / RF09
  CONSTRAINT uk_produto_nome_categoria UNIQUE (categoria_id, nome),  -- RN08 / P-09
  CONSTRAINT fk_produto_categoria      FOREIGN KEY (categoria_id)
                                       REFERENCES categoria (id) ON DELETE RESTRICT,
  CONSTRAINT fk_produto_unidade_medida FOREIGN KEY (unidade_medida_id)
                                       REFERENCES unidade_medida (id) ON DELETE RESTRICT,
  CONSTRAINT ck_produto_preco_venda    CHECK (preco_venda > 0),      -- RN01 / RF08
  CONSTRAINT ck_produto_preco_custo    CHECK (preco_custo >= 0),     -- RN02
  CONSTRAINT ck_produto_estoque        CHECK (estoque >= 0),
  CONSTRAINT ck_produto_ean13_formato  CHECK (ean13 IS NULL OR ean13 ~ '^[0-9]{13}$')
);
```

Figura 101 – TB-04: Codigo

TB-05 — Restrição UNIQUE impede SKU duplicado	Aprovado
Dados / passos utilizados	INSERT direto em produto reutilizando o SKU NTB-024197
Resultado esperado	Erro "duplicate key value violates unique constraint uk_produto_sku".
Resultado obtido	Banco rejeitou: duplicate key value violates unique constraint "uk_produto_sku".

TB-05 · UNIQUE do SKU

PSQL · SIGE_PRODUTOS (CONTAINER APP-PRODUCT-POSTGRES-1)

```
BEGIN;
BEGIN
INSERT INTO produto (sku, nome, categoria_id, unidade_medida_id, preco_venda) VALUES ('NTB-024197', 'Outro nome 0
24197', 9, 1, 5);
ERROR:  duplicate key value violates unique constraint "uk_produto_sku"
DETAIL:  Key (sku)=(NTB-024197) already exists.
ROLLBACK;
ROLLBACK
```

Figura 102 – TB-05: Unique

TB-06 — Chave estrangeira impede excluir categoria com produtos	Aprovado
Dados / passos utilizados	DELETE FROM categoria WHERE id = 8 (possui produtos) dentro de transação com ROLLBACK
Resultado esperado	Erro de violação de chave estrangeira (ON DELETE RESTRICT).
Resultado obtido	Banco rejeitou o DELETE: violates foreign key constraint "fk_produto_categoria".

TB-06 · FK produto → categoria

PSQL · SIGE_PRODUTOS (CONTAINER APP-PRODUCT-POSTGRES-1)

```
BEGIN;
BEGIN
DELETE FROM categoria WHERE id = 8;
ERROR:  update or delete on table "categoria" violates foreign key constraint "fk_produto_categoria" on table "pr
oduto"
DETAIL:  Key (id)=(8) is still referenced from table "produto".
ROLLBACK;
ROLLBACK
```

Figura 103 – TB-06: Fk

TB-07 — Trigger atualiza "atualizado_em" automaticamente	Aprovado
Dados / passos utilizados	Comparar criado_em × atualizado_em do produto {id} (editado no TP-02)
Resultado esperado	atualizado_em posterior a criado_em para um produto editado.
Resultado obtido	atualizado_em é posterior a criado_em (foi_atualizado = t).

TB-07 · Trigger de atualizado_em

PSQL · SIGE_PRODUTOS (CONTAINER APP-PRODUCT-POSTGRES-1)

```
SELECT id, sku, criado_em, atualizado_em, atualizado_em > criado_em AS foi_atualizado FROM produto WHERE id = 13;
 id |    sku     |           criado_em           |         atualizado_em         | foi_atualizado
----+------------+-------------------------------+-------------------------------+----------------
 13 | NTB-024197 | 2026-11-05 01:10:52.731273+00 | 2026-11-05 01:11:12.653406+00 | t
(1 row)
```

Figura 104 – TB-07: Trigger

TB-08 — Inativação é lógica: registro permanece no banco	Aprovado
Dados / passos utilizados	Inativar produto {id} (API) e consultar a coluna ativo
Resultado esperado	Linha continua existindo com ativo = false.
Resultado obtido	Registro mantido com ativo = f (inativo).

TB-08 · Inativação lógica

PSQL · SIGE_PRODUTOS (CONTAINER APP-PRODUCT-POSTGRES-1)

```
SELECT id, sku, nome, ativo FROM produto WHERE id = 14;
 id |    sku     |        nome        | ativo
----+------------+--------------------+-------
 14 | API-024197 | Produto API 024197 | f
(1 row)
```

Figura 105 – TB-08: Inativacao

TB-09 — Integridade referencial: nenhum produto sem categoria/unidade válida	Aprovado
Dados / passos utilizados	SELECT de órfãos: produtos sem categoria ou unidade correspondente (LEFT JOIN … IS NULL)
Resultado esperado	Contagem de órfãos = 0 em ambas as relações.
Resultado obtido	As três consultas retornaram 0.

TB-09 · Produtos órfãos

PSQL · SIGE_PRODUTOS (CONTAINER APP-PRODUCT-POSTGRES-1)

```
SELECT count(*) AS sem_categoria FROM produto p LEFT JOIN categoria c ON c.id = p.categoria_id WHERE c.id IS NUL
L;
 sem_categoria
---------------
             0
(1 row)

SELECT count(*) AS sem_unidade FROM produto p LEFT JOIN unidade_medida u ON u.id = p.unidade_medida_id WHERE u.id
IS NULL;
 sem_unidade
-------------
           0
(1 row)

SELECT count(*) AS historico_sem_produto FROM produto_preco_historico h LEFT JOIN produto p ON p.id = h.produto_i
d WHERE p.id IS NULL;
 historico_sem_produto
-----------------------
                     0
(1 row)
```

Figura 106 – TB-09: Orfaos

6. Testes de integração

O que é: Verificam se diferentes partes ou módulos do sistema trabalham corretamente em conjunto e se as informações são transmitidas entre eles.

Aplicação neste sistema: Cada teste percorre as camadas tela → API → banco de dados e confere se a mesma informação aparece em todas elas.

TI-01 — Cadastro pela tela → API → banco → listagem	**Aprovado**
Dados / passos utilizados	Cadastrar "Mouse Integração 024197" (SKU INT-024197) pela tela; conferir via API e via SELECT
Resultado esperado	Registro presente na resposta da API, na tabela produto e na listagem da tela.
Resultado obtido	Produto id 18: exibido na tela, retornado pela API (HTTP 200) e persistido no banco.

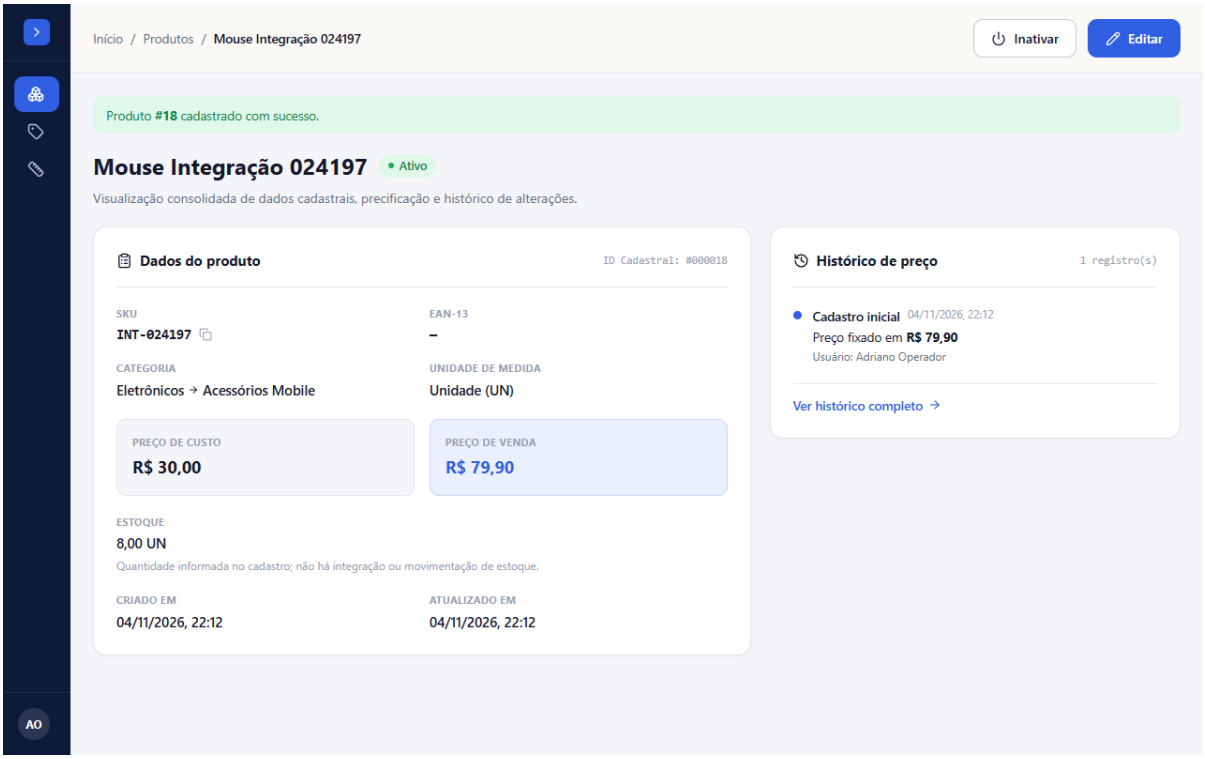


Figura 107 – TI-01: Tela

TI-01 · API vê o produto criado pela tela

REQUISIÇÃO

`GET` `http://localhost:3000/api/produtos/18`

(sem cabeçalhos adicionais)

RESPOSTA

`HTTP 200 OK · 10 ms`


```
{
  "id": 18,
  "sku": "INT-024197",
  "nome": "Mouse Integração 024197",
  "descricao": "",
  "ean13": null,
  "categoria": {
    "id": 8,
    "nome": "Acessórios Mobile",
    "categoria_pai": {
      "id": 1,
      "nome": "Eletrônicos"
    }
  },
  "unidade_medida": {
    "id": 1,
    "nome": "Unidade",
    "sigla": "UN"
  },
  "preco_custo": 30,
  "preco_venda": 79.9,
  "estoque": 8,
  "situacao": "ativo",
  "criado_em": "2026-11-05T01:12:56.498Z",
  "atualizado_em": "2026-11-05T01:12:56.498Z"
}
```


Figura 108 – TI-01: Api

TI-01 · Banco vê o produto criado pela tela

PSQL · SIGE_PRODUTOS (CONTAINER APP-PRODUCT-POSTGRES-1)

```
SELECT id, sku, nome, preco_venda, estoque FROM produto WHERE sku = 'INT-024197';
 id |    sku     |          nome           | preco_venda | estoque
----+------------+-------------------------+-------------+---------
 18 | INT-024197 | Mouse Integração 024197 |       79.90 |   8.000
(1 row)
```

Figura 109 – TI-01: Banco

TI-02 — Alteração de preço pela tela reflete no histórico (tela, API e banco)	**Aprovado**
Dados / passos utilizados	Editar "Mouse Integração 024197" pela tela: venda 79,90 → 89,90; conferir /precos, API e banco
Resultado esperado	Histórico atualizado em todas as camadas com valor anterior preservado.
Resultado obtido	Tela, API (3 registros) e banco concordam: venda 79.90 → 89.90.

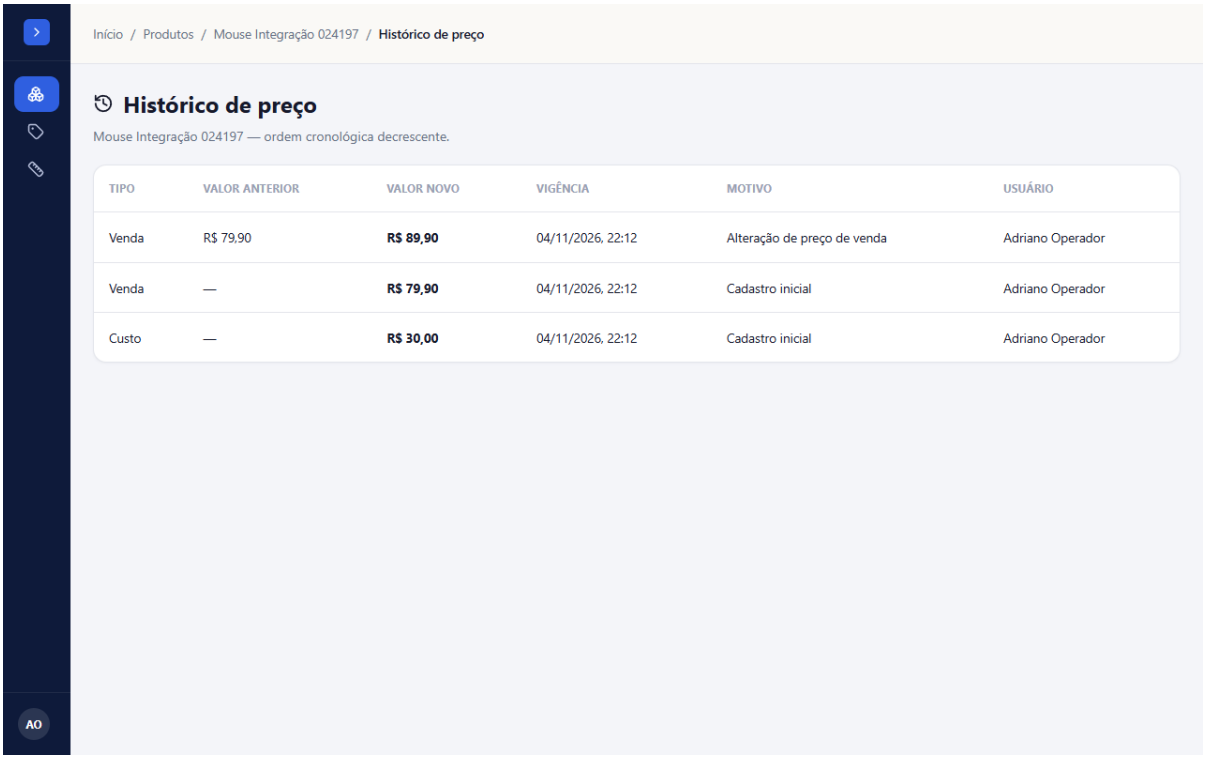


Figura 110 – TI-02: Historico tela

TI-02 · Histórico via API

REQUISIÇÃO

`GET` `http://localhost:3000/api/produtos/18/precos`

(sem cabeçalhos adicionais)

RESPOSTA

HTTP 200 OK · 11 ms


```
{
  "produto_id": 18,
  "historico": [
    {
      "id": 47,
      "tipo_preco": "venda",
      "valor_anterior": 79.9,
      "valor_novo": 89.9,
      "vigencia_inicio": "2026-11-05T01:12:59.328Z",
      "motivo": "Alteração de preço de venda",
      "usuario": {
        "id": 1,
        "nome": "Adriano Operador"
      }
    },
    {
      "id": 45,
      "tipo_preco": "venda",
      "valor_anterior": null,
      "valor_novo": 79.9,
      "vigencia_inicio": "2026-11-05T01:12:56.498Z",
      "motivo": "Cadastro inicial",
      "usuario": {
        "id": 1,
        "nome": "Adriano Operador"
      }
  … (15 linhas omitidas)
```


Figura 111 – TI-02: Api

TI-02 · Histórico no banco

PSQL · SIGE_PRODUTOS (CONTAINER APP-PRODUCT-POSTGRES-1)

```
SELECT tipo_preco, valor_anterior, valor_novo, motivo FROM produto_preco_historico WHERE produto_id = 18 ORDER BY
id;
 tipo_preco | valor_anterior | valor_novo |          motivo
------------+----------------+------------+---------------------------
 venda      |                |      79.90 | Cadastro inicial
 custo      |                |      30.00 | Cadastro inicial
 venda      |          79.90 |      89.90 | Alteração de preço de venda
(3 rows)
```

Figura 112 – TI-02: Banco

TI-03 — Categoria criada em um módulo aparece no formulário de produto	**Aprovado**
Dados / passos utilizados	Categoria "Categoria Ciclo03 024197" (criada no TP-06) deve estar disponível no combo de categoria de /produtos/novo
Resultado esperado	Opção encontrada ao pesquisar pelo nome.
Resultado obtido	Combo lista a subcategoria "Sub 024197" criada na tela de categorias.

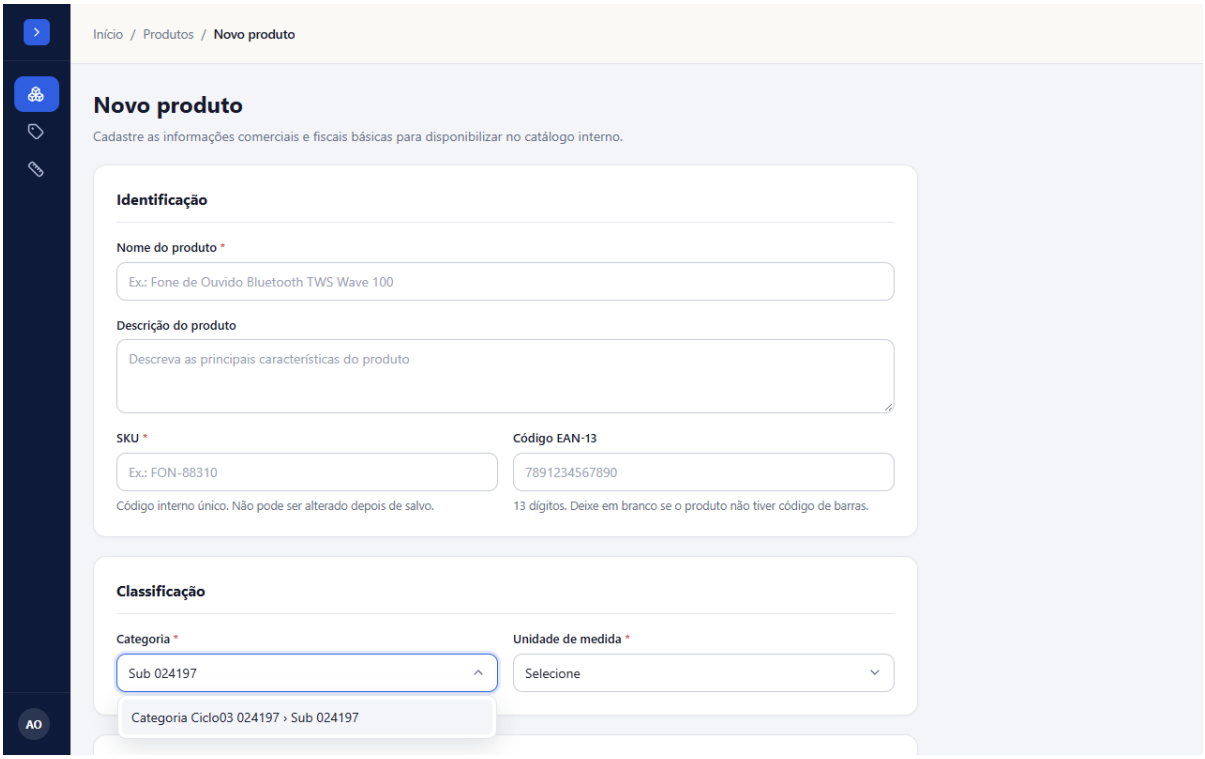


Figura 113 – TI-03: Combo categoria

TI-04 — Inativação pela tela reflete na API e no banco	Aprovado
Dados / passos utilizados	Inativar "Mouse Integração 024197" pela listagem; consultar API e banco
Resultado esperado	API retorna situacao "inativo" e banco registra ativo = false.
Resultado obtido	API: situacao=inativo · Banco: ativo=f.

TI-04 · API após inativar na tela

REQUISIÇÃO

GET http://localhost:3000/api/produtos/18

(sem cabeçalhos adicionais)

RESPOSTA

HTTP 200 OK · 24 ms


```
{
  "id": 18,
  "sku": "INT-024197",
  "nome": "Mouse Integração 024197",
  "descricao": "",
  "ean13": null,
  "categoria": {
    "id": 8,
    "nome": "Acessórios Mobile",
    "categoria_pai": {
      "id": 1,
      "nome": "Eletrônicos"
    }
  },
  "unidade_medida": {
    "id": 1,
    "nome": "Unidade",
    "sigla": "UN"
  },
  "preco_custo": 30,
  "preco_venda": 89.9,
  "estoque": 8,
  "situacao": "inativo",
  "criado_em": "2026-11-05T01:12:56.498Z",
  "atualizado_em": "2026-11-05T01:13:08.843Z"
}
```


Figura 114 – TI-04: Api

TI-04 · Banco após inativar na tela

PSQL · SIGE_PRODUTOS (CONTAINER APP-PRODUCT-POSTGRES-1)

```
SELECT id, sku, ativo FROM produto WHERE id = 18;
 id |    sku     | ativo
----+------------+-------
 18 | INT-024197 | f
(1 row)
```

Figura 115 – TI-04: Banco

TI-05 — Cadastro de usuário (tela) permite login e gravação de autoria no histórico	**Aprovado**
Dados / passos utilizados	Usuário criado no CT-02 cadastra produto via API; histórico deve registrar o usuario_id dele
Resultado esperado	usuario_id do histórico = id do usuário criado no CT-02.
Resultado obtido	Login HTTP 200; produto HTTP 201; histórico associado ao usuário 10 (ciclo03.024197@teste.com).

TI-05 · Login do usuário criado pela tela

REQUISIÇÃO

POST http://localhost:3000/api/auth/login

```
Content-Type: application/json

{
  "email": "ciclo03.024197@teste.com",
  "senha": "senha1234"
}
```

RESPOSTA

HTTP 200 OK · 81 ms


```
{
  "id": 10,
  "nome": "Testador Ciclo 03",
  "email": "ciclo03.024197@teste.com"
}
```


Figura 116 – TI-05: Login

TI-05 · Produto cadastrado por esse usuário

REQUISIÇÃO

POST http://localhost:3000/api/produtos

```
Content-Type: application/json
x-usuario-id: 10

{
  "sku": "USR-024197",
  "nome": "Produto do usuário 024197",
  "categoria_id": 15,
  "unidade_medida_id": 1,
  "preco_venda": 12.5
}
```

RESPOSTA

HTTP 201 Created · 45 ms


```
{
  "id": 19,
  "sku": "USR-024197",
  "nome": "Produto do usuário 024197",
  "descricao": null,
  "ean13": null,
  "categoria": {
    "id": 15,
    "nome": "Escrita",
    "categoria_pai": {
      "id": 4,
      "nome": "Papelaria"
    }
  },
  "unidade_medida": {
    "id": 1,
    "nome": "Unidade",
    "sigla": "UN"
  },
  "preco_custo": 0,
  "preco_venda": 12.5,
  "estoque": 0,
  "situacao": "ativo",
  "criado_em": "2026-11-05T01:13:11.124Z",
  "atualizado_em": "2026-11-05T01:13:11.124Z"
}
```


Figura 117 – TI-05: Produto

TI-05 · Autoria no histórico

PSQL · SIGE_PRODUTOS (CONTAINER APP-PRODUCT-POSTGRES-1)

```
SELECT h.produto_id, h.tipo_preco, h.valor_novo, h.usuario_id, u.email FROM produto_preco_historico h JOIN usuari
o u ON u.id = h.usuario_id WHERE h.produto_id = 19;
 produto_id | tipo_preco | valor_novo | usuario_id |          email
------------+------------+------------+------------+-------------------------
         19 | venda      |      12.50 |         10 | ciclo03.024197@teste.com
         19 | custo      |       0.00 |         10 | ciclo03.024197@teste.com
(2 rows)
```

Figura 118 – TI-05: Autoria

7. Testes de requisitos

O que é: Verificam se as funcionalidades e regras implementadas correspondem ao que foi solicitado ou definido para o sistema.

Aplicação neste sistema: Foram verificadas as regras de negócio identificadas no código e na documentação (RN01–RN14, RF04, RF22, RF30, RF32 e RNF03), cada uma com um teste específico.

TR-01 — RN01/RF08 — preço de venda deve ser maior que zero	Aprovado
Dados / passos utilizados	POST /api/produtos com preco_venda = 0
Resultado esperado	HTTP 422 VALIDACAO.
Resultado obtido	HTTP 422.

TR-01 · RN01 preço de venda > 0

REQUISIÇÃO

```
POST http://localhost:3000/api/produtos

Content-Type: application/json
x-usuario-id: 1

{
  "sku": "RN1-024197",
  "nome": "RN01",
  "categoria_id": 8,
  "unidade_medida_id": 1,
  "preco_venda": 0
}
```

RESPOSTA

```
HTTP 422 Unprocessable Entity  ·  10 ms

{
  "erro": "Dados inválidos",
  "codigo": "VALIDACAO",
  "detalhes": [
    {
      "campo": "preco_venda",
      "mensagem": "Preço de venda deve ser maior que zero"
    }
  ]
}
```

Figura 119 – TR-01: Evidência

TR-02 — RN02 — preço de custo não pode ser negativo	Aprovado
Dados / passos utilizados	POST /api/produtos com preco_custo = -5
Resultado esperado	HTTP 422 VALIDACAO.

Resultado obtido	HTTP 422.

TR-02 · RN02 custo ≥ 0

REQUISIÇÃO

POST http://localhost:3000/api/produtos

```
Content-Type: application/json
x-usuario-id: 1

{
  "sku": "RN2-024197",
  "nome": "RN02",
  "categoria_id": 8,
  "unidade_medida_id": 1,
  "preco_custo": -5,
  "preco_venda": 10
}
```

RESPOSTA

HTTP 422 Unprocessable Entity · 17 ms

```
{
  "erro": "Dados inválidos",
  "codigo": "VALIDACAO",
  "detalhes": [
    {
      "campo": "preco_custo",
      "mensagem": "Preço de custo não pode ser negativo"
    }
  ]
}
```

Figura 120 – TR-02: Evidência

TR-03 — RN03 — SKU imutável após o cadastro	**Aprovado**
Dados / passos utilizados	Tela de edição: campo SKU desabilitado; API PUT com "sku" rejeitada (ver Postman: UPDATE · Tentar alterar SKU)
Resultado esperado	Campo SKU bloqueado na edição e API rejeita alteração.
Resultado obtido	Campo SKU exibido desabilitado na edição ("Não pode ser alterado depois de salvo").

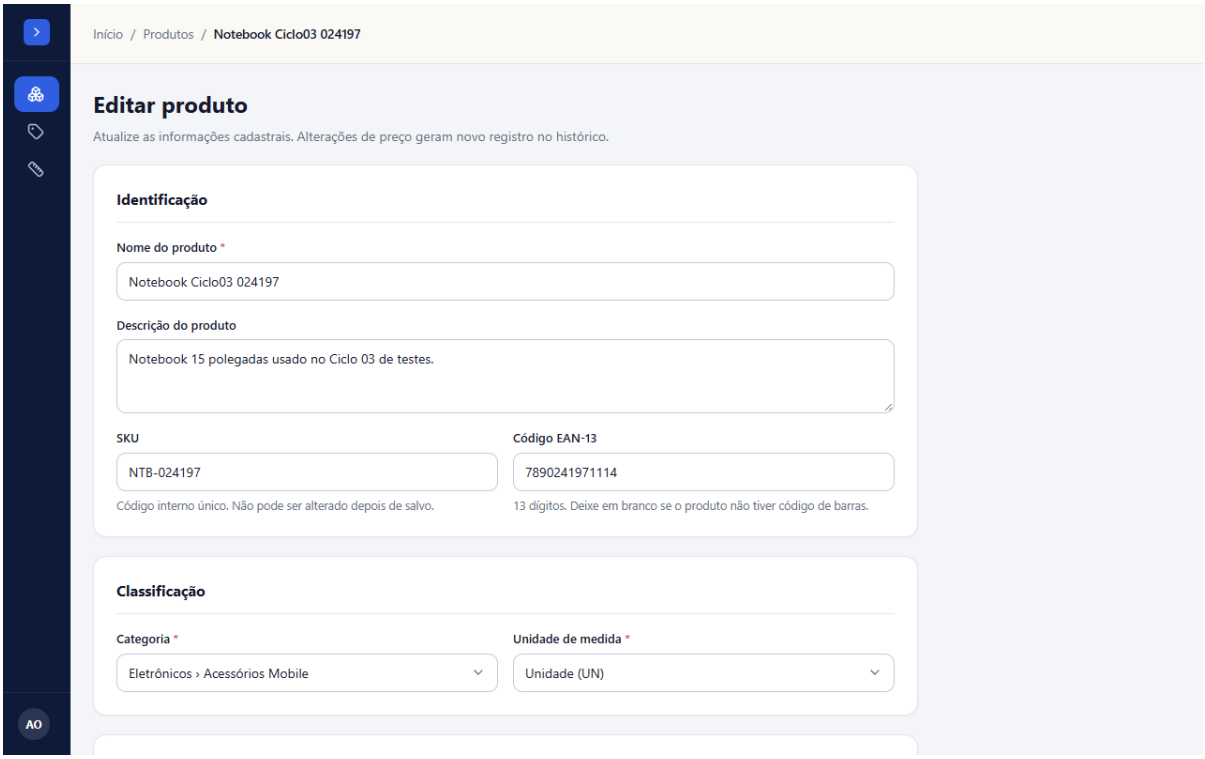


Figura 121 – TR-03: Sku bloqueado

TR-04 — RN04/RF22 — EAN-13 único e com dígito verificador válido	Aprovado
Dados / passos utilizados	POST /api/produtos reutilizando o EAN-13 7890241972227 do produto criado no TA-09
Resultado esperado	HTTP 409 EAN13_DUPLICADO.
Resultado obtido	HTTP 409 — EAN13_DUPLICADO.

TR-04 · RN04 EAN-13 único

REQUISIÇÃO

```
POST http://localhost:3000/api/produtos

Content-Type: application/json
x-usuario-id: 1

{
  "sku": "RN4-024197",
  "nome": "RN04 024197",
  "ean13": "7890241972227",
  "categoria_id": 8,
  "unidade_medida_id": 1,
  "preco_venda": 10
}
```

RESPOSTA

```
HTTP 409 Conflict  ·  14 ms

{
  "erro": "Já existe um produto com este código EAN-13",
  "codigo": "EAN13_DUPLICADO",
  "detalhes": []
}
```

Figura 122 – TR-04: Evidência

TR-05 — RN05/RF12 — exclusão física desabilitada (usar inativação)	Aprovado

Dados / passos utilizados	DELETE /api/produtos/:id de produto com histórico
Resultado esperado	HTTP 409 e mensagem orientando a inativar.
Resultado obtido	HTTP 409 — Exclusão física desabilitada. Utilize a inativação do produto.

TR-05 · RN05 exclusão física

REQUISIÇÃO

```
DELETE http://localhost:3000/api/produtos/18

x-usuario-id: 1
```

RESPOSTA

```
HTTP 409 Conflict  ·  16 ms

{
  "erro": "Exclusão física desabilitada. Utilize a inativação do produto",
  "codigo": "EXCLUSAO_NAO_PERMITIDA",
  "detalhes": []
}
```

Figura 123 – TR-05: Evidência

TR-05 · Regra de exclusão física

backend/src/servicos/produtos.servico.js linhas 112–124

```
112 export async function excluir(id) {
113   await buscarPorId(id);
114
115   const possuiHistorico = await historicoRepo.existeHistoricoParaProduto(id);
116   if (possuiHistorico) {
117     if (!env.PERMITIR_EXCLUSAO_FISICA) throw new ErroAplicacao('EXCLUSAO_NAO_PERMITIDA');
118     throw new ErroAplicacao('PRODUTO_COM_HISTORICO');
119   }
120
121   if (!env.PERMITIR_EXCLUSAO_FISICA) throw new ErroAplicacao('EXCLUSAO_NAO_PERMITIDA');
122
123   await produtosRepo.excluirFisicamente(id);
124 }
```

Figura 124 – TR-05: Codigo

TR-06 — RN06/RF05 — listagem padrão exibe somente produtos ativos	**Aprovado**
Dados / passos utilizados	GET /api/produtos (sem parâmetro de situação) deve excluir o produto inativado no TI-04
Resultado esperado	Produto inativo (id do TI-04) ausente na listagem padrão.
Resultado obtido	0 resultado(s) para o produto inativo.

TR-06 · RN06 listagem padrão só ativos

REQUISIÇÃO

GET http://localhost:3000/api/produtos?nome=Mouse%20Integra%C3%A7%C3%A3o%20024197

(sem cabeçalhos adicionais)

RESPOSTA

HTTP 200 OK · 19 ms


```
{
  "dados": [],
  "paginacao": {
    "pagina": 1,
    "limite": 20,
    "total": 0,
    "total_paginas": 0
  }
}
```


Figura 125 – TR-06: Evidência

TR-07 — RN08 — nome único dentro da mesma categoria	Aprovado
Dados / passos utilizados	Mesmo nome em categorias diferentes é permitido; na mesma categoria é bloqueado (TN-11)
Resultado esperado	Cadastro com nome repetido em OUTRA categoria retorna 201.
Resultado obtido	HTTP 201.

TR-07 · RN08 mesmo nome, outra categoria

REQUISIÇÃO

```
POST http://localhost:3000/api/produtos

Content-Type: application/json
x-usuario-id: 1

{
  "sku": "RN8-024197",
  "nome": "Notebook Ciclo03 024197",
  "categoria_id": 9,
  "unidade_medida_id": 1,
  "preco_venda": 10
}
```

RESPOSTA

```
HTTP 201 Created  ·  27 ms

{
  "id": 20,
  "sku": "RN8-024197",
  "nome": "Notebook Ciclo03 024197",
  "descricao": null,
  "ean13": null,
  "categoria": {
    "id": 9,
    "nome": "Áudio e Som",
    "categoria_pai": {
      "id": 1,
      "nome": "Eletrônicos"
    }
  },
  "unidade_medida": {
    "id": 1,
    "nome": "Unidade",
    "sigla": "UN"
  },
  "preco_custo": 0,
  "preco_venda": 10,
  "estoque": 0,
  "situacao": "ativo",
  "criado_em": "2026-11-05T01:13:14.026Z",
  "atualizado_em": "2026-11-05T01:13:14.026Z"
}
```

Figura 126 – TR-07: Evidência

TR-08 — RN09/RF13 — toda alteração de preço gera histórico	Aprovado
Dados / passos utilizados	Conferido no TB-02 e TI-02 (venda 3499,90→3299,90 e 79,90→89,90)
Resultado esperado	Histórico com valor anterior e novo, data e usuário.
Resultado obtido	Alterações registradas com valor_anterior, valor_novo, motivo, usuario_id e data.

TR-08 · RN09 histórico de preço

PSQL · SIGE_PRODUTOS (CONTAINER APP-PRODUCT-POSTGRES-1)

```
SELECT produto_id, tipo_preco, valor_anterior, valor_novo, motivo, usuario_id, vigencia_inicio::timestamp(0) FROM
produto_preco_historico WHERE valor_anterior IS NOT NULL AND produto_id IN (13, 18) ORDER BY id;
 produto_id | tipo_preco | valor_anterior | valor_novo |          motivo           | usuario_id |   vigencia_in
icio
------------+------------+----------------+------------+---------------------------+------------+--------------
-------
         13 | venda      |        3499.90 |    3299.90 | Alteração de preço de venda |          1 | 2026-11-05 0
1:10:59
         18 | venda      |          79.90 |      89.90 | Alteração de preço de venda |          1 | 2026-11-05 0
1:12:59
(2 rows)
```

Figura 127 – TR-08: Evidência

TR-09 — RN14 — senha com no mínimo 8 caracteres	Aprovado
Dados / passos utilizados	POST /api/auth/cadastro com senha de 5 caracteres
Resultado esperado	HTTP 422 VALIDACAO.
Resultado obtido	HTTP 422.

TR-09 · RN14 senha ≥ 8

REQUISIÇÃO

POST http://localhost:3000/api/auth/cadastro

```
Content-Type: application/json

{
  "nome": "Senha Curta",
  "email": "curta.024197@teste.com",
  "senha": "12345"
}
```

RESPOSTA

HTTP 422 Unprocessable Entity · 12 ms

```
{
  "erro": "Dados inválidos",
  "codigo": "VALIDACAO",
  "detalhes": [
    {
      "campo": "senha",
      "mensagem": "A senha deve ter no mínimo 8 caracteres"
    }
  ]
}
```

Figura 128 – TR-09: Evidência

TR-10 — RF30 — mensagem de login genérica (não revela se o e-mail existe)	Aprovado
Dados / passos utilizados	Login com e-mail inexistente × e-mail existente com senha errada
Resultado esperado	Mesma resposta (HTTP 401 / mesma mensagem) nos dois casos.
Resultado obtido	Ambos HTTP 401 com a mesma mensagem: "E-mail ou senha inválidos".

TR-10 · E-mail inexistente

REQUISIÇÃO

POST http://localhost:3000/api/auth/login

```
Content-Type: application/json

{
  "email": "naoexiste@teste.com",
  "senha": "qualquer123"
}
```

RESPOSTA

HTTP 401 Unauthorized · 14 ms

```
{
  "erro": "E-mail ou senha inválidos",
  "codigo": "CREDENCIAIS_INVALIDAS",
  "detalhes": []
}
```

Figura 129 – TR-10: Evidência

TR-10 · Senha errada

REQUISIÇÃO

```
POST http://localhost:3000/api/auth/login

Content-Type: application/json

{
  "email": "demo@sige.com.br",
  "senha": "qualquer123"
}
```

RESPOSTA

```
HTTP 401 Unauthorized  ·  75 ms

{
  "erro": "E-mail ou senha inválidos",
  "codigo": "CREDENCIAIS_INVALIDAS",
  "detalhes": []
}
```

Figura 130 – TR-10: Evidência

TR-11 — RNF03 — operações de escrita exigem usuário identificado	Aprovado
Dados / passos utilizados	POST /api/categorias sem x-usuario-id e com id inexistente (99999)
Resultado esperado	HTTP 401 nos dois casos.
Resultado obtido	HTTP 401 e HTTP 401.

TR-11 · Sem cabeçalho

REQUISIÇÃO

```
POST http://localhost:3000/api/categorias

Content-Type: application/json

{
  "nome": "Sem auth 024197"
}
```

RESPOSTA

```
HTTP 401 Unauthorized  ·  17 ms

{
  "erro": "Requisição não autenticada",
  "codigo": "NAO_AUTENTICADO",
  "detalhes": []
}
```

Figura 131 – TR-11: Evidência

TR-11 · Usuário inexistente

REQUISIÇÃO

```
POST http://localhost:3000/api/categorias

Content-Type: application/json
x-usuario-id: 99999

{
  "nome": "Sem auth 024197"
}
```

RESPOSTA

```
HTTP 401 Unauthorized  ·  17 ms

{
  "erro": "Requisição não autenticada",
  "codigo": "NAO_AUTENTICADO",
  "detalhes": []
}
```

Figura 132 – TR-11: Evidência

TR-11 · Exigência de usuário identificado nas escritas

backend/src/util/autenticacao.js linhas 5–14

```
5  export async function exigirUsuarioAutenticado(request) {
6    const usuarioId = Number(request.headers['x-usuario-id']);
7    if (!usuarioId || Number.isNaN(usuarioId)) throw new ErroAplicacao('NAO_AUTENTICADO');
8
9    const usuario = await usuariosRepo.buscarPorId(usuarioId);
10   if (!usuario) throw new ErroAplicacao('NAO_AUTENTICADO');
11
12   request.usuario = usuario;
13   return usuario.id;
14 }
```

Figura 133 – TR-11: Codigo

TR-13 — RF04 — busca na listagem por nome OU SKU (conforme o placeholder da tela)	**Reprovado**
Dados / passos utilizados	Listagem · digitar o SKU NTB-024197 no campo "Buscar por nome ou SKU"; comparar com GET /api/produtos?sku=NTB-024197
Resultado esperado	Produto encontrado ao buscar pelo SKU, como o próprio campo de busca anuncia.
Resultado obtido	DEFEITO: a tela retornou "Nenhum resultado para o filtro aplicado" ao buscar pelo SKU, embora a API encontre o produto via ?sku= (1 resultado). O front-end só envia o parâmetro "nome".

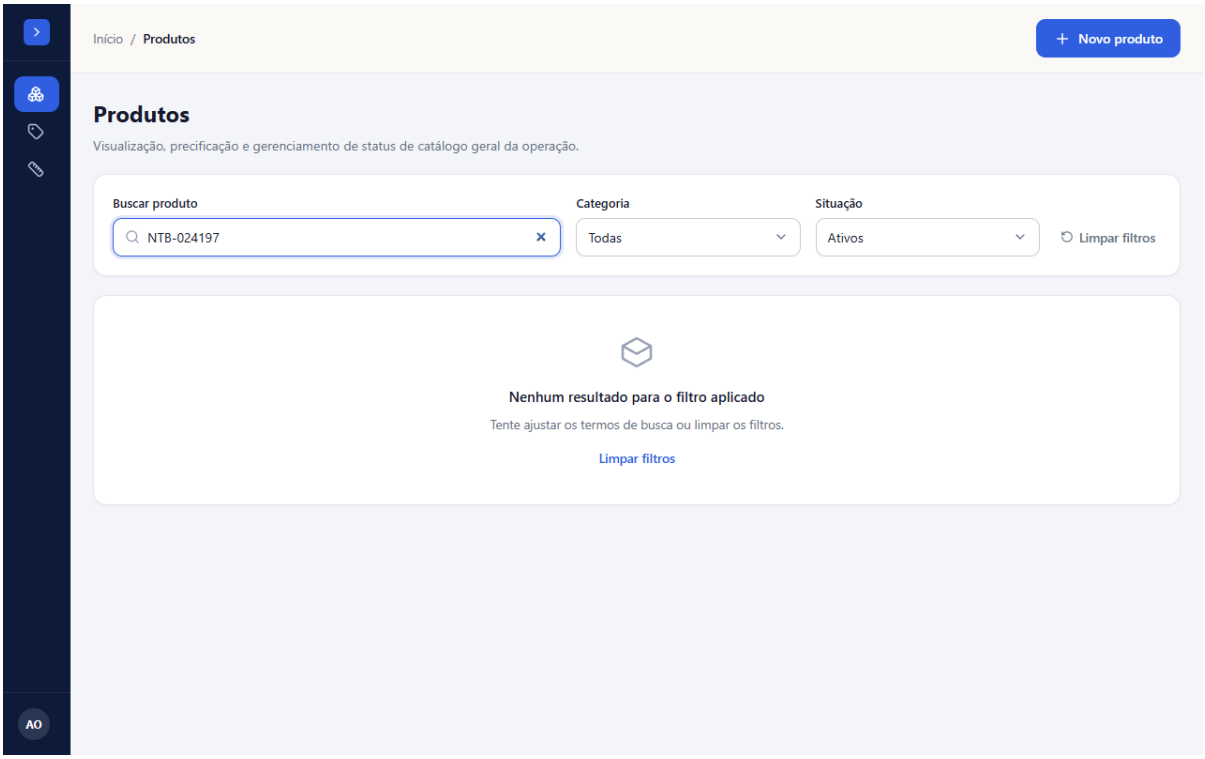


Figura 134 – TR-13: Busca sku tela

TR-13 · A API aceita o filtro sku

REQUISIÇÃO

`GET http://localhost:3000/api/produtos?sku=NTB-024197&situacao=todos`

(sem cabeçalhos adicionais)

RESPOSTA

`HTTP 200 OK · 9 ms`


```
{
  "dados": [
    {
      "id": 13,
      "sku": "NTB-024197",
      "nome": "Notebook Ciclo03 024197",
      "descricao": "Notebook 15 polegadas usado no Ciclo 03 de testes.",
      "ean13": "7890241971114",
      "categoria": {
        "id": 8,
        "nome": "Acessórios Mobile",
        "categoria_pai": {
          "id": 1,
          "nome": "Eletrônicos"
        }
      },
      "unidade_medida": {
        "id": 1,
        "nome": "Unidade",
        "sigla": "UN"
      },
      "preco_custo": 2500,
      "preco_venda": 3299.9,
      "estoque": 15,
      "situacao": "ativo",
      "criado_em": "2026-11-05T01:10:52.731Z",
  … (10 linhas omitidas)
```


Figura 135 – TR-13: Busca sku api

TR-13 · Causa do defeito: a tela só envia "nome"; a API faz AND entre nome e sku

frontend/src/paginas/ListagemProdutos.vue linhas 54–60 — Tela: o parâmetro sku nunca é enviado

```
54 const filtroAtivo = computed(() => ({
55   nome: filtros.nome || undefined,
56   categoria_id: filtros.categoria_id || undefined,
57   situacao: filtros.situacao,
58   pagina: filtros.pagina,
59   limite: filtros.limite,
60 }));
```

frontend/src/paginas/ListagemProdutos.vue linhas 129–129 — Placeholder promete busca por SKU

```
129           <input id="busca-produto" v-model="termoBusca" type="search" placeholder="Buscar por nome ou SKU" />
```

backend/src/repositorios/produtos.repo.js linhas 42–48 — Banco: nome ILIKE separado de sku ILIKE

```
42   if (nome) {
43     parametros.push(`%${nome}%`);
44     condicoes.push(`p.nome ILIKE $${parametros.length}`);
45   }
46   if (sku) {
47     parametros.push(`%${sku}%`);
48     condicoes.push(`p.sku ILIKE $${parametros.length}`);
```

Figura 136 – TR-13: Codigo

TR-12 — RF32 — sessão do usuário (recarregar a página)	**Aprovado**
Dados / passos utilizados	Logar, recarregar /produtos (F5) e verificar o comportamento
Resultado esperado	Conforme documentado no código: sessão mantida apenas em memória, portanto exige novo login após recarregar.
Resultado obtido	Após F5 a URL foi /login: sessão perdida, novo login exigido (comportamento intencional documentado — ver observação). Obs.: Registrado como melhoria (UX): a sessão não persiste ao recarregar.

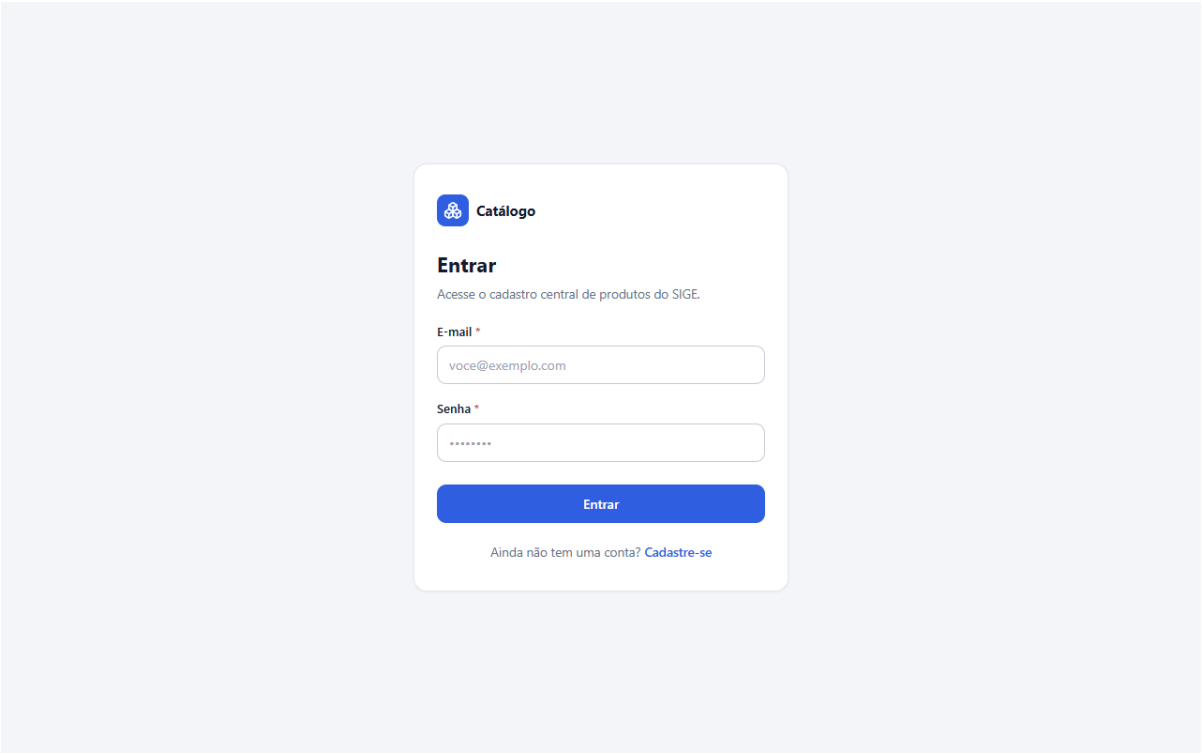


Figura 137 – TR-12: Reload

TR-12 · Sessão guardada apenas em memória

frontend/src/stores/sessao.js linhas 5–20 — nada vai para o localStorage

```
5  export const useSessaoStore = defineStore('sessao', {
6    state: () => ({
7      usuario: null,
8    }),
9    getters: {
10     estaAutenticado: (state) => Boolean(state.usuario?.id),
11   },
12   actions: {
13     definirUsuario(usuario) {
14       this.usuario = usuario;
15     },
16     encerrarSessao() {
17       this.usuario = null;
18     },
19   },
20 });
```

Figura 138 – TR-12: Codigo

8. Testes de impacto

O que é: Verificam se uma alteração ou correção realizada no sistema prejudicou funcionalidades que já funcionavam. Após uma mudança, partes relacionadas devem ser testadas novamente.

Aplicação neste sistema: Após alterar preços, descrições, situação e nomes de categorias, foram reconferidos o histórico, os demais campos e as consultas principais.

TM-01 — Editar sem mexer em preço NÃO gera novo histórico	**Aprovado**
Dados / passos utilizados	PUT /api/produtos/{id} alterando só a descrição e o estoque; contar histórico antes e depois
Resultado esperado	Quantidade de registros de histórico permanece igual.
Resultado obtido	Histórico: 3 → 3 registro(s).

TM-01 · Histórico ANTES

REQUISIÇÃO

GET http://localhost:3000/api/produtos/18/precos

(sem cabeçalhos adicionais)

RESPOSTA

HTTP 200 OK · 22 ms


```
{
  "produto_id": 18,
  "historico": [
    {
      "id": 47,
      "tipo_preco": "venda",
      "valor_anterior": 79.9,
      "valor_novo": 89.9,
      "vigencia_inicio": "2026-11-05T01:12:59.328Z",
      "motivo": "Alteração de preço de venda",
      "usuario": {
        "id": 1,
        "nome": "Adriano Operador"
      }
    },
    {
      "id": 45,
      "tipo_preco": "venda",
      "valor_anterior": null,
      "valor_novo": 79.9,
      "vigencia_inicio": "2026-11-05T01:12:56.498Z",
      "motivo": "Cadastro inicial",
      "usuario": {
        "id": 1,
        "nome": "Adriano Operador"
      }
  … (15 linhas omitidas)
```


Figura 139 – TM-01: Evidência

TM-01 · Alteração sem mexer em preço

REQUISIÇÃO

PUT http://localhost:3000/api/produtos/18

```
Content-Type: application/json
x-usuario-id: 1

{
  "descricao": "Descrição alterada no teste de impacto",
  "estoque": 20
}
```

RESPOSTA

HTTP 200 OK · 25 ms


```
{
  "id": 18,
  "sku": "INT-024197",
  "nome": "Mouse Integração 024197",
  "descricao": "Descrição alterada no teste de impacto",
  "ean13": null,
  "categoria": {
    "id": 8,
    "nome": "Acessórios Mobile",
    "categoria_pai": {
      "id": 1,
      "nome": "Eletrônicos"
    }
  },
  "unidade_medida": {
    "id": 1,
    "nome": "Unidade",
    "sigla": "UN"
  },
  "preco_custo": 30,
  "preco_venda": 89.9,
  "estoque": 20,
  "situacao": "inativo",
  "criado_em": "2026-11-05T01:12:56.498Z",
  "atualizado_em": "2026-11-05T01:13:27.487Z"
}
```


Figura 140 – TM-01: Evidência

TM-01 · Histórico DEPOIS

REQUISIÇÃO

`GET` http://localhost:3000/api/produtos/18/precos

(sem cabeçalhos adicionais)

RESPOSTA

HTTP 200 OK · 12 ms


```
{
  "produto_id": 18,
  "historico": [
    {
      "id": 47,
      "tipo_preco": "venda",
      "valor_anterior": 79.9,
      "valor_novo": 89.9,
      "vigencia_inicio": "2026-11-05T01:12:59.328Z",
      "motivo": "Alteração de preço de venda",
      "usuario": {
        "id": 1,
        "nome": "Adriano Operador"
      }
    },
    {
      "id": 45,
      "tipo_preco": "venda",
      "valor_anterior": null,
      "valor_novo": 79.9,
      "vigencia_inicio": "2026-11-05T01:12:56.498Z",
      "motivo": "Cadastro inicial",
      "usuario": {
        "id": 1,
        "nome": "Adriano Operador"
      }
  … (15 linhas omitidas)
```


Figura 141 – TM-01: Evidência

TM-01 · Código que só grava histórico quando o preço muda

backend/src/servicos/precos.servico.js linhas 27–55

```
27 export async function registrarAlteracoesDePreco(client, { produtoAntigo, camposAtualizados, usuarioId, motivo
   }) {
28   if (
29     camposAtualizados.preco_venda !== undefined &&
30     Number(camposAtualizados.preco_venda) !== Number(produtoAntigo.preco_venda)
31   ) {
32     await historicoRepo.inserir(client, {
33       produto_id: produtoAntigo.id,
34       tipo_preco: 'venda',
35       valor_anterior: produtoAntigo.preco_venda,
36       valor_novo: camposAtualizados.preco_venda,
37       motivo: motivo ?? 'Alteração de preço de venda',
38       usuario_id: usuarioId,
39     });
40   }
41
42   if (
43     camposAtualizados.preco_custo !== undefined &&
44     Number(camposAtualizados.preco_custo) !== Number(produtoAntigo.preco_custo)
45   ) {
46     await historicoRepo.inserir(client, {
47       produto_id: produtoAntigo.id,
48       tipo_preco: 'custo',
49       valor_anterior: produtoAntigo.preco_custo,
50       valor_novo: camposAtualizados.preco_custo,
51       motivo: motivo ?? 'Alteração de preço de custo',
52       usuario_id: usuarioId,
53     });
54   }
55 }
```

Figura 142 – TM-01: Codigo

TM-02 — Alterar preço de venda não altera preço de custo nem outros campos	Aprovado
Dados / passos utilizados	PUT preco_venda = 99,90 no produto do TI; comparar os demais campos
Resultado esperado	Custo (30,00), estoque (20) e nome permanecem; histórico ganha só 1 registro do tipo "venda".
Resultado obtido	preco_custo=30, estoque=20, preco_venda=99.9. Histórico recebeu apenas registros de venda.

TM-02 · Alterar só o preço de venda

REQUISIÇÃO

PUT http://localhost:3000/api/produtos/18

```
Content-Type: application/json
x-usuario-id: 1

{
  "preco_venda": 99.9
}
```

RESPOSTA

HTTP 200 OK · 44 ms


```
{
  "id": 18,
  "sku": "INT-024197",
  "nome": "Mouse Integração 024197",
  "descricao": "Descrição alterada no teste de impacto",
  "ean13": null,
  "categoria": {
    "id": 8,
    "nome": "Acessórios Mobile",
    "categoria_pai": {
      "id": 1,
      "nome": "Eletrônicos"
    }
  },
  "unidade_medida": {
    "id": 1,
    "nome": "Unidade",
    "sigla": "UN"
  },
  "preco_custo": 30,
  "preco_venda": 99.9,
  "estoque": 20,
  "situacao": "inativo",
  "criado_em": "2026-11-05T01:12:56.498Z",
  "atualizado_em": "2026-11-05T01:13:27.932Z"
}
```


Figura 143 – TM-02: Evidência

TM-02 · Campos e histórico após a alteração

PSQL · SIGE_PRODUTOS (CONTAINER APP-PRODUCT-POSTGRES-1)

```
SELECT nome, preco_custo, preco_venda, estoque FROM produto WHERE id = 18;
          nome           | preco_custo | preco_venda | estoque
-------------------------+-------------+-------------+---------
 Mouse Integração 024197 |       30.00 |       99.90 |  20.000
(1 row)

SELECT tipo_preco, valor_anterior, valor_novo FROM produto_preco_historico WHERE produto_id = 18 ORDER BY id;
 tipo_preco | valor_anterior | valor_novo
------------+----------------+------------
 venda      |                |      79.90
 custo      |                |      30.00
 venda      |          79.90 |      89.90
 venda      |          89.90 |      99.90
(4 rows)
```

Figura 144 – TM-02: Evidência

TM-03 — Inativar e reativar preserva dados e histórico	Aprovado
Dados / passos utilizados	Inativar e reativar o produto do TA-09; comparar produto e histórico
Resultado esperado	Preços, estoque e quantidade de histórico idênticos após o ciclo.
Resultado obtido	Preços e estoque idênticos; situacao=ativo; histórico com 3 registros.

TM-03 · Produto ANTES

REQUISIÇÃO

GET http://localhost:3000/api/produtos/14

(sem cabeçalhos adicionais)

RESPOSTA

HTTP 200 OK · 25 ms


```
{
  "id": 14,
  "sku": "API-024197",
  "nome": "Produto API 024197",
  "descricao": "Criado na preparação",
  "ean13": "7890241972227",
  "categoria": {
    "id": 9,
    "nome": "Áudio e Som",
    "categoria_pai": {
      "id": 1,
      "nome": "Eletrônicos"
    }
  },
  "unidade_medida": {
    "id": 1,
    "nome": "Unidade",
    "sigla": "UN"
  },
  "preco_custo": 20,
  "preco_venda": 54.9,
  "estoque": 90,
  "situacao": "ativo",
  "criado_em": "2026-11-05T01:12:35.218Z",
  "atualizado_em": "2026-11-05T01:12:49.971Z"
}
```


Figura 145 – TM-03: Evidência

TM-03 · Produto DEPOIS de inativar+reativar

REQUISIÇÃO

GET http://localhost:3000/api/produtos/14

(sem cabeçalhos adicionais)

RESPOSTA

HTTP 200 OK · 21 ms


```
{
  "id": 14,
  "sku": "API-024197",
  "nome": "Produto API 024197",
  "descricao": "Criado na preparação",
  "ean13": "7890241972227",
  "categoria": {
    "id": 9,
    "nome": "Áudio e Som",
    "categoria_pai": {
      "id": 1,
      "nome": "Eletrônicos"
    }
  },
  "unidade_medida": {
    "id": 1,
    "nome": "Unidade",
    "sigla": "UN"
  },
  "preco_custo": 20,
  "preco_venda": 54.9,
  "estoque": 90,
  "situacao": "ativo",
  "criado_em": "2026-11-05T01:12:35.218Z",
  "atualizado_em": "2026-11-05T01:13:28.721Z"
}
```


Figura 146 – TM-03: Evidência

TM-03 · Histórico

REQUISIÇÃO

GET http://localhost:3000/api/produtos/14/precos

(sem cabeçalhos adicionais)

RESPOSTA

HTTP 200 OK · 21 ms


```
{
  "produto_id": 14,
  "historico": [
    {
      "id": 41,
      "tipo_preco": "venda",
      "valor_anterior": 49.9,
      "valor_novo": 54.9,
      "vigencia_inicio": "2026-11-05T01:12:35.390Z",
      "motivo": "Alteração de preço de venda",
      "usuario": {
        "id": 1,
        "nome": "Adriano Operador"
      }
    },
    {
      "id": 39,
      "tipo_preco": "venda",
      "valor_anterior": null,
      "valor_novo": 49.9,
      "vigencia_inicio": "2026-11-05T01:12:35.218Z",
      "motivo": "Cadastro inicial",
      "usuario": {
        "id": 1,
        "nome": "Adriano Operador"
      }
  … (15 linhas omitidas)
```


Figura 147 – TM-03: Evidência

TM-04 — Renomear categoria reflete nos produtos já cadastrados	Aprovado
Dados / passos utilizados	PUT /api/categorias/{id} renomeando uma categoria que possui produto; conferir o detalhe do produto
Resultado esperado	Produto exibe o novo nome da categoria, sem perder o vínculo.
Resultado obtido	Produto passou a exibir "Categoria Áudio 024197"; nome original restaurado (HTTP 200); salvar o próprio nome atual retornou HTTP 200.

TM-04 · Renomear categoria Áudio e Som

REQUISIÇÃO

PUT http://localhost:3000/api/categorias/9

```
Content-Type: application/json
x-usuario-id: 1

{
  "nome": "Categoria Áudio 024197"
}
```

RESPOSTA

HTTP 200 OK · 21 ms


```
{
  "id": 9,
  "nome": "Categoria Áudio 024197",
  "categoria_pai_id": 1,
  "ativo": true
}
```


Figura 148 – TM-04: Evidência

TM-04 · Produto da categoria após renomear

REQUISIÇÃO

GET http://localhost:3000/api/produtos/14

```
(sem cabeçalhos adicionais)
```

RESPOSTA

HTTP 200 OK · 23 ms


```
{
  "id": 14,
  "sku": "API-024197",
  "nome": "Produto API 024197",
  "descricao": "Criado na preparação",
  "ean13": "7890241972227",
  "categoria": {
    "id": 9,
    "nome": "Categoria Áudio 024197",
    "categoria_pai": {
      "id": 1,
      "nome": "Eletrônicos"
    }
  },
  "unidade_medida": {
    "id": 1,
    "nome": "Unidade",
    "sigla": "UN"
  },
  "preco_custo": 20,
  "preco_venda": 54.9,
  "estoque": 90,
  "situacao": "ativo",
  "criado_em": "2026-11-05T01:12:35.218Z",
  "atualizado_em": "2026-11-05T01:13:28.721Z"
}
```


Figura 149 – TM-04: Evidência

TM-04 · Restaurar o nome original

REQUISIÇÃO

```
PUT http://localhost:3000/api/categorias/9

Content-Type: application/json
x-usuario-id: 1

{
  "nome": "Áudio e Som"
}
```

RESPOSTA

```
HTTP 200 OK  ·  23 ms

{
  "id": 9,
  "nome": "Áudio e Som",
  "categoria_pai_id": 1,
  "ativo": true
}
```

Figura 150 – TM-04: Evidência

TM-05 — Regressão geral (smoke) após todas as alterações	Aprovado
Dados / passos utilizados	Reexecutar: health, listagem, detalhe, histórico, categorias, unidades e telas principais
Resultado esperado	Todos os endpoints de consulta seguem respondendo 200 e a listagem da tela carrega.
Resultado obtido	200 /api/health · 200 /api/produtos · 200 /api/produtos/13 · 200 /api/produtos/13/precos · 200 /api/categorias · 200 /api/unidades-medida

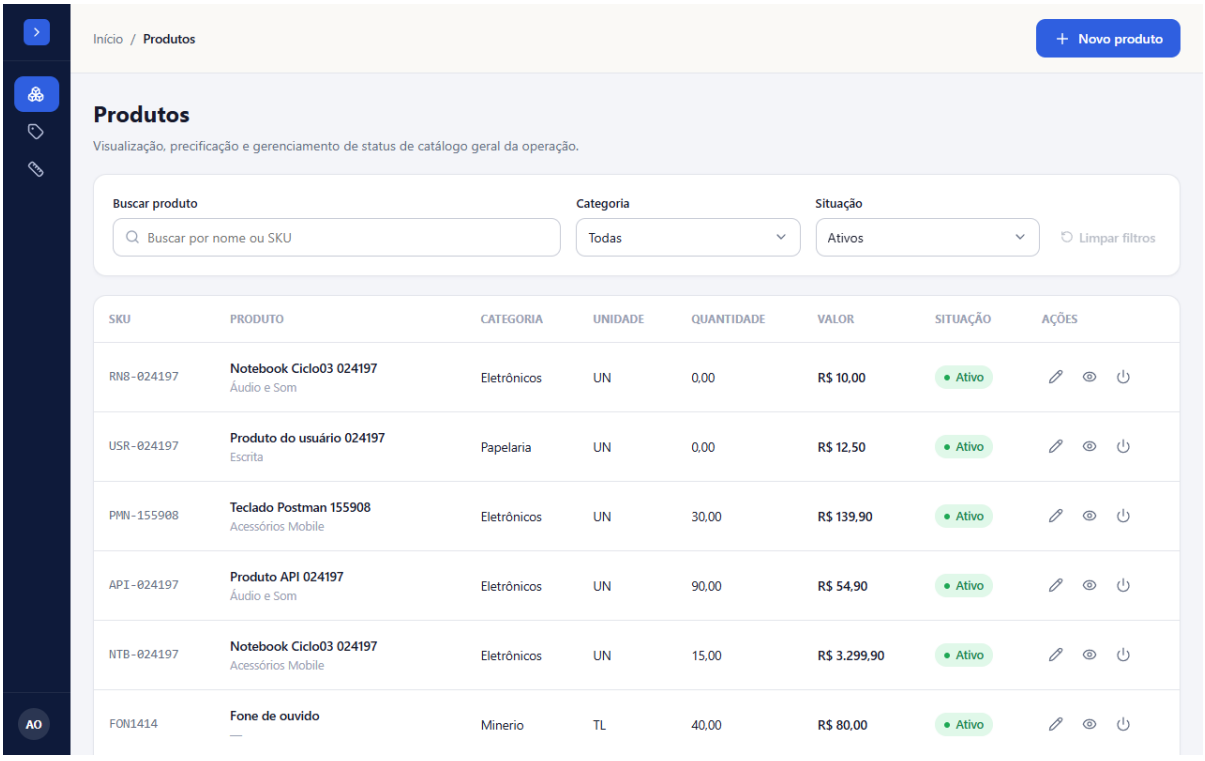


Figura 151 – TM-05: Listagem final

Registro dos testes realizados

Tabela consolidada com todos os testes executados (o detalhamento e as evidências de cada um estão na seção anterior).

Tipo / ID	O que foi testado?	Dados / passos utilizados	Resultado esperado	Resultado obtido	Status
CT-01	Login com credenciais válidas	Tela /login · e-mail demo@sige.com.br · senha demo1234 · clicar em "Entrar"	Usuário autenticado e redirecionado para a listagem de produtos (/produtos).	Redirecionado para /produtos; listagem de produtos exibida.	**Aprovado**
CT-02	Cadastro de novo usuário pela tela	Tela /cadastro · nome "Testador Ciclo 03" · e-mail ciclo03.024197@teste.com · senha senha1234	Conta criada, login automático e redirecionamento para /produtos.	Usuário criado e autenticado automaticamente; listagem de produtos exibida.	**Aprovado**
CT-03	Listagem de produtos e busca por nome	Usuário logado · tela /produtos · digitar "Fone" no campo de busca	Tabela filtrada exibindo apenas produtos cujo nome contém "Fone".	1 linha(s) retornada(s); todas contêm "Fone".	**Aprovado**
CT-04	Consulta de categorias e subcategorias	Usuário logado · menu Categorias (/categorias) · expandir "Eletrônicos"	Categorias principais listadas e subcategorias visíveis ao expandir o card.	Card "Eletrônicos" expandido exibindo as subcategorias "Acessórios Mobile" e "Áudio e Som".	**Aprovado**
CT-05	Consulta de unidades de medida	Usuário logado · menu Unidades de medida (/unidades-medida)	Unidades de medida cadastradas exibidas em lista (ex.: Unidade/UN).	Tela exibe as unidades cadastradas, incluindo "Unidade (UN)".	**Aprovado**
TP-01	Cadastrar produto com todos os campos válidos	Tela /produtos/novo · nome "Notebook Ciclo03 024197" · SKU NTB-024197 · EAN-13 7890241971114 · categoria Acessórios Mobile · unidade Unidade (UN) · custo 2500 · venda 3499,90 · estoque 15	Produto salvo, página de detalhe exibida com confirmação e produto presente na listagem.	Produto criado com ID 13; detalhe exibido e produto localizado na listagem pelo SKU.	**Aprovado**
TP-02	Editar o preço de venda de um produto	Tela /produtos/{id}/editar · preço de venda 3499,90 → 3299,90	Alteração salva e novo preço exibido no detalhe (R$ 3.299,90).	Detalhe do produto exibe R$ 3.299,90.	**Aprovado**
TP-03	Consultar o histórico de preço do produto	Tela /produtos/{id}/precos após o cadastro e a alteração de preço	Histórico listando o registro inicial e a alteração de venda (3.499,90 → 3.299,90).	Histórico apresenta o valor inicial (3.499,90) e o novo valor (3.299,90).	**Aprovado**
TP-04	Inativar produto pela listagem	Listagem · busca pelo nome · ação "Inativar produto" · confirmar no modal	Produto some da listagem padrão (apenas ativos) e aparece com selo "Inativo" no filtro Inativos.	Produto removido dos ativos e exibido com selo "Inativo" no filtro Inativos.	**Aprovado**
TP-05	Reativar produto inativo	Filtro Inativos · ação "Reativar produto" · confirmar	Produto volta a aparecer na listagem de ativos com selo "Ativo".	Produto reativado e visível na listagem de ativos.	**Aprovado**
TP-06	Cadastrar categoria principal e subcategoria pela interface	Tela /categorias · "Nova categoria" · nome "Categoria Ciclo03 024197" · depois adicionar a subcategoria "Sub 024197" no card	Categoria criada e subcategoria vinculada ao card da categoria principal.	Categoria e subcategoria criadas e exibidas na árvore.	**Aprovado**
TP-07	Cadastrar unidade de medida pela interface	Tela /unidades-medida · "Nova unidade" · nome "Caixa C024197" · sigla "CX197"	Unidade criada e presente na lista.	Unidade criada e exibida na lista.	**Aprovado**
TP-08	Filtrar produtos por categoria	Listagem · filtro Categoria = "Acessórios Mobile"	Somente produtos da categoria selecionada na tabela.	1 produto(s) listado(s), todos da categoria filtrada.	**Aprovado**
TN-01	Login com senha incorreta	Tela /login · e-mail demo@sige.com.br · senha "errada123"	Login negado com mensagem genérica (sem revelar qual campo errou); usuário permanece em /login.	Mensagem "E-mail ou senha inválidos" exibida; permaneceu em /login.	**Aprovado**

Tipo / ID	O que foi testado?	Dados / passos utilizados	Resultado esperado	Resultado obtido	Status
TN-02	Acesso a rota protegida sem sessão	Sem login, acessar diretamente /produtos/novo	Redirecionamento para /login.	URL final: /login?redirecionarPara=/produtos/novo	**Aprovado**
TN-03	Cadastro de usuário com senha curta e e-mail inválido	Tela /cadastro · nome "Ab" · e-mail "semarroba" · senha "123"	Mensagens de validação por campo e cadastro bloqueado.	Exibidas as mensagens "Informe seu nome", "E-mail inválido" e "A senha deve ter no mínimo 8 caracteres".	**Aprovado**
TN-04	Cadastrar produto com campos obrigatórios vazios	Tela /produtos/novo · clicar em "Salvar produto" sem preencher nada	Mensagens de obrigatoriedade (nome, SKU, categoria, unidade, preço de venda) e nenhum produto criado.	Exibidas mensagens para nome, categoria, unidade de medida e demais campos obrigatórios; permaneceu em /produtos/novo.	**Aprovado**
TN-05	Preço de venda zero/negativo e estoque negativo	Tela /produtos/novo · preço de venda = -10 · estoque = -5 · preço de custo = -1	Mensagens "Preço de venda deve ser maior que zero", "Estoque não pode ser negativo" e "Preço de custo não pode ser negativo".	As três mensagens de validação foram exibidas e o envio foi bloqueado.	**Aprovado**
TN-06	Cadastrar produto com EAN-13 de dígito verificador inválido	Tela /produtos/novo · EAN-13 = 7890241971115	Mensagem "Dígito verificador do EAN-13 inválido" e envio bloqueado.	Mensagem de dígito verificador inválido exibida.	**Aprovado**
TN-07	Cadastrar produto com SKU já existente	Tela /produtos/novo · SKU NTB-024197 (já usado no TP-01) · demais campos válidos	Mensagem "Já existe um produto com este SKU" no campo SKU; produto não duplicado.	Mensagem "Já existe um produto com este SKU" exibida.	**Aprovado**
TN-08	Cadastrar categoria sem nome	Tela /categorias · "Nova categoria" · salvar com nome vazio	Mensagem "Informe o nome da categoria" e categoria não criada.	Mensagem de validação exibida; modal permaneceu aberto.	**Aprovado**
TN-09	Cadastrar categoria com nome duplicado	POST /api/categorias {"nome":"Eletrônicos"} (nome já existente)	HTTP 409 com código NOME_CATEGORIA_DUPLICADO.	HTTP 409 — Já existe uma categoria com este nome (NOME_CATEGORIA_DUPLICADO)	**Aprovado**
TN-10	Cadastrar usuário com e-mail já existente	POST /api/auth/cadastro com e-mail demo@sige.com.br	HTTP 409 EMAIL_JA_CADASTRADO.	HTTP 409 — E-mail já cadastrado (EMAIL_JA_CADASTRADO)	**Aprovado**
TN-11	Nome de produto duplicado na mesma categoria	POST /api/produtos com nome "Notebook Ciclo03 024197" na categoria 8 e SKU novo	HTTP 409 NOME_DUPLICADO_NA_CATEGORIA.	HTTP 409 — Já existe um produto com este nome nesta categoria (NOME_DUPLICADO_NA_CATEGORIA)	**Aprovado**
TA-01	[Autenticação] Login válido (POST)	Postman · POST http://localhost:3000/api/auth/login · corpo: { "email": "demo@sige.com.br", "senha": "demo1234" }	Status code é 200; Retorna o id do usuário; Não expõe a senha; Tempo de resposta < 1000 ms.	HTTP 200 OK em 102 ms. Testes do Postman: 4/4 PASS.	**Aprovado**
TA-02	[Autenticação] Login com senha errada (negativo)	Postman · POST http://localhost:3000/api/auth/login · corpo: { "email": "demo@sige.com.br", "senha": "senhaErrada" }	Status code é 401; Código de erro é CREDENCIAIS_INVALIDAS.	HTTP 401 Unauthorized em 90 ms. Testes do Postman: 2/2 PASS.	**Aprovado**
TA-03	[Autenticação] Cadastrar usuário (POST)	Postman · POST http://localhost:3000/api/auth/cadastro · corpo: { "nome": "Usuário Postman", "email": "postman.155908@teste.com", "senha": "senha1234" }	Status code é 201; E-mail gravado em minúsculas.	HTTP 201 Created em 81 ms. Testes do Postman: 2/2 PASS.	**Aprovado**
TA-04	[Autenticação] Cadastrar usuário com e-mail repetido (negativo)	Postman · POST http://localhost:3000/api/auth/cadastro · corpo: { "nome": "Usuário Postman", "email": "postman.155908@teste.com", "senha": "senha1234" }	Status code é 409; Código de erro é EMAIL_JA_CADASTRADO.	HTTP 409 Conflict em 7 ms. Testes do Postman: 2/2 PASS.	**Aprovado**

Tipo / ID	O que foi testado?	Dados / passos utilizados	Resultado esperado	Resultado obtido	Status
TA-05	[Autenticação] Cadastrar usuário com senha curta (negativo)	Postman · POST http://localhost:3000/api/auth/cadastro · corpo: { "nome": "Senha Curta", "email": "curta.155908@teste.com", "senha": "123" }	Status code é 422; Código de erro é VALIDACAO.	HTTP 422 Unprocessable Entity em 11 ms. Testes do Postman: 2/2 PASS.	**Aprovado**
TA-06	[Produtos (CRUD)] CREATE · Cadastrar produto (POST)	Postman · POST http://localhost:3000/api/produtos · x-usuario-id: 1 · corpo: { "sku": "PMN-155908", "nome": "Teclado Postman 155908", "descricao": "Criado pela coleção Postman", "ean13": "7891559085555", "categoria_id": 8, "unida…	Status code é 201; Retorna o produto com id e situação ativo; Preço de venda gravado.	HTTP 201 Created em 20 ms. Testes do Postman: 3/3 PASS.	**Aprovado**
TA-07	[Produtos (CRUD)] CREATE · Sem autenticação (negativo)	Postman · POST http://localhost:3000/api/produtos · corpo: { "sku": "SEMAUTH-1", "nome": "Sem auth", "categoria_id": 8, "unidade_medida_id": 1, "preco_venda": 10 }	Status code é 401; Código de erro é NAO_AUTENTICADO.	HTTP 401 Unauthorized em 8 ms. Testes do Postman: 2/2 PASS.	**Aprovado**
TA-08	[Produtos (CRUD)] CREATE · Dados inválidos (negativo)	Postman · POST http://localhost:3000/api/produtos · x-usuario-id: 1 · corpo: { "sku": "minúsculo", "nome": "", "categoria_id": 8, "unidade_medida_id": 1, "preco_venda": -5 }	Status code é 422; Código de erro é VALIDACAO; Detalha os campos inválidos.	HTTP 422 Unprocessable Entity em 10 ms. Testes do Postman: 3/3 PASS.	**Aprovado**
TA-09	[Produtos (CRUD)] CREATE · SKU duplicado (negativo)	Postman · POST http://localhost:3000/api/produtos · x-usuario-id: 1 · corpo: { "sku": "PMN-155908", "nome": "Outro nome 155908", "categoria_id": 8, "unidade_medida_id": 1, "preco_venda": 10 }	Status code é 409; Código de erro é SKU_DUPLICADO.	HTTP 409 Conflict em 10 ms. Testes do Postman: 2/2 PASS.	**Aprovado**
TA-10	[Produtos (CRUD)] CREATE · EAN-13 com dígito inválido (negativo)	Postman · POST http://localhost:3000/api/produtos · x-usuario-id: 1 · corpo: { "sku": "EAN-155908", "nome": "EAN errado 155908", "ean13": "7891234567890", "categoria_id": 8, "unidade_medida_id": 1, "preco_venda": 10 }	Status code é 422; Mensagem do dígito verificador.	HTTP 422 Unprocessable Entity em 9 ms. Testes do Postman: 2/2 PASS.	**Aprovado**
TA-11	[Produtos (CRUD)] READ · Listar produtos paginado (GET)	Postman · GET http://localhost:3000/api/produtos?pagina=1&limite=5	Status code é 200; Estrutura dados + paginacao; Tempo de resposta < 1000 ms.	HTTP 200 OK em 10 ms. Testes do Postman: 3/3 PASS.	**Aprovado**
TA-12	[Produtos (CRUD)] READ · Pesquisar por SKU (GET)	Postman · GET http://localhost:3000/api/produtos?sku=PMN-155908	Status code é 200; Encontra exatamente o produto criado.	HTTP 200 OK em 12 ms. Testes do Postman: 2/2 PASS.	**Aprovado**
TA-13	[Produtos (CRUD)] READ · Buscar por ID (GET)	Postman · GET http://localhost:3000/api/produtos/15	Status code é 200; ID confere.	HTTP 200 OK em 8 ms. Testes do Postman: 2/2 PASS.	**Aprovado**
TA-14	[Produtos (CRUD)] READ · ID inexistente (negativo)	Postman · GET http://localhost:3000/api/produtos/999999	Status code é 404; Código de erro é PRODUTO_NAO_ENCONTRADO.	HTTP 404 Not Found em 25 ms. Testes do Postman: 2/2 PASS.	**Aprovado**
TA-15	[Produtos (CRUD)] READ · ID inválido (negativo)	Postman · GET http://localhost:3000/api/produtos/abc	Status code é 400.	HTTP 400 Bad Request em 4 ms. Testes do Postman: 1/1 PASS.	**Aprovado**
TA-16	[Produtos (CRUD)] UPDATE · Alterar preço e estoque (PUT)	Postman · PUT http://localhost:3000/api/produtos/15 · x-usuario-id: 1 · corpo: { "preco_venda": 139.9, "estoque": 30 }	Status code é 200; Novo preço retornado; SKU não mudou.	HTTP 200 OK em 16 ms. Testes do Postman: 3/3 PASS.	**Aprovado**

Tipo / ID	O que foi testado?	Dados / passos utilizados	Resultado esperado	Resultado obtido	Status
TA-17	[Produtos (CRUD)] UPDATE · Tentar alterar SKU (negativo)	Postman · PUT http://localhost:3000/api/produtos/15 · x-usuario-id: 1 · corpo: { "sku": "NOVO-SKU" }	Status 4xx (SKU imutável).	HTTP 422 Unprocessable Entity em 8 ms. Testes do Postman: 1/1 PASS.	**Aprovado**
TA-18	[Produtos (CRUD)] UPDATE · Preço de venda zero (negativo)	Postman · PUT http://localhost:3000/api/produtos/15 · x-usuario-id: 1 · corpo: { "preco_venda": 0 }	Status code é 422; Código de erro é VALIDACAO.	HTTP 422 Unprocessable Entity em 7 ms. Testes do Postman: 2/2 PASS.	**Aprovado**
TA-19	[Produtos (CRUD)] READ · Histórico de preço (GET)	Postman · GET http://localhost:3000/api/produtos/15/precos	Status code é 200; 3 registros: venda inicial, custo inicial e alteração; Alteração preserva valor anterior.	HTTP 200 OK em 10 ms. Testes do Postman: 3/3 PASS.	**Aprovado**
TA-20	[Produtos (CRUD)] DELETE lógico · Inativar (PATCH)	Postman · PATCH http://localhost:3000/api/produtos/15/inativar · x-usuario-id: 1	Status code é 200; Situação inativo.	HTTP 200 OK em 15 ms. Testes do Postman: 2/2 PASS.	**Aprovado**
TA-21	[Produtos (CRUD)] DELETE lógico · Inativar de novo (negativo)	Postman · PATCH http://localhost:3000/api/produtos/15/inativar · x-usuario-id: 1	Status code é 409; Código de erro é PRODUTO_JA_INATIVO.	HTTP 409 Conflict em 11 ms. Testes do Postman: 2/2 PASS.	**Aprovado**
TA-22	[Produtos (CRUD)] READ · Listagem padrão oculta inativos (GET)	Postman · GET http://localhost:3000/api/produtos?sku=PMN-155908	Status code é 200; Produto inativo não aparece.	HTTP 200 OK em 7 ms. Testes do Postman: 2/2 PASS.	**Aprovado**
TA-23	[Produtos (CRUD)] UPDATE · Reativar (PATCH)	Postman · PATCH http://localhost:3000/api/produtos/15/reativar · x-usuario-id: 1	Status code é 200; Situação ativo.	HTTP 200 OK em 13 ms. Testes do Postman: 2/2 PASS.	**Aprovado**
TA-24	[Produtos (CRUD)] DELETE · Exclusão física bloqueada (negativo)	Postman · DELETE http://localhost:3000/api/produtos/15 · x-usuario-id: 1	Status code é 409; Código de erro é EXCLUSAO_NAO_PERMITIDA.	HTTP 409 Conflict em 12 ms. Testes do Postman: 2/2 PASS.	**Aprovado**
TA-25	[Categorias (CRUD)] CREATE · Cadastrar categoria (POST)	Postman · POST http://localhost:3000/api/categorias · x-usuario-id: 1 · corpo: { "nome": "Categoria Postman 155908", "categoria_pai_id": null }	Status code é 201; Retorna id.	HTTP 201 Created em 11 ms. Testes do Postman: 2/2 PASS.	**Aprovado**
TA-26	[Categorias (CRUD)] CREATE · Nome duplicado (negativo)	Postman · POST http://localhost:3000/api/categorias · x-usuario-id: 1 · corpo: { "nome": "Categoria Postman 155908" }	Status code é 409; Código de erro é NOME_CATEGORIA_DUPLICADO.	HTTP 409 Conflict em 9 ms. Testes do Postman: 2/2 PASS.	**Aprovado**
TA-27	[Categorias (CRUD)] CREATE · Sem nome (negativo)	Postman · POST http://localhost:3000/api/categorias · x-usuario-id: 1 · corpo: { "nome": "" }	Status code é 422.	HTTP 422 Unprocessable Entity em 8 ms. Testes do Postman: 1/1 PASS.	**Aprovado**
TA-28	[Categorias (CRUD)] READ · Listar em árvore (GET)	Postman · GET http://localhost:3000/api/categorias?formato=arvore	Status code é 200; Categoria criada aparece.	HTTP 200 OK em 25 ms. Testes do Postman: 2/2 PASS.	**Aprovado**
TA-29	[Categorias (CRUD)] UPDATE · Renomear categoria (PUT)	Postman · PUT http://localhost:3000/api/categorias/31 · x-usuario-id: 1 · corpo: { "nome": "Categoria Postman 155908 (editada)" }	Status code é 200; Nome atualizado.	HTTP 200 OK em 12 ms. Testes do Postman: 2/2 PASS.	**Aprovado**
TA-30	[Categorias (CRUD)] UPDATE · Categoria inexistente (negativo)	Postman · PUT http://localhost:3000/api/categorias/999999 · x-usuario-id: 1 · corpo: { "nome": "X 155908" }	Status code é 404; Código de erro é CATEGORIA_NAO_ENCONTRADA.	HTTP 404 Not Found em 9 ms. Testes do Postman: 2/2 PASS.	**Aprovado**

Tipo / ID	O que foi testado?	Dados / passos utilizados	Resultado esperado	Resultado obtido	Status
TA-31	[Unidades de medida (CRUD)] CREATE · Cadastrar unidade (POST)	Postman · POST http://localhost:3000/api/unidades-medida · x-usuario-id: 1 · corpo: { "nome": "Pacote 155908", "sigla": "P155908", "casas_decimais": 0 }	Status code é 201.	HTTP 201 Created em 12 ms. Testes do Postman: 1/1 PASS.	**Aprovado**
TA-32	[Unidades de medida (CRUD)] CREATE · Sigla duplicada (negativo)	Postman · POST http://localhost:3000/api/unidades-medida · x-usuario-id: 1 · corpo: { "nome": "Outra 155908", "sigla": "P155908" }	Status code é 409; Código de erro é SIGLA_UNIDADE_DUPLICADA.	HTTP 409 Conflict em 9 ms. Testes do Postman: 2/2 PASS.	**Aprovado**
TA-33	[Unidades de medida (CRUD)] READ · Listar unidades (GET)	Postman · GET http://localhost:3000/api/unidades-medida	Status code é 200; Lista não vazia.	HTTP 200 OK em 6 ms. Testes do Postman: 2/2 PASS.	**Aprovado**
TA-34	[Unidades de medida (CRUD)] UPDATE · Alterar unidade (PUT)	Postman · PUT http://localhost:3000/api/unidades-medida/15 · x-usuario-id: 1 · corpo: { "descricao": "Atualizada pelo Postman", "casas_decimais": 2 }	Status code é 200; Casas decimais = 2.	HTTP 200 OK em 32 ms. Testes do Postman: 2/2 PASS.	**Aprovado**
TA-35	[Unidades de medida (CRUD)] UPDATE · Casas decimais acima do limite (negativo)	Postman · PUT http://localhost:3000/api/unidades-medida/15 · x-usuario-id: 1 · corpo: { "casas_decimais": 9 }	Status code é 422.	HTTP 422 Unprocessable Entity em 9 ms. Testes do Postman: 1/1 PASS.	**Aprovado**
TA-36	Documentação OpenAPI (Swagger) disponível	Abrir http://localhost:3000/api/docs no navegador	Página de documentação interativa listando os endpoints.	Swagger UI exibido com os endpoints de Produtos, Categorias, Unidades e Autenticação.	**Aprovado**
TB-01	Produto cadastrado pela interface foi gravado corretamente	SELECT id, sku, nome, ean13, categoria_id, unidade_medida_id, preco_custo, preco_venda, estoque, ativo FROM produto WHERE sku = 'NTB-024197'	Uma linha com os valores digitados no TP-01 (custo 2500, venda 3299,90 após edição, estoque 15).	Registro encontrado com os valores esperados (preco_venda=3299.90, preco_custo=2500.00, estoque=15.000).	**Aprovado**
TB-02	Histórico de preço gravado no cadastro e na alteração	SELECT tipo_preco, valor_anterior, valor_novo, motivo, usuario_id FROM produto_preco_historico WHERE produto_id = {id}	3 linhas: venda inicial, custo inicial e alteração de venda (3499,90 → 3299,90).	3 registros de histórico, incluindo valor_anterior=3499.90 e valor_novo=3299.90.	**Aprovado**
TB-03	Senha do usuário armazenada como hash bcrypt	SELECT id, nome, email, left(senha,7) AS prefixo_hash, length(senha) FROM usuario WHERE email = '{email}'	Coluna senha com hash bcrypt (prefixo $2b$10$, 60 caracteres) e não "senha1234".	Senha armazenada como hash bcrypt de 60 caracteres; comparação com texto puro = false.	**Aprovado**
TB-04	Restrição CHECK impede preço de venda menor ou igual a zero	INSERT direto em produto com preco_venda = 0 (dentro de transação com ROLLBACK)	Erro "violates check constraint ck_produto_preco_venda"; nenhuma linha inserida.	Banco rejeitou o INSERT: violates check constraint "ck_produto_preco_venda".	**Aprovado**
TB-05	Restrição UNIQUE impede SKU duplicado	INSERT direto em produto reutilizando o SKU NTB-024197	Erro "duplicate key value violates unique constraint uk_produto_sku".	Banco rejeitou: duplicate key value violates unique constraint "uk_produto_sku".	**Aprovado**
TB-06	Chave estrangeira impede excluir categoria com produtos	DELETE FROM categoria WHERE id = 8 (possui produtos) dentro de transação com ROLLBACK	Erro de violação de chave estrangeira (ON DELETE RESTRICT).	Banco rejeitou o DELETE: violates foreign key constraint "fk_produto_categoria".	**Aprovado**
TB-07	Trigger atualiza "atualizado_em" automaticamente	Comparar criado_em × atualizado_em do produto {id} (editado no TP-02)	atualizado_em posterior a criado_em para um produto editado.	atualizado_em é posterior a criado_em (foi_atualizado = t).	**Aprovado**

Tipo / ID	O que foi testado?	Dados / passos utilizados	Resultado esperado	Resultado obtido	Status
TB-08	Inativação é lógica: registro permanece no banco	Inativar produto {id} (API) e consultar a coluna ativo	Linha continua existindo com ativo = false.	Registro mantido com ativo = f (inativo).	**Aprovado**
TB-09	Integridade referencial: nenhum produto sem categoria/unidade válida	SELECT de órfãos: produtos sem categoria ou unidade correspondente (LEFT JOIN … IS NULL)	Contagem de órfãos = 0 em ambas as relações.	As três consultas retornaram 0.	**Aprovado**
TI-01	Cadastro pela tela → API → banco → listagem	Cadastrar "Mouse Integração 024197" (SKU INT-024197) pela tela; conferir via API e via SELECT	Registro presente na resposta da API, na tabela produto e na listagem da tela.	Produto id 18: exibido na tela, retornado pela API (HTTP 200) e persistido no banco.	**Aprovado**
TI-02	Alteração de preço pela tela reflete no histórico (tela, API e banco)	Editar "Mouse Integração 024197" pela tela: venda 79,90 → 89,90; conferir /precos, API e banco	Histórico atualizado em todas as camadas com valor anterior preservado.	Tela, API (3 registros) e banco concordam: venda 79.90 → 89.90.	**Aprovado**
TI-03	Categoria criada em um módulo aparece no formulário de produto	Categoria "Categoria Ciclo03 024197" (criada no TP-06) deve estar disponível no combo de categoria de /produtos/novo	Opção encontrada ao pesquisar pelo nome.	Combo lista a subcategoria "Sub 024197" criada na tela de categorias.	**Aprovado**
TI-04	Inativação pela tela reflete na API e no banco	Inativar "Mouse Integração 024197" pela listagem; consultar API e banco	API retorna situacao "inativo" e banco registra ativo = false.	API: situacao=inativo · Banco: ativo=f.	**Aprovado**
TI-05	Cadastro de usuário (tela) permite login e gravação de autoria no histórico	Usuário criado no CT-02 cadastra produto via API; histórico deve registrar o usuario_id dele	usuario_id do histórico = id do usuário criado no CT-02.	Login HTTP 200; produto HTTP 201; histórico associado ao usuário 10 (ciclo03.024197@teste.com).	**Aprovado**
TR-01	RN01/RF08 — preço de venda deve ser maior que zero	POST /api/produtos com preco_venda = 0	HTTP 422 VALIDACAO.	HTTP 422.	**Aprovado**
TR-02	RN02 — preço de custo não pode ser negativo	POST /api/produtos com preco_custo = -5	HTTP 422 VALIDACAO.	HTTP 422.	**Aprovado**
TR-03	RN03 — SKU imutável após o cadastro	Tela de edição: campo SKU desabilitado; API PUT com "sku" rejeitada (ver Postman: UPDATE · Tentar alterar SKU)	Campo SKU bloqueado na edição e API rejeita alteração.	Campo SKU exibido desabilitado na edição ("Não pode ser alterado depois de salvo").	**Aprovado**
TR-04	RN04/RF22 — EAN-13 único e com dígito verificador válido	POST /api/produtos reutilizando o EAN-13 7890241972227 do produto criado no TA-09	HTTP 409 EAN13_DUPLICADO.	HTTP 409 — EAN13_DUPLICADO.	**Aprovado**
TR-05	RN05/RF12 — exclusão física desabilitada (usar inativação)	DELETE /api/produtos/:id de produto com histórico	HTTP 409 e mensagem orientando a inativar.	HTTP 409 — Exclusão física desabilitada. Utilize a inativação do produto.	**Aprovado**
TR-06	RN06/RF05 — listagem padrão exibe somente produtos ativos	GET /api/produtos (sem parâmetro de situação) deve excluir o produto inativado no TI-04	Produto inativo (id do TI-04) ausente na listagem padrão.	0 resultado(s) para o produto inativo.	**Aprovado**
TR-07	RN08 — nome único dentro da mesma categoria	Mesmo nome em categorias diferentes é permitido; na mesma categoria é bloqueado (TN-11)	Cadastro com nome repetido em OUTRA categoria retorna 201.	HTTP 201.	**Aprovado**
TR-08	RN09/RF13 — toda alteração de preço gera histórico	Conferido no TB-02 e TI-02 (venda 3499,90→3299,90 e 79,90→89,90)	Histórico com valor anterior e novo, data e usuário.	Alterações registradas com valor_anterior, valor_novo, motivo, usuario_id e data.	**Aprovado**
TR-09	RN14 — senha com no mínimo 8 caracteres	POST /api/auth/cadastro com senha de 5 caracteres	HTTP 422 VALIDACAO.	HTTP 422.	**Aprovado**

Tipo / ID	O que foi testado?	Dados / passos utilizados	Resultado esperado	Resultado obtido	Status
TR-10	RF30 — mensagem de login genérica (não revela se o e-mail existe)	Login com e-mail inexistente × e-mail existente com senha errada	Mesma resposta (HTTP 401 / mesma mensagem) nos dois casos.	Ambos HTTP 401 com a mesma mensagem: "E-mail ou senha inválidos".	**Aprovado**
TR-11	RNF03 — operações de escrita exigem usuário identificado	POST /api/categorias sem x-usuario-id e com id inexistente (99999)	HTTP 401 nos dois casos.	HTTP 401 e HTTP 401.	**Aprovado**
TR-13	RF04 — busca na listagem por nome OU SKU (conforme o placeholder da tela)	Listagem · digitar o SKU NTB-024197 no campo "Buscar por nome ou SKU"; comparar com GET /api/produtos?sku=NTB-024197	Produto encontrado ao buscar pelo SKU, como o próprio campo de busca anuncia.	DEFEITO: a tela retornou "Nenhum resultado para o filtro aplicado" ao buscar pelo SKU, embora a API encontre o produto via ?sku= (1 resultado). O front-end só envia o parâmetro "nome".	**Reprovado**
TR-12	RF32 — sessão do usuário (recarregar a página)	Logar, recarregar /produtos (F5) e verificar o comportamento	Conforme documentado no código: sessão mantida apenas em memória, portanto exige novo login após recarregar.	Após F5 a URL foi /login: sessão perdida, novo login exigido (comportamento intencional documentado — ver observação).	**Aprovado**
TM-01	Editar sem mexer em preço NÃO gera novo histórico	PUT /api/produtos/{id} alterando só a descrição e o estoque; contar histórico antes e depois	Quantidade de registros de histórico permanece igual.	Histórico: 3 → 3 registro(s).	**Aprovado**
TM-02	Alterar preço de venda não altera preço de custo nem outros campos	PUT preco_venda = 99,90 no produto do TI; comparar os demais campos	Custo (30,00), estoque (20) e nome permanecem; histórico ganha só 1 registro do tipo "venda".	preco_custo=30, estoque=20, preco_venda=99.9. Histórico recebeu apenas registros de venda.	**Aprovado**
TM-03	Inativar e reativar preserva dados e histórico	Inativar e reativar o produto do TA-09; comparar produto e histórico	Preços, estoque e quantidade de histórico idênticos após o ciclo.	Preços e estoque idênticos; situacao=ativo; histórico com 3 registros.	**Aprovado**
TM-04	Renomear categoria reflete nos produtos já cadastrados	PUT /api/categorias/{id} renomeando uma categoria que possui produto; conferir o detalhe do produto	Produto exibe o novo nome da categoria, sem perder o vínculo.	Produto passou a exibir "Categoria Áudio 024197"; nome original restaurado (HTTP 200); salvar o próprio nome atual retornou HTTP 200.	**Aprovado**
TM-05	Regressão geral (smoke) após todas as alterações	Reexecutar: health, listagem, detalhe, histórico, categorias, unidades e telas principais	Todos os endpoints de consulta seguem respondendo 200 e a listagem da tela carrega.	200 /api/health · 200 /api/produtos · 200 /api/produtos/13 · 200 /api/produtos/13/precos · 200 /api/categorias · 200 /api/unidades-medida	**Aprovado**

Problemas encontrados

Lista dos problemas identificados durante os testes, indicando em qual funcionalidade ocorreram.

ID	Funcionalidade	Teste(s)	Descrição do problema	Severidade
P-01	Listagem de produtos — campo de busca	TR-13 (reprovado)	O campo de busca anuncia “Buscar por nome ou SKU”, mas digitar um SKU retorna “Nenhum resultado para o filtro aplicado”. Causa: ListagemProdutos.vue (filtroAtivo, linha 55) envia só o parâmetro nome; o repositório (produtos.repo.js, linhas 42–48) filtra nome e SKU em condições separadas (AND), então o SKU nunca é pesquisado pela tela. A API funciona com ?sku= (comprovado no Postman, TA-12).	Média — funcionalidade anunciada na tela não funciona
P-02	Sessão do usuário	TR-12 (aprovado, com observação)	Ao recarregar a página (F5) o usuário é deslogado, porque a sessão fica só em memória (stores/sessao.js). É um comportamento intencional e documentado no código, mas prejudica a experiência de uso.	Baixa — melhoria de usabilidade
P-03	Autenticação da API	TR-11, TA-07	As rotas de escrita só exigem o cabeçalho x-usuario-id com um id existente. Qualquer cliente que saiba um id válido (ex.: 1) consegue gravar dados. O próprio código registra isso como limitação conhecida (P-18: “não é controle de acesso real”).	Média — risco de segurança fora de ambiente acadêmico

Correções realizadas e melhorias recomendadas

- **Correção recomendada para P-01:** Trocar o parâmetro nome por um parâmetro de busca única (ex.: busca) e, no montarFiltro, usar (p.nome ILIKE $n OR p.sku ILIKE $n). Alternativa mínima: no front, enviar o termo também como sku quando ele tiver o formato de SKU (letras maiúsculas, números e hífen). Depois da correção, reexecutar TR-13, CT-03 e TP-08 (teste de impacto).
- **Sessão (P-02):** Guardar o usuário em sessionStorage e revalidar com GET /api/auth/eu no carregamento — o guard de rotas já faz essa revalidação.
- **Autenticação (P-03):** Substituir o cabeçalho x-usuario-id por token assinado (JWT com expiração) emitido no login.
- **Testes automatizados:** Versionar a coleção Postman (docs/postman) e executá-la com Newman em pipeline de CI a cada alteração, servindo como teste de regressão da API.
- **Correções realizadas neste ciclo:** Nenhuma alteração foi feita no código do sistema durante este ciclo: os problemas foram registrados com causa e correção recomendada, para serem corrigidos e retestados no próximo ciclo.

Limitações e testes não executados

- Teste de desempenho com a massa de 5.000 produtos (database/seed/massa-5000.js) não foi executado, para não inserir dados sintéticos no banco de trabalho; os tempos de resposta medidos nos testes de API ficaram abaixo de 1 segundo.
- O Postman desktop não está instalado na máquina de teste: a coleção foi executada pelo Newman (executor oficial do Postman), e os prints reproduzem a requisição, a resposta e o resultado dos testes daquela execução. A mesma coleção pode ser importada no Postman e executada pelo Collection Runner.

- Testes de responsividade (tablet e celular, previstos nos wireframes) e de acessibilidade não fizeram parte deste ciclo.
- Os registros criados pelos testes permanecem no banco e podem ser identificados pelo carimbo de execução presente no nome ou SKU.

Conclusão

O Módulo Produtos atendeu a quase todos os testes executados: os fluxos principais (login, cadastro, edição, inativação, reativação e histórico de preço) funcionam pela interface, a API responde corretamente às operações de CRUD e às entradas inválidas, e o banco de dados garante a integridade por meio de restrições CHECK, UNIQUE e de chave estrangeira.

Foi encontrado um defeito funcional (busca por SKU na tela de listagem), cuja causa foi localizada no código e cuja correção está recomendada acima. As demais observações são melhorias de usabilidade e de segurança